Introduction Générale

Dans un monde où la transformation numérique occupe une place centrale, les systèmes informatiques constituent l'épine dorsale du fonctionnement des organisations modernes. Les universités, entreprises et institutions publiques dépendent fortement de leurs infrastructures numériques pour assurer la continuité de leurs activités, la gestion de leurs données et la prestation de leurs services. Dans ce contexte, la sécurité et la disponibilité des données deviennent des enjeux stratégiques majeurs. Une panne de serveur, une erreur humaine, un sinistre matériel ou une attaque informatique peut entraîner la perte ou la corruption de données essentielles, avec des conséquences parfois irréversibles.

Pour faire face à ces risques, la mise en place d'un système de sauvegarde (backup) et de supervision efficace constitue une exigence incontournable. Un système de backup bien conçu permet de garantir la préservation, la restauration rapide et la continuité des services en cas d'incident. Cependant, les solutions de sauvegarde traditionnelles présentent aujourd'hui des limites importantes : elles sont souvent centralisées, peu automatisées, et surtout vulnérables à certaines menaces modernes telles que les ransomwares, les pannes simultanées ou les erreurs de configuration humaines. Par ailleurs, l'absence d'un mécanisme de supervision en temps réel réduit considérablement la visibilité sur l'état des sauvegardes et la santé du système.

Dans le contexte spécifique des environnements d'enseignement à distance, tels que celui de l'Université Assane Seck de Ziguinchor (UASZ), la problématique est d'autant plus cruciale. En effet, la gestion de plateformes pédagogiques en ligne (telles que Moodle), de serveurs web et de bases de données requiert une haute disponibilité, une sécurité renforcée et une fiabilité sans faille. Une perte de données dans un tel environnement peut compromettre la continuité pédagogique, la crédibilité institutionnelle et la satisfaction des apprenants.

Le présent mémoire s'inscrit dans cette problématique et porte sur le thème :
« Étude et réalisation d'un système de backup et de supervision de serveurs web et bases de données pour un environnement d'enseignement à distance : Cas de l'Université Assane Seck de Ziguinchor ».

L'objectif principal de ce travail est de concevoir et mettre en œuvre une solution complète, sécurisée et automatisée, capable d'assurer à la fois la sauvegarde des données critiques et la supervision proactive des serveurs. La solution proposée repose sur des technologies open source telles que BorgBackup pour la sauvegarde chiffrée et dédupliquée, et Zabbix pour la supervision en temps réel des serveurs. L'architecture envisagée vise également à intégrer des mécanismes de chiffrement, d'authentification multi-facteurs (MFA) et de contrôle d'accès basé sur les rôles (RBAC) afin de renforcer la sécurité globale du système.

Ce travail se justifie par la nécessité de doter l'UASZ d'une infrastructure de sauvegarde et de supervision robuste, performante et adaptée à ses besoins spécifiques, tout en exploitant des solutions open source fiables et économiquement viables.

Pour atteindre ces objectifs, ce mémoire est structuré en quatre chapitres principaux :

- Le premier chapitre, intitulé Contexte général et problématique, présente le cadre de l'étude, la justification du choix du thème ainsi que la méthodologie adoptée.
- Le deuxième chapitre, État de l'art et fondements théoriques, explore les concepts liés à la sauvegarde, à la supervision, aux protocoles de sécurité et aux outils open source pertinents, tout en mettant en évidence les approches existantes et leurs limites.
- Le troisième chapitre, Conception et architecture du système proposé, détaille la modélisation du système, les choix technologiques, la politique de sécurité ainsi que le schéma d'intégration des modules de sauvegarde et de supervision.
- Le quatrième chapitre, Implémentation et validation de la solution, décrit les étapes pratiques de déploiement, les tests effectués, l'analyse des résultats obtenus et les perspectives d'amélioration futures.

Enfin, une conclusion générale viendra clôturer le mémoire en résumant les principaux résultats atteints, en identifiant les limites rencontrées et en ouvrant des pistes pour les travaux futurs, notamment vers l'intégration de l'intelligence artificielle pour la détection prédictive des pannes et la planification intelligente des sauvegardes.

CHAPITRE 1 : CONTEXTE GENERALE ET PROBLEMATIQUE

Contexte générale

La transformation numérique a profondément modifié la manière dont les organisations, publiques comme privées, gèrent leurs activités et interagissent avec leurs usagers. Dans le secteur de l'enseignement supérieur, cette évolution se manifeste par une dépendance accrue aux technologies de l'information et de la communication (TIC). Les universités et établissements de formation intègrent de plus en plus des plateformes numériques pour la gestion pédagogique, la communication, la recherche et la formation à distance.

Cette digitalisation, bien qu'indispensable, introduit de nouveaux défis liés à la sécurité, la fiabilité et la disponibilité des données. Les serveurs hébergeant des applications critiques telles que Moodle, les bases de données MySQL/PostgreSQL, ou encore les serveurs web Apache/Nginx, sont exposés à des risques multiples : pannes matérielles, erreurs humaines, cyberattaques, ou sinistres physiques. Ces menaces peuvent engendrer des interruptions de service et des pertes de données importantes, compromettant la continuité pédagogique et la confiance des utilisateurs.

Dans ce contexte, la mise en place d'un système de sauvegarde et de supervision fiable apparaît comme une nécessité stratégique. Un tel dispositif permet non seulement de garantir la restauration rapide des données en cas d'incident, mais aussi de surveiller en permanence la santé du système afin d'anticiper les défaillances.

À l'Université Assane Seck de Ziguinchor (UASZ), comme dans la plupart des établissements d'enseignement à distance, l'infrastructure numérique joue un rôle central dans la gestion académique et la diffusion des contenus pédagogiques. Cependant, l'absence d'un système intégré de backup et de supervision automatisé limite la résilience et la sécurité de l'environnement numérique, d'où la pertinence de ce travail.

Problématique

La dépendance croissante aux services numériques impose une exigence forte en matière de continuité de service et de protection des données. Dans le cas d'un établissement universitaire, la perte ou la corruption de données peut avoir des conséquences graves : interruption des cours, perte de résultats d'examens, indisponibilité du site institutionnel, voire atteinte à l'image de l'université.
Or, dans la plupart des infrastructures existantes :

- Les sauvegardes ne sont pas systématiques ni automatisées.
- Les données sauvegardées ne sont pas toujours vérifiées ou testées.
- Le stockage des sauvegardes n'est pas chiffré, exposant les données sensibles à des risques de fuite.

- Les solutions de supervision sont absentes ou non intégrées, empêchant la détection proactive des pannes.
- L'absence de gestion d'accès basée sur les rôles (RBAC) et d'authentification multi-facteurs (MFA) expose les systèmes à des intrusions potentielles.

Ces constats posent la question suivante :
Comment concevoir et mettre en œuvre une solution de sauvegarde et de supervision intégrée, automatisée et sécurisée, capable d'assurer la continuité des services web et bases de données dans un environnement d'enseignement à distance tel que celui de l'UASZ ?

Justification du choix du thème

Le choix de ce thème repose sur plusieurs considérations :

- Pertinence institutionnelle : l'UASZ développe de plus en plus de services en ligne (formation à distance, plateforme Moodle, serveurs de ressources numériques). Assurer la disponibilité de ces services est vital pour la réussite du modèle d'enseignement numérique.
- Importance académique et scientifique : le projet s'inscrit dans les préoccupations actuelles de la cybersécurité, de la haute disponibilité et de la gestion intelligente des infrastructures IT.
- Valeur ajoutée pratique : la solution proposée repose sur des outils open source (BorgBackup, Zabbix, MariaDB, etc.), accessibles, robustes et adaptables à des environnements à ressources limitées.
- Motivation personnelle : ce travail représente une opportunité de mettre en pratique les connaissances acquises en administration système, en sécurité réseau et en gestion des infrastructures, dans un projet à fort impact réel pour l'université.

Méthodologie de travail

La méthodologie adoptée repose sur une démarche expérimentale et itérative, structurée en plusieurs étapes :

- ◆ Analyse des besoins et de l'existant : identification des serveurs, des données critiques et des risques.
- ◆ Étude comparative des solutions open source (Borg, Duplicity, Restic, Zabbix, Nagios, etc.).
- ◆ Conception de l'architecture globale : définition des flux, des rôles et des politiques de sécurité.
- ◆ Implémentation pratique : installation, configuration et tests des différents composants sur deux serveurs (Moodle et Backup).
- ◆ Validation : tests de sauvegarde, de restauration et de supervision.
- ◆ Évaluation : mesure des performances, de la sécurité et de la fiabilité du système.

Documentation et rédaction : rapport détaillé et recommandations pour l'amélioration continue.

Conclusion du Chapitre :

En somme, l'analyse du contexte et la définition de la problématique mettent en évidence la nécessité de concevoir une solution intégrée de sauvegarde et de supervision adaptée aux exigences de l'enseignement à distance. Le chapitre suivant présente les concepts théoriques, les solutions existantes ainsi que leurs limites.

CHAPITRE 2 : ETAT DE L'ART ET FONDEMENTS TH ÉORIQUES

2.1 Introduction

La conception d'un système de backup et de supervision pour des serveurs web et bases de données nécessite une compréhension approfondie des concepts fondamentaux, des mécanismes techniques et des outils existants. Les environnements d'enseignement à distance, tels que celui de l'Université Assane Seck de Ziguinchor, reposent sur des plateformes web critiques comme Moodle, des bases de données volumineuses et des services réseau constamment sollicités. Afin d'assurer leur disponibilité, leur sécurité et leur résilience, il est indispensable de s'appuyer sur des concepts éprouvés en matière de sauvegarde, de supervision et de gestion des infrastructures informatiques.
Ce chapitre présente l'état de l'art des pratiques actuelles en matière de sauvegarde des données, de surveillance des infrastructures, des mécanismes de sécurité modernes, ainsi qu'un panorama des outils open source utilisés dans le cadre de ce projet. L'objectif est de fournir une base théorique solide permettant de justifier les choix technologiques adoptés dans les chapitres suivants.

2.2 Fondements de la sauvegarde des données

2.2.1 Définition de la sauvegarde

La **sauvegarde**, ou *backup*, désigne l'ensemble des techniques et procédures permettant de réaliser une copie des données vers un emplacement distinct du système principal. L'objectif est de garantir la possibilité de restaurer ces données en cas d'incident, de perte, de corruption ou d'indisponibilité du système source.

Il s'agit d'un mécanisme essentiel de la gestion de la continuité d'activité, car il permet de préserver :

- **La continuité de service** : en assurant une reprise rapide des activités après un incident.
- **L'intégrité et la disponibilité des données** : en protégeant les informations contre les altérations involontaires ou malveillantes.
- **La résilience du système** : en offrant une parade efficace face aux pannes matérielles, erreurs humaines, sinistres ou cyberattaques (notamment ransomwares).
- **La conformité institutionnelle et réglementaire** : en répondant aux exigences de protection des données et de gestion documentaire imposées aux établissements publics et universitaires.

Ainsi, la sauvegarde n'est pas uniquement un mécanisme technique, mais un **processus stratégique** visant à garantir la fiabilité et la pérennité du patrimoine informationnel d'une organisation.

2.2.2 Différence entre sauvegarde et archivage

Bien que souvent confondus, **la sauvegarde** et **l'archivage** répondent à des objectifs différents au sein d'un système d'information. Leur distinction est essentielle pour comprendre la stratégie globale de gestion, de protection et de conservation des données.

Sauvegarde (Backup)

La sauvegarde consiste à créer une copie opérationnelle des données afin de permettre leur restauration rapide en cas d'incident.
Elle a pour finalité principale la protection contre la perte de données qu'elle soit due à une panne matérielle, une suppression accidentelle, une corruption du système, une cyberattaque (Exemple : ransomware), ou tout autre événement compromettant le système en production.

Caractéristiques clés de la sauvegarde :

- **Objectif :** Restaurer le système ou les fichiers tels qu'ils étaient avant l'incident.
- **Horizon temporel :** Court terme (jours ou semaines).
- **Fréquence :** Quotidienne, horaire, voire en continu.
- **Nature des données :** Données actives utilisées dans les opérations.
- **Durée de conservation :** Limitée selon une politique de rétention (exemple : 7 jours, 4 semaines…).
- **Exemple :** Sauvegarde quotidienne de la base de données Moodle pour restaurer les cours après une panne.

Archivage (archive)

L'archivage, quant à lui, vise à conserver durablement des données qui ne sont plus activement utilisées. Mais qui doivent être préservées pour des raisons historiques, réglementaires, administratives, ou patrimoniales.
Il s'agit d'un stockage à long terme, souvent réalisé sur un support optimisé pour la conservation plutôt que pour la vitesse d'accès.

Caractéristiques clés de l'archivage :

- **Objectif :** Conserver des données à long terme et en assurer la traçabilité.
- **Horizon temporel :** Long terme (années ou décennies).
- **Fréquence :** Occasionnelle ou selon les besoins.
- **Nature des données :** Données inactives, dossiers clôturés, historiques, logs institutionnels, etc.
- **Durée de conservation :** Très longue, parfois imposée par des normes ou lois.
- **Exemple :** Conservation des archives administratives d'une université pendant 10 ans.

En résumé :

- **La sauvegarde assure la disponibilité et la sécurité opérationnelle**,
- **L'archivage assure la conservation historique et réglementaire**.

Les deux sont complémentaires mais ne répondent pas aux mêmes exigences techniques ni réglementaires.

2.2.3 Types de sauvegardes

Les méthodes de sauvegarde peuvent être classées selon la quantité de données copiées et la stratégie de conservation adoptée. Les principaux types sont :

a. Sauvegarde Complète (Full backup)

Elle consiste à copier **l'intégralité des données** à chaque exécution.
Avantages : Restauration rapide et simple, vision complète du système à un instant donné.
Inconvénients : Très consommatrice en espace disque, durée d'exécution élevée.

b. Sauvegarde Incrémentielle

Elle copie uniquement les fichiers **modifiés depuis la dernière sauvegarde** (qu'elle soit complète ou incrémentielle).
Avantages : peu d'espace de stockage, très rapide à exécuter.
Inconvénients : restauration plus complexe, car elle nécessite la dernière sauvegarde complète + toutes les incrémentielles suivantes.

c. Sauvegarde Différentielle

Elle copie les fichiers **modifiés depuis la dernière sauvegarde complète**.
Avantages : restauration plus simple que l'incrémentielle, compromis entre rapidité et stockage.
Inconvénients : taille des sauvegardes intermédiaires augmente progressivement.

2.2.4 Stratégies de sauvegarde

La mise en place d'un système de sauvegarde efficace ne se limite pas au choix d'un outil ; elle repose également sur l'adoption d'une stratégie adaptée aux risques, aux contraintes et aux objectifs de continuité de service de l'organisation. Plusieurs approches reconnues permettent d'optimiser la résilience et la disponibilité des données.

a. La règle 3-2-1

La règle **3-2-1** est un principe fondamental en matière de protection des données, largement adopté dans les environnements critiques (universités, banques, hôpitaux,

administrations). Elle consiste à :

- **3 copies des données** : une copie de production et deux copies de sauvegarde.
- **2 types de supports différents** : par exemple disque local + NAS, ou disque + cloud.
- **1 copie hors site** : stockée à l'extérieur du lieu principal (cloud, datacenter distant, serveur off-site).

Cette stratégie garantit une capacité de récupération même en cas de sinistre majeur (incendie, vol, panne totale, ransomware).
Elle constitue la base de la résilience moderne des systèmes d'information.

b. Sauvegarde Locale vs Sauvegarde distante

Les solutions de sauvegarde peuvent être classées en fonction de leur emplacement :

- **La Sauvegarde Locale**

Elle s'effectue sur un support disponible dans le même site que le serveur : disque dur externe, NAS interne, deuxième serveur local.
Avantages : rapidité d'exécution, coût réduit, restauration rapide
Limites : vulnérabilité face aux sinistres locaux : panne électrique, incendie, vol, ransomware.

- **La Sauvegarde distante**

La sauvegarde distante consiste à copier les données vers serveur hors site, datacenter distant ou espace cloud chiffré.
Avantages : Protection contre les catastrophes locales, meilleures tolérances en pannes, possibilités d'intégration avec politiques de continuité d'activités.
Limites : dépendance à la bande passante, coût potentiellement plus élevé (cloud).

L'architecture robuste combine souvent **les deux approches** : une copie locale pour la rapidité, et une copie distante pour la sécurité.

c. La Sauvegarde automatisée

Dans les environnements modernes, la sauvegarde doit être **automatisée**, afin de réduire les risques liés à l'intervention humaine et de garantir une régularité parfaite.
Une sauvegarde automatisée implique :

- **Une planification** via des tâches programmées (comme cron),
- **Une exécution sans intervention humaine**,
- **Des vérifications automatiques** (tests d'intégrité),
- **Une supervision en temps réel** via un outil comme Zabbix.

L'automatisation améliore la fiabilité du système et permet d'assurer que les sauvegardes s'exécutent à l'heure prévue, les erreurs sont immédiatement détectées, la restauration est possible à tout moment.
Cette approche est particulièrement essentielle dans un environnement d'enseignement à distance, où la disponibilité des plateformes pédagogiques doit être

continue.

2.2.5 Limites des systèmes de sauvegarde traditionnels

Les solutions de sauvegarde classiques, longtemps utilisées dans les infrastructures informatiques, montrent aujourd'hui plusieurs limites face aux exigences modernes de performance, de sécurité et d'automatisation. Ces limitations deviennent particulièrement critiques dans les environnements sensibles tels que les plateformes d'enseignement à distance, où la disponibilité et l'intégrité des données sont essentielles.

Parmi les principales faiblesses observées, on peut citer :

- ✓ **Absence de chiffrement natif**

La plupart des outils traditionnels ne proposent pas de mécanisme de chiffrement intégré. Les données sauvegardées peuvent donc être accessibles en clair, exposant les organisations à des risques de fuite, d'espionnage ou de compromission en cas d'accès non autorisé au support de sauvegarde.

- ✓ **Duplication excessive et non optimisée des données**

Les anciennes méthodes reposent sur une copie brute et complète des fichiers, même lorsque ceux-ci ont peu ou pas changé. Cela entraîne une consommation importante d'espace disque, une optimisation insuffisante des ressources de stockage et des coûts accrus en cas d'extension.

- ✓ **Lenteur lors de la gestion de gros volumes**

Le traitement de données volumineuses ou de bases de données complexes devient rapidement inefficace. Les sauvegardes complètes répétées, souvent nécessaires, rallongent considérablement les temps d'exécution et peuvent entraîner une surcharge du système principal.

- ✓ **Manque de supervision intégrée**

Les systèmes traditionnels n'incluent généralement pas de mécanisme de supervision ou de rapport automatisé. En cas d'erreur silencieuse ou d'échec de sauvegarde non détecté, les administrateurs peuvent découvrir le problème seulement lors d'un incident nécessitant une restauration.

- ✓ **Risques élevés d'erreurs humaines ou de mauvaise configuration**

L’absence de contrôle automatisé et la dépendance à des scripts manuels augmentent la probabilité de mauvaises manipulations, de configurations incohérentes ou d’oublis dans l’exécution des sauvegardes.

Face à ces contraintes, l’industrie s’est progressivement tournée vers des solutions modernes comme **BorgBackup, Restic, Duplicity ou Veeam**. Ces outils sont spécialement conçus pour répondre aux besoins actuels : performances améliorées, chiffrement natif, déduplication intelligente, automatisation avancée et intégration avec des systèmes de supervision tels que Zabbix.

Ils permettent ainsi aux infrastructures critiques comme les environnements d’enseignement à distance de bénéficier d’un niveau de fiabilité et de sécurité adapté aux enjeux contemporains.

2.3 Les fondements de la supervision

La supervision constitue un pilier essentiel dans la gestion moderne des infrastructures informatiques. Elle regroupe l’ensemble des mécanismes permettant d’observer, analyser et contrôler en continu l’état d’un système informatique afin d’assurer sa disponibilité, sa performance et sa sécurité. Dans un contexte où les services numériques deviennent critiques notamment dans les environnements d’enseignement à distance, disposer d’une supervision fiable et automatisée est indispensable pour garantir la continuité pédagogique et prévenir toute interruption des services.

2.3.1 Définition de la supervision

La supervision, dans le domaine des systèmes d’information, désigne l’ensemble des processus et outils permettant de :

- Surveiller en temps réel l’état des serveurs, réseaux, applications et services ;
- Collecter des métriques (CPU, mémoire, espace disque, disponibilité, latence, erreurs) ;
- Détecter automatiquement les anomalies ou incidents ;
- Alerter les administrateurs via des notifications ;
- Analyser l’historique des performances pour faciliter la prise de décision.

Elle s’appuie sur des agents logiciels, des sondes réseau, des protocoles comme SNMP, ICMP ou SSH, ainsi que des serveurs centralisés de collecte tels que Zabbix ou Nagios.

La supervision vise donc à **prévenir plutôt que guérir**, en anticipant les pannes possibles avant qu’elles n’affectent les utilisateurs.

2.3.2 Objectifs de la supervision

La mise en place d'un système de supervision poursuit plusieurs objectifs majeurs :

1. **Assurer la disponibilité des services**

En détectant rapidement les pannes (serveur indisponible, service arrêté, saturation des ressources).

2. **Garantir la performance**

En mesurant la charge système, les temps de réponse, l'utilisation des disques et bande passante.

3. **Renforcer la sécurité**

Par la détection d'activités anormales (tentatives de connexion, pics inhabituels de trafic).

4. **Automatiser la surveillance**

Réduisant la dépendance aux interventions humaines grâce aux alertes et tableaux de bord.

5. **Disposer d'un historique précis**

Utile pour la planification de capacité, l'audit, la résolution d'incidents et l'optimisation.

2.3.3. Les types de supervision

On distingue généralement trois grandes catégories de supervision :

a) Supervision d'infrastructure

Elle concerne les composants matériels et système :

- Processeur (CPU),
- Mémoire RAM,
- Utilisation des disques,
- État du réseau,
- Disponibilité du serveur.

Elle permet de vérifier la santé physique et logique des équipements.

b) Supervision applicative

Elle porte sur les services et logiciels :

- Disponibilité du service web (HTTP),
- Fonctionnement d'une base de données (MySQL/MariaDB),
- État d'un service spécifique (Moodle, Apache, Nginx),
- Vérification du certificat SSL, du nom de domaine, etc.

Elle garantit que les applications critiques fonctionnent correctement.

c) Supervision transactionnelle

Elle simule des actions réelles pour tester l'expérience utilisateur :

- Ouverture d'une page Moodle,
- Connexion d'un étudiant,

- Recherche dans la plateforme, etc.

Ce type de supervision est souvent utilisé dans les environnements pédagogiques pour assurer la fluidité des services.

2.3.4 La supervision proactive vs réactive

- **Supervision réactive :** traite les incidents après leur apparition.
  Exemple : alerte lorsque le serveur Moodle est déjà indisponible.
- **Supervision proactive :** anticipe les incidents en détectant leur précurseur.
  Exemple : alerte quand la charge CPU dépasse 90% ou quand le disque atteint 80% d'utilisation.

Dans les environnements critiques comme l'enseignement à distance, la supervision proactive est vivement recommandée.

2.3.5 Supervision centralisée

Les systèmes modernes s'appuient souvent sur une architecture centralisée dans laquelle :

- Chaque serveur (Moodle, base de données, backup…) possède un agent de monitoring,
- Toutes les données de supervision sont envoyées vers un serveur central,
- L'analyse, la visualisation et les alertes sont gérées depuis une console unique.

Des outils comme **Zabbix**, **Centreon**, **Nagios**, **Prometheus** ou **Grafana** permettent cette centralisation. Cette approche facilite la corrélation des incidents, le diagnostic global, la gestion unifiée de l'infrastructure et l'évolutivité du système.

2.3.6 Rôle de la supervision dans un environnement d'enseignement à distance

Dans une université comme l'UASZ, les services numériques soutiennent la pédagogie et l'administration. La supervision joue un rôle critique dans la disponibilité de la plateforme MOODLE, la fiabilité des bases de données académiques, la surveillance des serveurs web institutionnelles, la vérification des sauvegardes (fréquence, taille, succès ou échec), la détection d'incidents pouvant perturber les cours en ligne.

Ainsi, la supervision devient un instrument stratégique pour assurer la qualité du service d'enseignement à distance et renforcer la confiance des utilisateurs.

2.4 Fondements de la sécurité informatique

La sécurité informatique constitue un pilier fondamental de toute infrastructure numérique moderne, en particulier dans les environnements critiques tels que les universités et les plateformes d'enseignement à distance. Elle vise à protéger les systèmes, les réseaux et les données contre les accès non autorisés, les altérations, les

pertes et les interruptions de service.

Dans le cadre d'un système de sauvegarde et de supervision, la sécurité ne se limite pas à la protection des serveurs de production, mais s'étend également aux mécanismes de sauvegarde, aux canaux de transmission des données et aux outils de supervision. Les principaux objectifs de la sécurité informatique reposent sur :

- **Confidentialité** : garantir que seules les personnes autorisées accèdent aux données ;
- **Intégrité** : assurer que les données ne sont pas modifiées de manière non autorisée ;
- **Disponibilité** : maintenir l'accès aux services et aux données en toutes circonstances.

Dans ce contexte, plusieurs mécanismes de sécurité sont essentiels, notamment le chiffrement des données, l'authentification multi-facteurs, le contrôle d'accès basé sur les rôles et la sécurisation du réseau.

2.4.1 Chiffrement des données

Le chiffrement est une technique fondamentale de protection des données qui consiste à transformer les informations lisibles en un format illisible à l'aide d'algorithmes cryptographiques. Seules les entités disposant de la clé de déchiffrement peuvent accéder au contenu original.

Dans un système de sauvegarde, le chiffrement joue un rôle crucial à deux niveaux :

- **Chiffrement des données au repos** : les sauvegardes stockées sur un serveur distant ou dans le cloud doivent être chiffrées afin d'éviter toute exploitation en cas de vol ou de compromission du support de stockage.
- **Chiffrement des données en transit** : lors du transfert des sauvegardes entre le serveur de production et le serveur de backup, les données doivent être protégées contre les interceptions (via SSH, TLS, etc.).

Des outils comme **BorgBackup** intègrent nativement le chiffrement fort (AES-256), garantissant que même l'administrateur du serveur de sauvegarde ne peut lire les données sans la clé secrète. Cette approche est particulièrement adaptée aux environnements universitaires où les données académiques et personnelles doivent être strictement protégées.

2.4.2 Authentification multi-facteurs (MFA)

L'authentification multi-facteurs (Multi-Factor Authentication – MFA) renforce la sécurité des accès en exigeant la combinaison d'au moins deux facteurs distincts

parmi :

- Quelque chose que l'utilisateur **connaît** (mot de passe) ;
- Quelque chose que l'utilisateur **possède** (token, téléphone, clé SSH) ;
- Quelque chose que l'utilisateur **est** (empreinte biométrique).

Dans le cadre d'un système de backup et de supervision, la mise en œuvre du MFA permet de réduire considérablement les risques liés au vol de mots de passe ou aux attaques par force brute. Par exemple l'accès aux serveurs de sauvegarde peut être sécurisé par des clés SSH associées à un mot de passe ; l'accès à l'interface de supervision (Zabbix) peut être protégé par un second facteur d'authentification.

Cette approche est essentielle dans un environnement d'enseignement à distance, où les accès administratifs peuvent être effectués à distance et sont donc plus exposés aux attaques.

2.4.3 Contrôle d'accès basé sur les rôles (RBAC)

Le contrôle d'accès basé sur les rôles (Role-Based Access Control – **RBAC**) consiste à attribuer des permissions aux utilisateurs en fonction de leur rôle plutôt que de leur identité individuelle. Chaque utilisateur se voit accorder uniquement les droits nécessaires à l'accomplissement de ses tâches, selon le principe du moindre privilège.

Dans une infrastructure universitaire, le **RBAC** permet de distinguer clairement les administrateurs système, les administrateurs de bases de données, les responsables pédagogiques, et les utilisateurs finaux.

Appliqué aux systèmes de sauvegarde et de supervision, le RBAC permet par exemple de limiter l'accès aux configurations critiques de BorgBackup,
de restreindre les actions possibles dans Zabbix (lecture seule, administration, maintenance), ou de réduire les risques d'erreurs humaines ou de manipulations malveillantes.

Cette séparation des rôles améliore la sécurité globale du système tout en facilitant sa gestion.

2.4.4 Sécurisation réseau

La sécurisation du réseau vise à protéger les échanges de données et les services contre les attaques externes et internes. Elle repose sur un ensemble de mécanismes complémentaires tels que :

- Les pare-feu (firewalls) pour filtrer le trafic réseau ;
- La segmentation du réseau (VLAN, sous-réseaux) afin d'isoler les serveurs

critiques ;
- L'utilisation de protocoles sécurisés (SSH, HTTPS, TLS) ;
- La limitation des ports et services exposés.

Dans le cadre du système proposé, les serveurs web, bases de données, sauvegarde et supervision doivent être correctement isolés et protégés. Par exemple :

- Le serveur de sauvegarde ne doit accepter que les connexions SSH provenant des serveurs autorisés ;
- Les interfaces d'administration doivent être accessibles uniquement depuis le réseau interne de l'université ;
- Les flux de supervision doivent être chiffrés et contrôlés.

Une bonne sécurisation réseau contribue non seulement à prévenir les intrusions, mais aussi à renforcer la fiabilité et la disponibilité des services d'enseignement à distance.

2.5 Présentation des outils open source

Dans le cadre de la mise en place d'un système de sauvegarde et de supervision fiable et sécurisé pour un environnement d'enseignement à distance, le choix des outils logiciels constitue une étape déterminante. Les solutions open source offrent de nombreux avantages, notamment la transparence du code, la flexibilité, l'absence de coûts de licence et une large communauté de support. Cette section présente les principaux outils étudiés, ceux retenus pour la réalisation du système, ainsi que les alternatives analysées.

2.5.1 BorgBackup

BorgBackup est un outil open source de sauvegarde avancé, conçu pour répondre aux exigences modernes en matière de sécurité, de performance et d'optimisation du stockage. Il repose sur plusieurs mécanismes innovants qui en font une solution particulièrement adaptée aux environnements critiques.

L'une des caractéristiques majeures de BorgBackup est la **déduplication des données**. Celle-ci permet d'éliminer les blocs de données redondants entre les différentes sauvegardes, réduisant considérablement l'espace disque utilisé. Ce mécanisme est particulièrement efficace pour les sauvegardes fréquentes de serveurs web et de bases de données, dont les changements sont souvent partiels.

BorgBackup intègre également un **chiffrement fort côté client** (AES-256), garantissant la confidentialité des données avant leur transfert vers le serveur de sauvegarde. Ainsi, même en cas de compromission du serveur distant, les données sauvegardées restent illisibles sans la clé de chiffrement.

Par ailleurs, Borg propose une **gestion intelligente des versions** grâce à des politiques de rétention (journalière, hebdomadaire, mensuelle), permettant de conserver un historique des sauvegardes tout en maîtrisant l'espace de stockage. Il supporte également les sauvegardes distantes via SSH, renforçant la sécurité des communications.

Ces caractéristiques font de BorgBackup un choix pertinent pour la sauvegarde des serveurs Moodle, web et bases de données dans le cadre de l'UASZ.

2.5.2 Alternatives étudiées

Avant le choix définitif de BorgBackup, plusieurs autres solutions open source ont été étudiées :

- **Restic** : outil moderne de sauvegarde chiffrée, simple à utiliser et compatible avec de nombreux backends (S3, Azure, SSH). Toutefois, sa déduplication est moins flexible dans certains scénarios complexes.
- **Duplicity** : solution de sauvegarde incrémentielle utilisant GPG pour le chiffrement. Bien que robuste, elle présente des performances limitées pour de grands volumes de données.
- **Rsync** : outil largement utilisé pour la synchronisation de fichiers. Cependant, il ne propose ni chiffrement natif, ni gestion avancée des versions.

Ces outils offrent des fonctionnalités intéressantes, mais présentent certaines limites par rapport aux exigences de performance, de sécurité et d'automatisation du projet.

2.5.3 Zabbix : système de supervision retenu

Zabbix est une plateforme open source de supervision complète, largement utilisée pour la surveillance des infrastructures informatiques. Elle permet de superviser en temps réel les serveurs, services, applications et équipements réseau.

Zabbix se distingue par sa capacité à collecter des métriques détaillées telles que : l'utilisation du CPU, de la mémoire et du disque, l'état des services web et bases de données, la disponibilité des serveurs, le suivi des tâches de sauvegarde (état, durée, succès ou échec).

Grâce à son système d'alertes configurable, Zabbix permet une **détection proactive des incidents**, en envoyant des notifications en cas d'anomalie (échec de sauvegarde, saturation disque, indisponibilité d'un service).

Un autre avantage majeur de Zabbix est son **extensibilité**, notamment via les UserParameters, qui permettent de superviser des éléments personnalisés comme les dépôts BorgBackup (taille, nombre d'archives, intégrité). Cette flexibilité est

particulièrement utile dans le cadre du projet.

Enfin, Zabbix offre une interface web intuitive, des tableaux de bord graphiques et une gestion des accès par rôles, renforçant la sécurité et l'exploitation du système.

2.5.4 Alternatives non retenues

Plusieurs autres solutions de supervision ont été analysées, mais n'ont pas été retenues pour la réalisation du système :

- **Nagios** : outil historique de supervision, robuste mais nécessitant une configuration complexe et peu intuitive.
- **Centreon** : solution puissante basée sur Nagios, mais plus lourde à administrer et parfois dépendante de composants commerciaux.
- **Prometheus** : très performant pour la supervision orientée métriques, mais moins adapté à la supervision complète de services hétérogènes sans outils complémentaires.

Au regard des besoins du projet simplicité, extensibilité, supervision des sauvegardes et intégration avec BorgBackup **Zabbix** s'est imposé comme la solution la plus adaptée.

2.5.5 Choix de l'outil rclone

rclone est un outil en ligne de commande largement utilisé pour la synchronisation et la gestion de fichiers entre des systèmes locaux et des services de stockage cloud. Il présente plusieurs avantages :

- Compatibilité avec de nombreux fournisseurs cloud,
- Support du chiffrement côté client,
- Synchronisation incrémentielle efficace,
- Intégration facile dans des scripts automatisés.

Dans le cadre de ce projet, **rclone** est utilisé avec un **remote chiffré**, garantissant que les données stockées dans le cloud ne sont jamais accessibles en clair.

2.6 Environnements d'enseignement à distance et enjeux numériques

L'enseignement à distance (EAD) repose fortement sur les technologies numériques pour assurer la continuité pédagogique en dehors des cadres traditionnels de formation présentielle. Il s'appuie sur des plateformes en ligne, des serveurs web, des bases de données et des services réseau qui doivent fonctionner de manière fiable, sécurisée et continue. Dans ce contexte, la disponibilité et la protection des données constituent des enjeux majeurs pour les établissements d'enseignement supérieur.

À l'Université Assane Seck de Ziguinchor (UASZ), comme dans de nombreuses universités africaines et internationales, l'EAD occupe une place de plus en plus stratégique, notamment à travers l'utilisation de plateformes pédagogiques telles que Moodle, des portails web institutionnels et des systèmes de gestion académique. Cette dépendance accrue au numérique rend indispensable la mise en place de mécanismes robustes de sauvegarde et de supervision afin de garantir la qualité et la pérennité des services offerts aux apprenants et au personnel académique.

2.6.1 Exigences spécifiques des environnements d'enseignement à distance

Les environnements d'enseignement à distance présentent des exigences particulières qui les distinguent des systèmes informatiques classiques :

- **Haute disponibilité des services** :
  Les plateformes pédagogiques doivent être accessibles en permanence, souvent 24h/24 et 7j/7, afin de permettre aux étudiants de consulter les cours, soumettre des devoirs et passer des évaluations en ligne sans interruption.
- **Intégrité et fiabilité des données pédagogiques** :
  Les données telles que les contenus de cours, les résultats d'examens, les travaux des étudiants et les informations administratives doivent être protégées contre toute perte, modification non autorisée ou corruption
- **Scalabilité et performance** :
  Les systèmes doivent pouvoir supporter un nombre important d'utilisateurs simultanés, notamment lors des périodes critiques comme les examens, les inscriptions ou les soutenances en ligne.
- **Sécurité renforcée** :
  Les plateformes EAD manipulent des données sensibles (données personnelles, résultats académiques, identifiants de connexion). Elles doivent donc intégrer des mécanismes de chiffrement, d'authentification forte et de contrôle d'accès.
- **Automatisation et fiabilité opérationnelle** :
  Les opérations de sauvegarde et de supervision doivent être automatisées afin de réduire la dépendance à l'intervention humaine et minimiser les risques d'erreurs.

Ces exigences imposent l'adoption de solutions techniques modernes, fiables et adaptées aux contraintes budgétaires et techniques des universités.

2.6.2 Risques identifiés dans les environnements d'enseignement à distance

Malgré leurs avantages, les environnements d'enseignement à distance sont exposés à de nombreux risques, qui peuvent compromettre la continuité pédagogique et la crédibilité institutionnelle :

- **Pertes de données** :

Une panne matérielle, une erreur humaine ou un défaut de configuration peut entraîner la perte de bases de données critiques, rendant impossible la récupération des informations pédagogiques.

- **Cyberattaques** :
  Les plateformes EAD sont des cibles privilégiées pour les attaques par ransomware, les intrusions, les attaques par déni de services distribués(DDoS), ou les tentatives de vol de données.
- **Indisponibilité des services** :
  Une absence de supervision proactive peut empêcher la détection précoce des anomalies (surcharge CPU, manque d'espace disque, défaillance réseau), conduisant à des interruptions prolongées de service.
- **Absence ou faiblesse des politiques de sauvegarde** :
  Des sauvegardes non chiffrées, non testées ou stockées sur un seul site exposent l'université à des risques majeurs en cas de sinistre.
- **Impact pédagogique et institutionnel** :
  L'indisponibilité des plateformes peut entrainer des retards académiques, une perte de confiance des étudiants et enseignants, et nuire à l'image de l'établissement.

Face à ces risques, il devient impératif de mettre en place un **système intégré de backup et de supervision**, capable non seulement de restaurer rapidement les données, mais aussi d'anticiper les incidents grâce à une surveillance continue et intelligente.

2.7 Synthèse du chapitre

Ce chapitre a permis de poser les bases théoriques et conceptuelles nécessaires à la compréhension et à la mise en œuvre d'un système de backup et de supervision adapté à un environnement d'enseignement à distance.

Dans un premier temps, les concepts fondamentaux liés à la sauvegarde des données ont été présentés, en mettant en évidence les différents types de sauvegarde, les stratégies recommandées telles que la règle 3-2-1, ainsi que les limites des solutions traditionnelles. Cette analyse a montré que les approches classiques ne répondent plus pleinement aux exigences actuelles en matière de sécurité, de performance et de résilience, notamment face aux menaces modernes comme les ransomwares ou les pannes critiques.

Dans un second temps, les fondements de la supervision ont été abordés. La supervision apparaît comme un élément complémentaire et indispensable à la sauvegarde, car elle permet une surveillance proactive des serveurs, des services et des performances du système. Elle offre une visibilité en temps réel sur l'état des infrastructures et facilite la détection rapide des anomalies avant qu'elles n'impactent les utilisateurs finaux.

Le chapitre a également mis en évidence l'importance des mécanismes de sécurité informatique, notamment le chiffrement des données, l'authentification multi-facteurs (MFA), le contrôle d'accès basé sur les rôles (RBAC) et la sécurisation réseau. Ces mécanismes constituent des piliers essentiels pour garantir la confidentialité, l'intégrité et la disponibilité des données sensibles dans un contexte universitaire.

Par ailleurs, une étude des outils open source a été réalisée, mettant en avant BorgBackup comme solution de sauvegarde retenue pour ses capacités de chiffrement, de déduplication et d'automatisation, ainsi que Zabbix pour la supervision grâce à sa flexibilité, sa richesse fonctionnelle et son intégration aisée dans des environnements hétérogènes. Les alternatives existantes ont également été analysées afin de justifier les choix technologiques effectués.
Enfin, les spécificités des environnements d'enseignement à distance ont été examinées, en soulignant leurs exigences élevées en matière de disponibilité, de sécurité et de continuité de service, ainsi que les risques liés à la perte de données et à l'indisponibilité des plateformes pédagogiques.

L'ensemble de ces éléments théoriques constitue un socle solide pour la suite de ce travail. Le chapitre suivant sera consacré à la **conception et à l'architecture du système proposé**, dans lequel seront détaillés les choix techniques, l'organisation des composants et les mécanismes d'intégration de la sauvegarde et de la supervision au sein de l'infrastructure de l'Université Assane Seck de Ziguinchor.

CHAPITRE 3 : CONCEPTION ET ARCHITECTURE DU SYSTÈME PROPOSÉ

Introduction du chapitre

Après avoir présenté, dans le chapitre précédent, les fondements théoriques liés à la sauvegarde, à la supervision et à la sécurité informatique, ce chapitre est consacré à la **conception globale du système proposé**.

Il vise à décrire l'architecture retenue, les choix technologiques effectués ainsi que les mécanismes de sécurité mis en place pour assurer une sauvegarde fiable et une supervision efficace des serveurs web et bases de données de l'environnement d'enseignement à distance de l'Université Assane Seck de Ziguinchor (UASZ).

La conception du système repose sur une approche modulaire, sécurisée et évolutive, intégrant à la fois des mécanismes de sauvegarde automatisée et une supervision en temps réel.

3.1 Analyse des besoins et contraintes

La conception d'un système de backup et de supervision efficace repose avant tout sur une analyse rigoureuse des besoins fonctionnels et des contraintes techniques de l'environnement cible. Cette étape permet de définir une solution réaliste, sécurisée et adaptée aux ressources disponibles, tout en répondant aux exigences de continuité de service propres à un environnement d'enseignement à distance.

3.1.1 Besoins fonctionnels

Les besoins fonctionnels expriment ce que le système doit impérativement accomplir pour répondre aux attentes de l'Université Assane Seck de Ziguinchor dans le cadre de la gestion de ses serveurs web et bases de données.

a) Sauvegarde des données critiques

Le système doit assurer la sauvegarde régulière et fiable des éléments suivants :

- Les fichiers des serveurs web (sites institutionnels, plateforme Moodle).
- Les bases de données associées (MySQL/MariaDB ou PostgreSQL).
- Les répertoires de données applicatives sensibles (moodledata, fichiers utilisateurs, ressources pédagogiques).

b) Automatisation complète des sauvegardes

Les opérations de sauvegarde doivent être :

- Entièrement automatisées, sans intervention humaine.
- Planifiées à intervalles réguliers (quotidien, hebdomadaire).
- Fiables et reproductibles, afin de limiter les erreurs humaines.

c) Sécurisation des données sauvegardées

Le système doit garantir :

- Le chiffrement des données au repos et en transit.
- La protection contre les accès non autorisés.
- L'intégrité des sauvegardes (détection de corruption).

d) Sauvegarde distante et résilience

Afin de se prémunir contre les sinistres locaux :

- Les sauvegardes doivent être stockées sur un serveur distant (off-site).
- Une politique de rétention des sauvegardes doit être définie (journalière, hebdomadaire, mensuelle).

e) Supervision en temps réel

Le système doit permettre :

- La surveillance de l'état des serveurs (CPU, RAM, disque).
- Le suivi du bon déroulement des sauvegardes.
- La détection proactive des anomalies et pannes.

f) Alertes et rapports

Le système de supervision doit être capable de :

- Générer des alertes en cas d'échec de sauvegarde ou de surcharge.
- Fournir des indicateurs clairs sur l'état du système.
- Faciliter le suivi administratif et technique.

3.1.2 Contraintes techniques

En parallèle des besoins fonctionnels, plusieurs contraintes techniques doivent être prises en compte afin d'assurer la faisabilité et la durabilité de la solution.

a) Contraintes budgétaires

L'université dispose de ressources financières limitées, ce qui impose :

- L'utilisation de solutions open source.
- L'absence de coûts de licence élevés.
- Une optimisation de l'infrastructure existante.

b) Contraintes matérielles

Le système doit fonctionner sur :

- Des serveurs aux capacités matérielles limitées (CPU, RAM, stockage).
- Une infrastructure déjà en production, sans interruption prolongée des services.

c) Contraintes réseau

- La bande passante disponible peut être limitée.
- Les sauvegardes distantes doivent être optimisées (déduplication,

compression).

- La latence réseau doit être prise en compte lors des transferts de données.

d) Contraintes de sécurité

Le système doit respecter :

- Les bonnes pratiques de sécurité informatique.
- La séparation des privilèges entre utilisateurs.
- La protection des accès SSH et des services critiques.

e) Contraintes humaines et organisationnelles

- Le personnel technique peut être limité en nombre.
- La solution doit être simple à administrer et à maintenir.
- La documentation et l'automatisation sont essentielles pour garantir la pérennité du système.

3.2. Conception globale du système de backup et de supervision

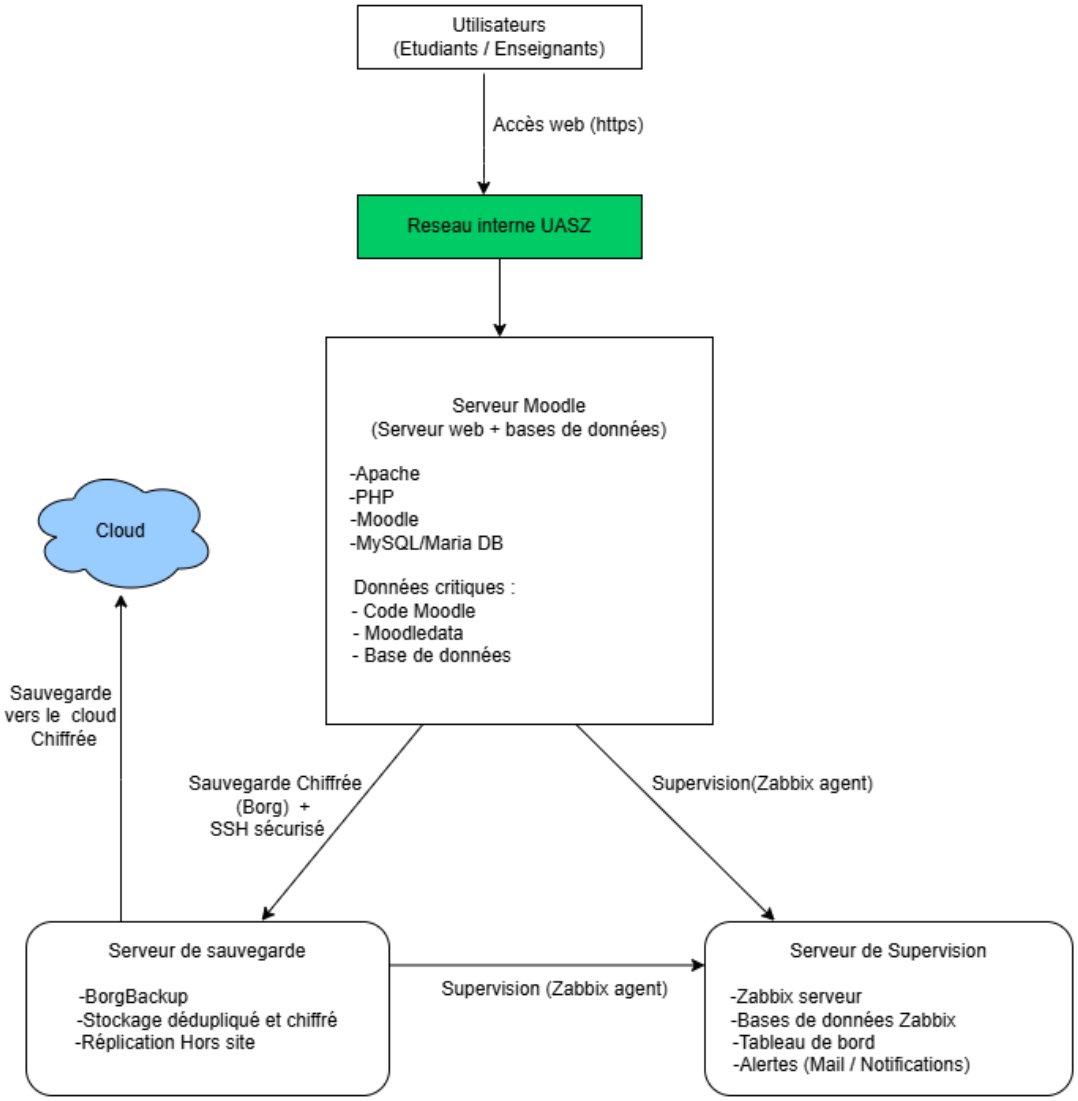
La conception du système de backup et de supervision repose sur une approche modulaire, sécurisée et évolutive, adaptée aux besoins d'un environnement d'enseignement à distance tel que celui de l'Université Assane Seck de Ziguinchor. L'objectif est de mettre en place une architecture capable d'assurer la **protection des données critiques**, la **continuité de service** et une **visibilité en temps réel** sur l'état des serveurs.

La solution proposée s'articule autour de deux axes complémentaires :

- Un **système de sauvegarde automatisé et chiffré**,
- Un **système de supervision centralisé**.

3.2.1 Schéma global de l'architecture proposée

Figure 3.2.1: Architecture globale du système de backup et de supervision pour l'environnement d'enseignement à distance de l'UASZ



L'architecture globale repose sur une séparation claire des rôles entre les différents serveurs afin de renforcer la sécurité et la résilience du système. Elle comprend principalement :

- Un **serveur applicatif** hébergeant les services pédagogiques (serveur web Moodle et base de données),
- Un **serveur de sauvegarde dédié**, situé sur un site distinct ou dans un réseau interne sécurisé,
- Un **serveur de supervision**, chargé de collecter, analyser et afficher les métriques de performance et d'état.
- **Le Cloud** pour assurer la sauvegarde à distance

Cette séparation permet de limiter l'impact d'un incident sur un composant donné et facilite la maintenance et l'évolution du système.

3.2.2 Architecture du système de sauvegarde

Le système de sauvegarde est conçu selon une architecture **client–serveur**, basée sur l'outil BorgBackup.

- Le **serveur Moodle** joue le rôle de client Borg :
  - Il effectue les sauvegardes des répertoires critiques (fichiers applicatifs, données Moodle, dumps de bases de données),
  - Les sauvegardes sont planifiées automatiquement via des tâches cron.
- Le **serveur de backup** héberge le dépôt Borg :
  - Les données y sont stockées sous forme chiffrée,
  - La déduplication permet de réduire considérablement l'espace disque utilisé,
  - L'accès au dépôt est sécurisé via SSH avec authentification par clé.

Les sauvegardes sont réalisées de manière incrémentale, garantissant des temps d'exécution réduits et une consommation optimale des ressources réseau et disque.

3.2.3 Architecture du système de supervision

La supervision repose sur une architecture centralisée utilisant **Zabbix**, composée de :

- Un **serveur Zabbix** :
  - Responsable de la collecte, du stockage et de l'analyse des données,
  - Héberge la base de données de supervision et l'interface web.
- Des **agents Zabbix** installés sur les serveurs supervisés :
  - Serveur Moodle
  - Serveur de Backup

Les agents collectent des métriques telles que :

- L'utilisation du CPU, de la mémoire et du disque,
- L'état des services (Apache/Nginx, MySQL/MariaDB, SSH),
- Le statut des sauvegardes Borg (succès, échec, durée, taille).

Les alertes sont configurées afin de notifier automatiquement les administrateurs en cas d'anomalie détectée.

3.2.4 Intégration entre sauvegarde et supervision

Un point clé de la conception est l'intégration entre le système de sauvegarde et celui de supervision. Cette intégration permet :

- De surveiller l'exécution des scripts de sauvegarde,
- De contrôler le nombre d'archives disponibles,
- De vérifier la taille du dépôt de sauvegarde,

- De détecter rapidement les échecs ou anomalies.

Les informations issues des sauvegardes sont exposées à Zabbix à travers des **UserParameters**, garantissant une supervision fine et personnalisée.

3.2.5 Sécurité de l'architecture proposée

La conception intègre dès le départ des mécanismes de sécurité essentiels :

- Chiffrement des données de sauvegarde (BorgBackup),
- Authentification par clés SSH pour les accès distants,
- Séparation des privilèges entre les utilisateurs système,
- Contrôle d'accès à l'interface de supervision,
- Limitation des flux réseau aux ports strictement nécessaires.

Cette approche permet de réduire significativement la surface d'attaque et d'assurer la confidentialité et l'intégrité des données.

Conclusion du Chapitre 3

Ce chapitre a été consacré à la **conception et à l'architecture du système de backup et de supervision** proposé pour l'environnement d'enseignement à distance de l'Université Assane Seck de Ziguinchor. À travers une analyse approfondie des besoins fonctionnels et des contraintes techniques, il a été possible de définir une solution adaptée aux exigences de disponibilité, de sécurité et de fiabilité des serveurs web et des bases de données.

L'étude a permis d'identifier les éléments critiques à protéger, notamment la plateforme Moodle, les données pédagogiques et les bases de données associées, tout en tenant compte des contraintes liées aux ressources matérielles, à la bande passante et à la sécurité. Sur cette base, une **architecture globale cohérente** a été conçue, reposant sur une séparation claire des rôles entre le serveur de production (serveur Moodle) et le serveur de sauvegarde distant.

Le choix d'une solution basée sur **BorgBackup** pour la sauvegarde chiffrée et dédupliquée, combinée à **Zabbix** pour la supervision en temps réel, permet d'assurer une protection efficace des données tout en offrant une visibilité permanente sur l'état du système. L'intégration de mécanismes de sécurité tels que le chiffrement, le contrôle d'accès et la supervision proactive renforce la résilience globale de l'infrastructure.

Ainsi, ce chapitre constitue une étape clé reliant les fondements théoriques présentés précédemment à la mise en œuvre pratique de la solution. Il prépare le terrain pour le chapitre suivant, qui portera sur **l'implémentation concrète, les tests et la validation du système**, afin d'évaluer son efficacité et sa pertinence dans un contexte réel d'enseignement à distance.

CHAPITRE 4 : IMPLÉMENTATION ET VALIDATION DE LA SOLUTION

4.1 Environnement de mise en œuvre

4.1.1 Présentation de l'infrastructure matérielle

La solution proposée a été implémentée dans un environnement représentant l'infrastructure informatique de l'Université Assane Seck de Ziguinchor, en particulier les serveurs dédiés à l'enseignement à distance.

L'architecture repose sur trois serveurs principaux :

- Un serveur Moodle (**serveur applicatif)**, hébergeant :
  - La plateforme d'enseignement à distance (Moodle),
  - Les fichiers applicatifs du site web,
  - Les données pédagogiques des utilisateurs,
  - La base de données associée.
- Un serveur de sauvegarde **(serveur Backup)**, physiquement ou logiquement distinct du serveur

Moodle dédié exclusivement :

  - au stockage sécurisé des sauvegardes,
  - à l'hébergement du dépôt BorgBackup,
- Un serveur Zabbix (**serveur de supervision**), qui permet :
  - La supervision du serveur applicatif et le système de sauvegarde.

Cette séparation permet de limiter les risques liés aux pannes locales, aux attaques informatiques et aux erreurs humaines, tout en respectant la règle de sauvegarde **hors-site logique**.

4.1.2 Présentation de l'infrastructure logicielle

L'environnement logiciel utilisé repose entièrement sur des solutions **open source**, garantissant flexibilité, sécurité et maîtrise des coûts.

Les principaux composants logiciels sont :

- **Système d'exploitation**
  Linux (distribution Ubuntu Server), choisi pour sa stabilité et son support communautaire.

- **Serveur Web**
  Apache, utilisé pour l'hébergement de la plateforme Moodle.

- **Base de données**
  MySQL / MariaDB, utilisée pour stocker les données pédagogiques et administratives.

- **Outil de sauvegarde**
  BorgBackup, choisi pour :
    - Le chiffrement natif,
    - La déduplication,
    - La sauvegarde incrémentielle efficace.

  Rclone est choisi pour la sauvegarde vers le cloud

- **Outil de supervision**
  Zabbix, utilisé pour :
    - La surveillance des serveurs,
    - Le suivi des sauvegardes,
    - La génération d'alertes automatiques.

- **Automatisation**
  Scripts Bash et planification via **CRON.**

4.1.3 Architecture réseau de déploiement

L'architecture réseau adoptée repose sur un **réseau interne sécurisé**, dans lequel :

- Le serveur Moodle dispose de l'adresse IP **192.168.1.41**,
- Le serveur de backup dispose l'adresse IP **192.168.1.250**
- Le serveur de supervision dispose l'adresse IP **192.168.1.50**
- Les communications entre les serveurs sont effectuées :

✓ via SSH sécurisé,
✓ avec authentification par clé,
✓ sur un réseau interne isolé.

Les flux principaux sont :

- Sauvegarde des données Moodle vers le serveur de backup,
- Sauvegarde des données du serveur backup vers le cloud,
- Supervision du serveur Moodle par Zabbix,
- Supervision du serveur backup par Zabbix,
- Consultation des tableaux de bord par l'administrateur.

4.2 Implémentation du système de sauvegarde

Cette section décrit les différentes étapes pratiques de déploiement du mécanisme de sauvegarde des serveurs web et bases de données dans l'environnement de l'Université Assane Seck de Ziguinchor. L'objectif est de garantir une sauvegarde fiable, automatisée, sécurisée et restaurable des données critiques de la plateforme d'enseignement à distance.

4.2.1 Préparation de l'environnement de sauvegarde

Avant la mise en œuvre du système de backup, une phase de préparation a été nécessaire afin d'assurer un fonctionnement optimal et sécurisé.

a) Mise en place du serveur de sauvegarde

Un serveur dédié a été configuré pour héberger les sauvegardes. Ce choix permet de :

- séparer physiquement les données de production et les données sauvegardées ;
- réduire les risques liés aux pannes locales ;
- renforcer la sécurité globale du système.

Le serveur de backup dispose :

- D'un espace disque suffisant pour stocker plusieurs versions des sauvegardes ;
- D'un utilisateur dédié (backupuser) afin de limiter les privilèges ;
- D'un répertoire réservé au dépôt Borg (/home/backupuser/borg_repo).

b) Sécurisation des accès

Pour sécuriser les échanges entre le serveur Moodle et le serveur de backup :

- L'authentification SSH par clé a été mise en place ;
- L'accès par mot de passe a été désactivé ;
- Les droits d'accès au dépôt Borg ont été strictement contrôlés.

Cette approche permet d'éviter les accès non autorisés et de garantir la confidentialité des données transférées.

4.2.2 Installation et configuration de BorgBackup

a) Installation de BorgBackup

BorgBackup a été installé sur :

- le serveur Moodle (client de sauvegarde),

```
alassane@alassane:~$ sudo apt install borgbackup -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
borgbackup est déjà la version la plus récente (1.2.0-1).
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libapache2-mod-php8.1 libevent-core-2.1-7 libevent-pthreads-2.1-7 libmecab2
  libprotobuf-lite23 libsnappy1v5 libwpe-1.0-1 libwpebackend-fdo-1.0-1
  linux-headers-6.8.0-65-generic linux-hwe-6.8-headers-6.8.0-65
  linux-hwe-6.8-tools-6.8.0-65 linux-image-6.8.0-65-generic
  linux-modules-6.8.0-65-generic linux-modules-extra-6.8.0-65-generic
  linux-tools-6.8.0-65-generic mecab-ipadic mecab-ipadic-utf8 mecab-utils
  php8.1 php8.1-bcmath php8.1-cli php8.1-curl php8.1-gd php8.1-intl
  php8.1-mbstring php8.1-mysql php8.1-soap php8.1-xml php8.1-xmlrpc php8.1-zip
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
0 mis à jour, 0 nouvellement installés, 0 à enlever et 5 non mis à jour.
```

Screenshot 1 :

- le serveur de backup (serveur de stockage).

```
adminbackup@backupuasz:~$ sudo apt install borgbackup -y
[sudo] Mot de passe de adminbackup :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
borgbackup est déjà la version la plus récente (1.2.0-1).
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
0 mis à jour, 0 nouvellement installés, 0 à enlever et 5 non mis à jour.
```

Screenshot 2 :

Cette installation permet d'exécuter les sauvegardes de manière distante tout en conservant un dépôt centralisé.

b) Configuration de BorgBackup

```
alassane@alassane:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/alassane/.ssh/id_ed25519):
Created directory '/home/alassane/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alassane/.ssh/id_ed25519
Your public key has been saved in /home/alassane/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:+lxJJWxRkOVunUKIFwGSYD29v+P41YYJM5dBuJ+gRmU alassane@alassane
The key's randomart image is:
+--[ED25519 256]--+
|    oo.o..=B+    |
|   .  +..oE*     |
|       ..+*.=    |
|        ooo= + . |
|       .So=o*.o  |
|       .o..Bo=   |
|      ..  o.+ o  |
|       o oo. .   |
|        +oo.     |
+----[SHA256]-----+
```

Screenshot 3 :

Une authentification par clé SSH de type ED25519 a été mise en place afin de sécuriser les connexions entre les serveurs tout en permettant l'automatisation des opérations de sauvegarde sans intervention humaine.

```
alassane@alassane:~$ ssh-copy-id backupuser@192.168.1.250
The authenticity of host '192.168.1.250 (192.168.1.250)' can't be established.
ED25519 key fingerprint is SHA256:l20voTJ6D2ZVp8rtJnvaNNiR4+GYqayBK4h32ifStcw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
backupuser@192.168.1.250's password:
Permission denied, please try again.
backupuser@192.168.1.250's password:
Permission denied, please try again.
backupuser@192.168.1.250's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'backupuser@192.168.1.250'"
and check to make sure that only the key(s) you wanted were added.

alassane@alassane:~$
```

Screenshot 4:

Et puis nous avons copiés les clés sur le serveur Backup via **ssh-copy.**

c) Initialisation du dépôt Borg sur le serveur backup

```
adminbackup@backupuasz:~$ sudo mkdir -p /home/backupuser/borg_repo
adminbackup@backupuasz:~$ sudo chown -R backupuser:backupuser /home/backupuser/borg_repo
adminbackup@backupuasz:~$ su - backupuser
Mot de passe :
backupuser@backupuasz:~$
```

Screenshot 5:

```
backupuser@backupuasz:~$ borg init --encryption=repokey /home/backupuser/borg_repo
Enter new passphrase:
Enter same passphrase again:
Do you want your passphrase to be displayed for verification? [yN]: y
Your passphrase (between double-quotes): "backupuasz"
Make sure the passphrase displayed above is exactly what you wanted.

By default repositories initialized with this version will produce security
errors if written to with an older version (up to and including Borg 1.0.8).

If you want to use these older versions, you can disable the check by running:
borg upgrade --disable-tam /home/backupuser/borg_repo

See https://borgbackup.readthedocs.io/en/stable/changes.html#pre-1-0-9-manifest-spoofing-vulnerability for details about the securit
y implications.

IMPORTANT: you will need both KEY AND PASSPHRASE to access this repo!
If you used a repokey mode, the key is stored in the repo, but you should back it up separately.
Use "borg key export" to export the key, optionally in printable format.
Write down the passphrase. Store both at safe place(s).

backupuser@backupuasz:~$
```

Screenshot 6:

Le dépôt Borg a été initialisé sur le serveur de backup avec les caractéristiques suivantes :

- Chiffrement natif activé (mode repokey) ;
- Protection par clé cryptographique ;
- Stockage dédupliqué pour optimiser l'espace disque.

Cette étape garantit que les données sauvegardées restent illisibles même en cas d'accès non autorisé au serveur de backup.

4.2.3 Sauvegarde des serveurs web et bases de données vers le serveur backup

```
alassane@alassane:~$ sudo nano /usr/local/bin/moodle_backup.sh
[sudo] Mot de passe de alassane :
alassane@alassane:~$ sudo chmod +x /usr/local/bin/moodle_backup.sh
```

Screenshot 7:

```
# Création du répertoire temporaire
mkdir -p $TMP_DIR

echo "[INFO] Dump de la base de données..."
mysqldump -u $DB_USER $DB_NAME > $TMP_DIR/db-${DB_NAME}-${DATE}.sql

DB_DUMP_PATH=$TMP_DIR/db-${DB_NAME}-${DATE}.sql

# calcule de la taille du DUMP
DB_SIZE_MB=$(du -sm "$DB_DUMP_PATH" | cut -f1 || echo 0)

echo "[INFO] Lancement de la sauvegarde Borg..."
borg create --stats --progress \
    $BORG_REPO::moodle-${DATE} \
    $MOODLE_PATH \
    $MOODLEDATA_PATH \
    $TMP_DIR \
    --exclude '/var/moodledata/sessions/*' \
    --exclude '/var/moodledata/temp/*' \
    --exclude '/var/moodledata/cache/*' \
    --exclude '/var/moodledata/localcache/*' \
    --exclude '/var/moodledata/secret/*'

RC=$?

echo "[INFO] Nettoyage des sauvegardes anciennes..."
borg prune -v --list $BORG_REPO --keep-daily=7 --keep-weekly=4 --keep-monthly=6

echo "[INFO] Suppression des fichiers temporaires..."
rm -rf $TMP_DIR

echo "[INFO] Sauvegarde terminée avec succès !"
```

Screenshot 8:

Un scripts shell a été développé pour :

- Lancer les sauvegardes Borg ;
- Horodater les archives ;
- Gérer les erreurs et les journaux d'exécution.

a) Sauvegarde des fichiers web

Les répertoires critiques du serveur Moodle ont été identifiés, notamment :

- Le répertoire d'installation de Moodle(/var/www/html/moodle) ;
- Le répertoire des données utilisateurs (moodledata).

Ces éléments sont essentiels au bon fonctionnement de la plateforme pédagogique.

b) Sauvegarde des bases de données

La base de données MySQL/MariaDB est sauvegardées via :

- Des exports automatisés (mysqldump ou équivalent) ;
- Un stockage temporaire avant intégration dans BorgBackup.

Cette méthode assure la cohérence des données et facilite la restauration complète du système.

c) Politique de rétention des sauvegardes

Une politique de rétention a été définie afin d'équilibrer :

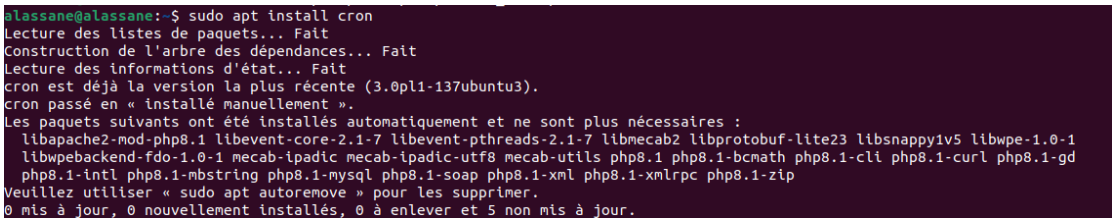
- La conservation historique des données ;
- L'optimisation de l'espace disque.

La stratégie appliquée permet par exemple de conserver :

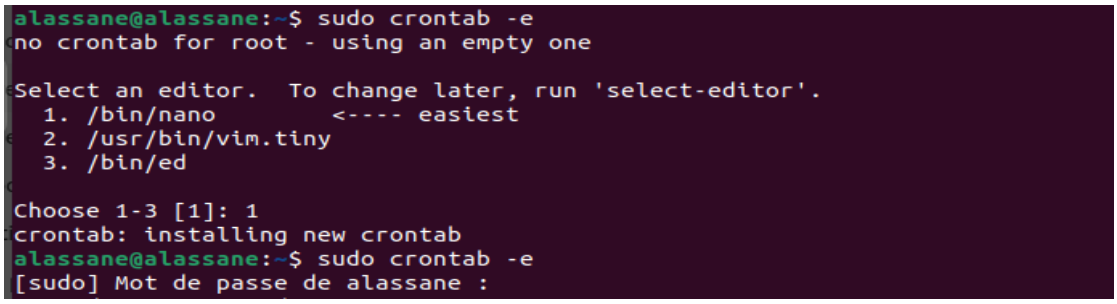
- Plusieurs sauvegardes quotidiennes récentes ;
- Des sauvegardes hebdomadaires et mensuelles plus anciennes.

d) Automatisation des sauvegardes

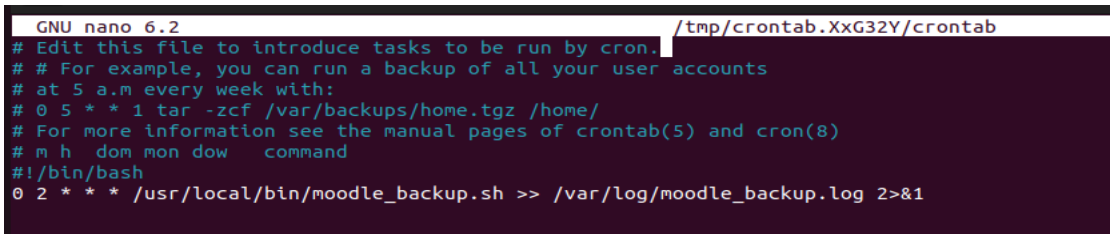
Afin d'éliminer les erreurs humaines et d'assurer une régularité des sauvegardes, l'ensemble du processus a été automatisé.



Screenshot 9: Installation cron



Screenshot 10 : Création du fichier crontab



Screenshot 11 : Edition du fichier crontab

Le script **moodle_backup.sh** est exécuté automatiquement tous les jours à 2h 00min à l'aide de **CRON**, ce qui permet :

- Une exécution sans intervention humaine ;
- Une traçabilité claire des sauvegardes ;
- Une conformité aux bonnes pratiques de sauvegarde.

e) Sécurisation de la connexion entre les serveurs

```
alassane@alassane:~$ sudo mkdir -p /root/.ssh
```

Screenshot 12 : Création du dossier /root/.ssh

```
alassane@alassane:~$ sudo cp /home/alassane/.ssh/id_ed25519 /root/.ssh/
alassane@alassane:~$ sudo cp /home/alassane/.ssh/id_ed25519.pub /root/.ssh/
alassane@alassane:~$ sudo chmod 600 /root/.ssh/id_rsa
alassane@alassane:~$ sudo chmod 600 /root/.ssh/id_ed25519
alassane@alassane:~$ sudo chmod 644 /root/.ssh/id_ed25519.pub
alassane@alassane:~$ sudo chown -R root:root /root/.ssh
```

Screenshot 13 : Copie des clés privé et publique sur /root/.ssh

```
alassane@alassane:~$ sudo cp /home/alassane/.ssh/known_hosts /root/.ssh/
alassane@alassane:~$ sudo chmod 644 /root/.ssh/known_hosts
alassane@alassane:~$
```

Screenshot 14 : Copie de known_hosts sur /root/.ssh

```
adminbackup@backupuasz:~$ sudo nano /etc/ssh/sshd_config.d/backupuser.conf
adminbackup@backupuasz:~$ sudo nano /etc/ssh/sshd_config.d/backupuser.conf
adminbackup@backupuasz:~$ sudo systemctl restart ssh
```

Screenshot 15 : Création du fichier backupuser.conf

Afin d'assurer une communication sécurisée et automatisée entre les serveurs impliqués dans le système de sauvegarde, une authentification par clés SSH a été mise en place. Cette méthode permet d'éliminer l'utilisation de mots de passe en clair lors des connexions distantes et renforce significativement la sécurité des échanges.

Dans un premier temps, une paire de clés SSH utilisant l'algorithme **Ed25519**, reconnu pour sa robustesse et ses performances, a été générée à l'aide de la commande ssh-keygen. La clé privée et la clé publique ont été créées dans le répertoire .ssh de l'utilisateur, garantissant ainsi une identification cryptographique unique du serveur source.

Ensuite, afin de permettre l'exécution automatisée des scripts de sauvegarde avec des privilèges administratifs, les clés générées ont été copiées dans le répertoire /root/.ssh. Les permissions ont été configurées conformément aux bonnes pratiques de sécurité (600 pour les clés privées et 644 pour les clés publiques), et la propriété des fichiers a été attribuée à l'utilisateur root, empêchant tout accès non autorisé.

Le fichier known_hosts a également été transféré et configuré afin d'éviter toute interruption liée à la vérification interactive de l'empreinte du serveur distant lors des connexions SSH automatisées.

Par ailleurs, un fichier de configuration dédié nommé **backupuser.conf** a été créé afin d'encadrer précisément les accès et les droits de l'utilisateur de sauvegarde (backupuser). Ce fichier permet de centraliser les paramètres liés à l'authentification et aux autorisations, garantissant que seules les opérations nécessaires à la sauvegarde et à la synchronisation des données puissent être exécutées. Cette approche contribue

à l'application du principe du moindre privilège et renforce la sécurité globale du système.

L'ensemble de cette configuration permet d'assurer des sauvegardes automatisées, fiables et sécurisées, tout en réduisant les risques liés aux erreurs humaines et aux accès non autorisés, ce qui est essentiel dans un environnement d'enseignement à distance tel que celui de l'Université Assane Seck de Ziguinchor.



Figure 2 : Schéma du mécanisme d'authentification SSH par clé entre le serveur Moodle et le serveur Backup

La figure ci-dessus illustre le mécanisme d'authentification SSH par clé utilisé pour sécuriser les échanges entre le serveur Moodle et le serveur Backup. Ce mécanisme repose sur une paire de clés cryptographiques ED25519, garantissant une communication chiffrée, une authentification forte et l'automatisation sécurisée des opérations de sauvegarde.

Étapes du processus SSH

- Le serveur Moodle initie la connexion SSH.
- Le serveur Backup envoie un défi cryptographique
- Le serveur Moodle signe le défi avec la clé privée
- Le serveur Backup vérifie avec la clé publique
- Connexion sécurisée établie
- Exécution automatique de la sauvegarde Borg

4.2.4 Sauvegarde des données du serveur backup vers le cloud

Dans le cadre de la mise en œuvre de la sauvegarde distante des données du serveur Backup vers le cloud, un projet dédié a été créé sur la plateforme **Google Cloud Platform (GCP)** afin de permettre l'accès sécurisé à l'API Google Drive via l'outil **rclone**. Cette étape est indispensable pour autoriser l'authentification OAuth2 et assurer une synchronisation fiable des sauvegardes.

a) Installation de Rclone

```
adminbackup@backupuasz:~$ sudo apt install rclone -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
Les NOUVEAUX paquets suivants seront installés :
  rclone
0 mis à jour, 1 nouvellement installés, 0 à enlever et 9 non mis à jour.
Il est nécessaire de prendre 11,7 Mo dans les archives.
Après cette opération, 42,6 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 http://sn.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 rclone amd64 1.53.3-4ubuntu1.22.04.3 [11,7 MB]
11,7 Mo réceptionnés en 31s (377 ko/s)
Sélection du paquet rclone précédemment désélectionné.
(Lecture de la base de données... 210754 fichiers et répertoires déjà installés.
)
Préparation du dépaquetage de .../rclone_1.53.3-4ubuntu1.22.04.3_amd64.deb ...
Dépaquetage de rclone (1.53.3-4ubuntu1.22.04.3) ...
Paramétrage de rclone (1.53.3-4ubuntu1.22.04.3) ...
Traitement des actions différées (« triggers ») pour man-db (2.10.2-1) ...
adminbackup@backupuasz:~$
```

Screenshot 16 : Installation de l'outil Rclone sur le serveur Backup

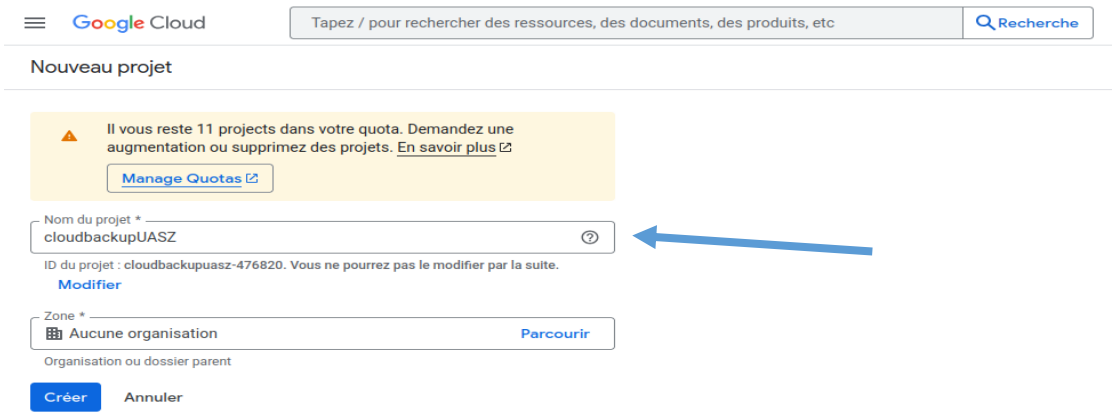
b) Configuration de Rclone

```
adminbackup@backupuasz:~$ rclone config
2025/10/31 19:56:43 NOTICE: Config file "/home/adminbackup/.config/rclone/rclone.conf" not found - using defaults
No remotes found - make a new one
n) New remote
s) Set configuration password
q) Quit config
n/s/q> n
name> cloudbackup
Type of storage to configure.
Enter a string value. Press Enter for the default ("").
Choose a number from below, or type in your own value
 1 / 1Fichier
   \ "fichier"
 2 / Alias for an existing remote
   \ "alias"
 3 / Amazon Drive
   \ "amazon cloud drive"
 4 / Amazon S3 Compliant Storage Provider (AWS, Alibaba, Ceph, Digital Ocean, Dreamhost, IBM COS, Minio, Tencent COS, etc)
   \ "s3"
 5 / Backblaze B2
   \ "b2"
 6 / Box
   \ "box"
 7 / Cache a remote
   \ "cache"
 8 / Citrix Sharefile
   \ "sharefile"
 9 / Dropbox
   \ "dropbox"
10 / Encrypt/Decrypt a remote
   \ "crypt"
11 / FTP Connection
   \ "ftp"
12 / Google Cloud Storage (this is not Google Drive)
   \ "google cloud storage"
13 / Google Drive
   \ "drive"
14 / Google Photos
   \ "google photos"
```

Screenshot 17 : Configuration de Rclone

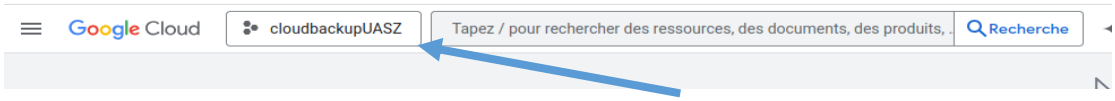
Lors de la configuration, on a choisi le numéro 13(Google Drive) comme Cloud pour stocker les données du serveur backup.

Ensuite il nous demande un **client_id** et un **client_secret** qu'on doit générer via la plateforme google cloud **(https://console.cloud.google.com/)**

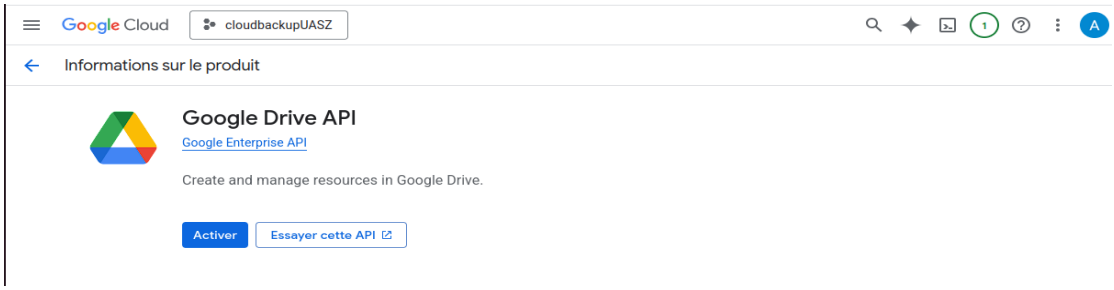


Screenshot 18 : Création du projet Google Cloud

On crée un projet Google Cloud nommé **cloudbackupUASZ**, comme illustré sur le **Screenshot 18**. Ce projet sert de conteneur logique pour l'ensemble des configurations liées à l'accès aux services cloud, notamment l'authentification et la gestion des permissions.



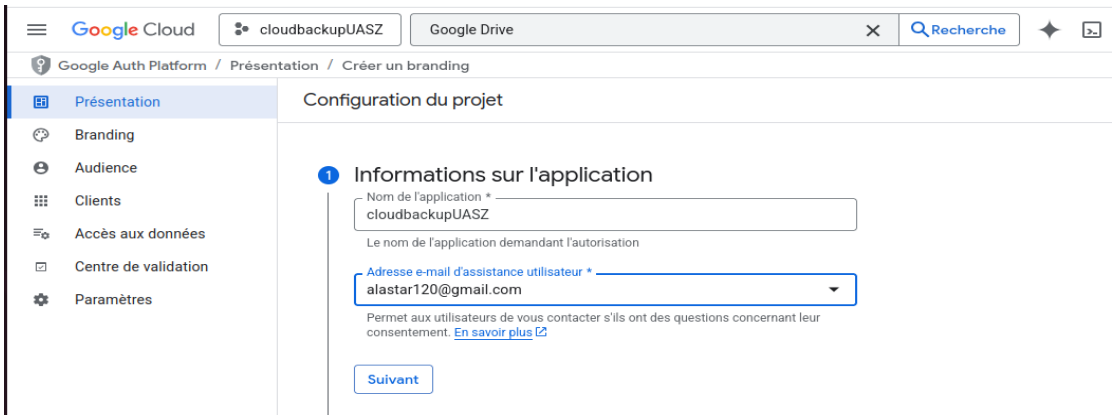
Screenshot 19 : Projet cloudbackupUASZ crée



Screenshot 20 : Activation Google drive API

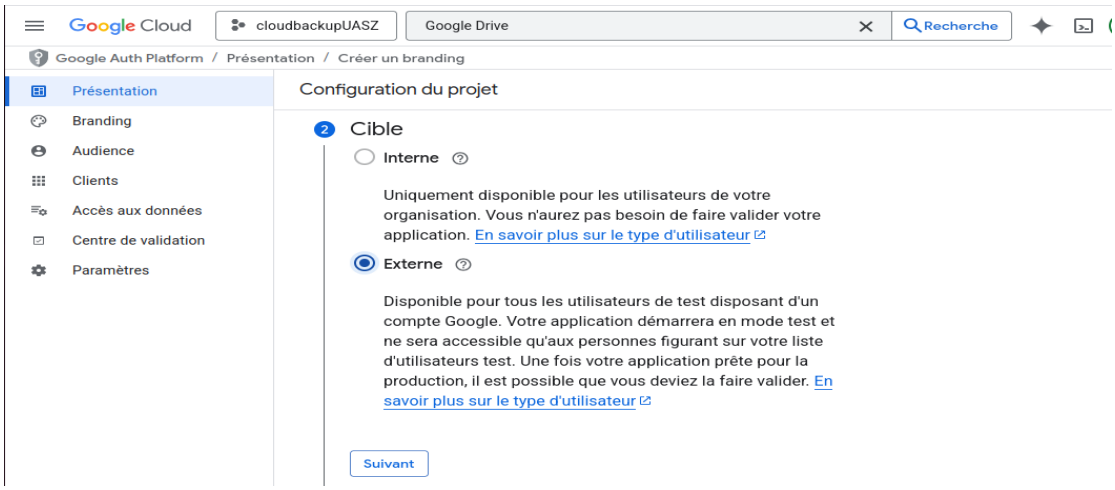
Dans la barre de recherche on tape **Google drive API** puis on l'active pour pouvoir utiliser Google drive.

- **Configuration du projet google cloud (cloudbackupUASZ)**



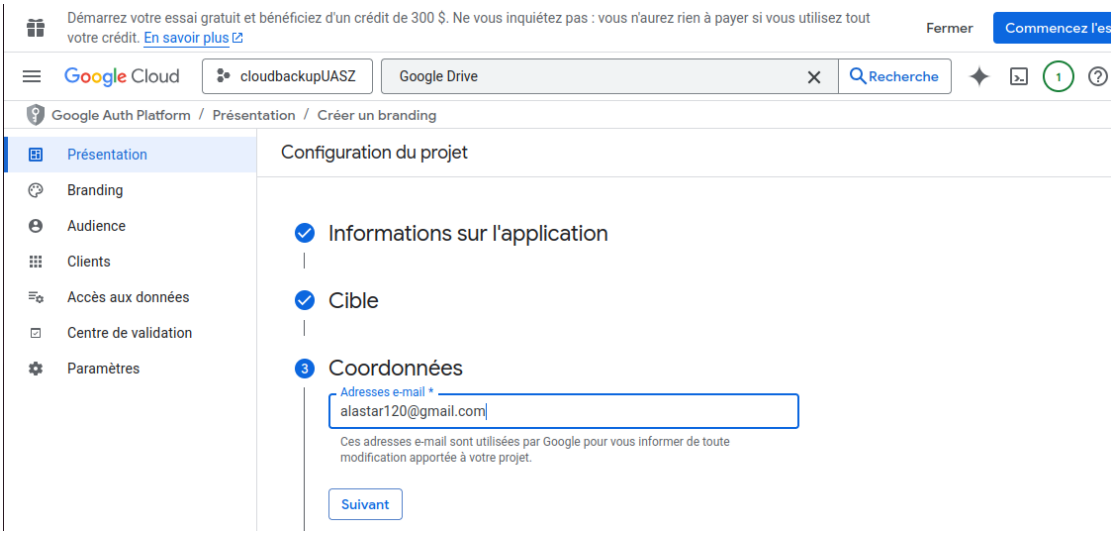
Screenshot 21 : Configuration du projet (Informations sur l'application)

Les informations relatives à l'application, telles que le nom de l'application (cloudbackupUASZ) et l'adresse e-mail d'assistance, ont été renseignées afin d'identifier clairement l'application lors des demandes d'autorisation.



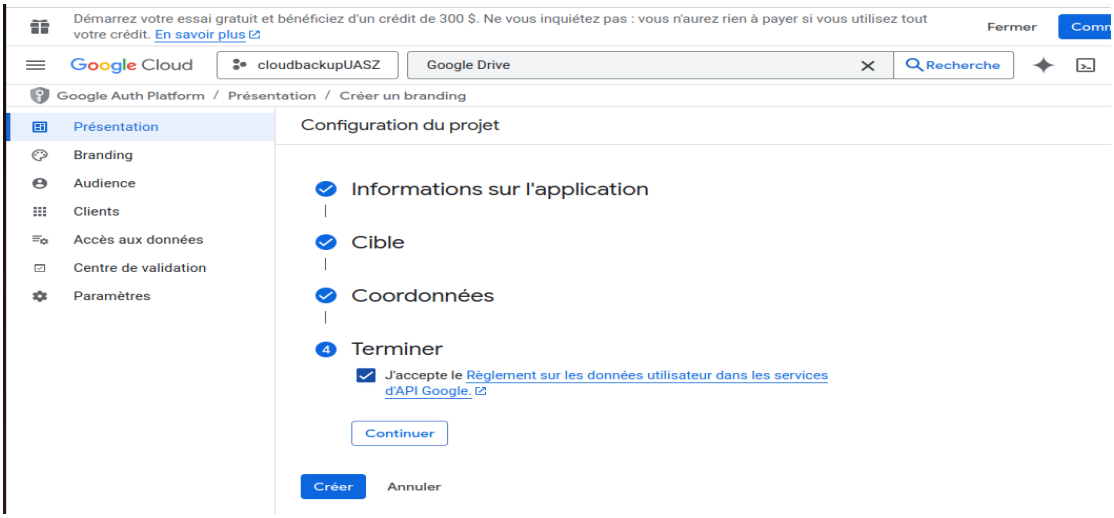
Screenshot 22 : Configuration du projet (Cible)

La configuration de l'écran de consentement OAuth a été réalisée dans la section *Google Auth Platform*. Le type d'application a été défini comme **application externe**, afin de permettre l'accès depuis un compte Google standard, ce qui est adapté à un usage universitaire sans domaine Google Workspace dédié. Cette configuration autorise l'utilisation du projet en mode test, suffisant pour les besoins de sauvegarde automatisée.



Screenshot 23 : Configuration du projet (Coordonnées)

Une adresse e-mail de contact a également été définie pour recevoir les notifications importantes liées au projet.

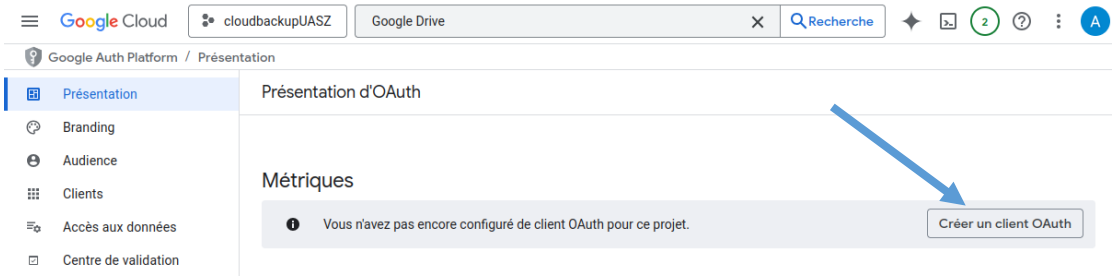


Screenshot 24 : Configuration du projet (terminer)

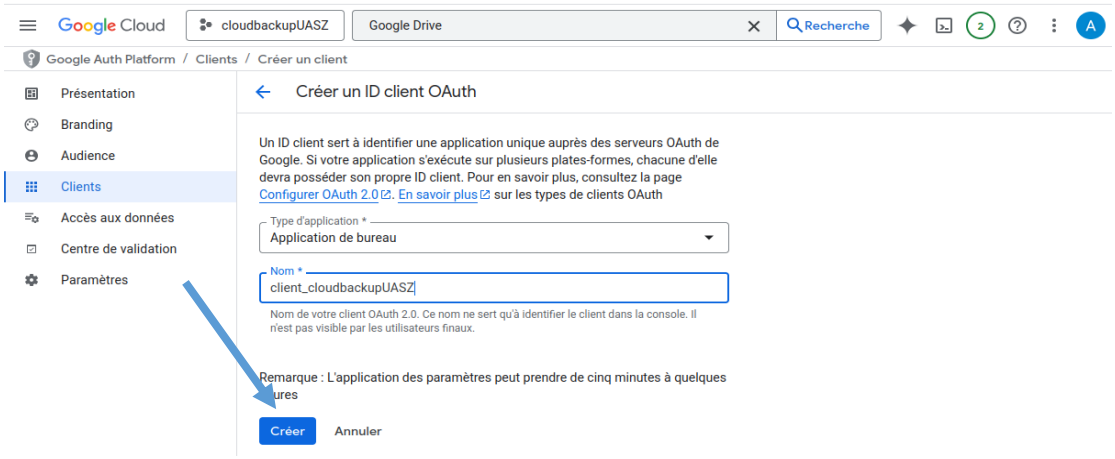
Cette étape garantit que les accès au service Google Drive sont réalisés de manière **contrôlée, traçable et sécurisée**, conformément aux bonnes pratiques de sécurité recommandées par Google. Elle constitue un prérequis fondamental pour l'utilisation de **rclone avec un backend Google Drive chiffré**, utilisé ultérieurement pour la synchronisation des sauvegardes Borg depuis le serveur Backup vers le cloud.

- **Création d'un client OAuth**

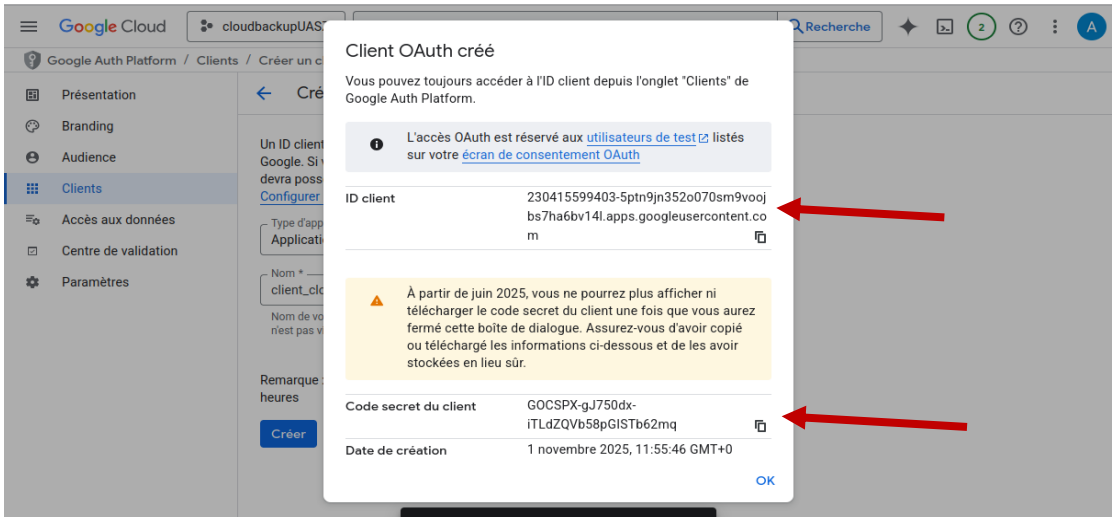
Le client OAuth 2.0 est utilisé pour permettre au système de sauvegarde d'accéder aux services Google (Gmail/Drive) de manière sécurisée, sans exposer les identifiants du compte administrateur.



Screenshot 25 : Créer un client OAuth



Screenshot 26 : Génération d'un client_id OAuth



Screenshot 27 : client_id et client_secret générés

Afin de permettre l'accès sécurisé du serveur de sauvegarde au service Google Drive, un **client OAuth 2.0** a été créé via la plateforme **Google Cloud Console**. OAuth 2.0

est un protocole standard d'autorisation qui permet à une application d'accéder à des ressources protégées sans exposer les identifiants de l'utilisateur, garantissant ainsi un haut niveau de sécurité.

La création du client OAuth débute par l'accès à la section **Google Auth Platform**, puis à l'onglet **Clients**. Un nouveau client est ensuite déclaré en sélectionnant le type d'application **Application de bureau (Desktop App)**, choix particulièrement adapté aux outils en ligne de commande tels que rclone (voir **Screenshot 25 et 26**). Ce type de client permet l'authentification via un navigateur web lors de la première configuration, suivie de la génération automatique de jetons d'accès et de rafraîchissement.

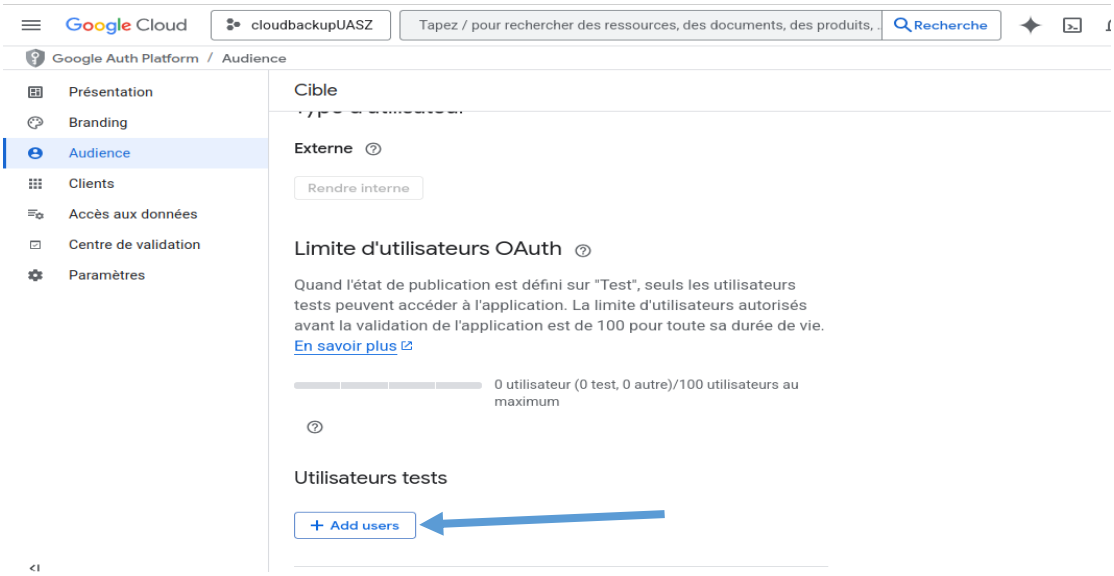
Une fois le client créé, Google fournit deux éléments essentiels :

- un **Client ID**,
- un **Client Secret**. (voir **Screenshot 27**)

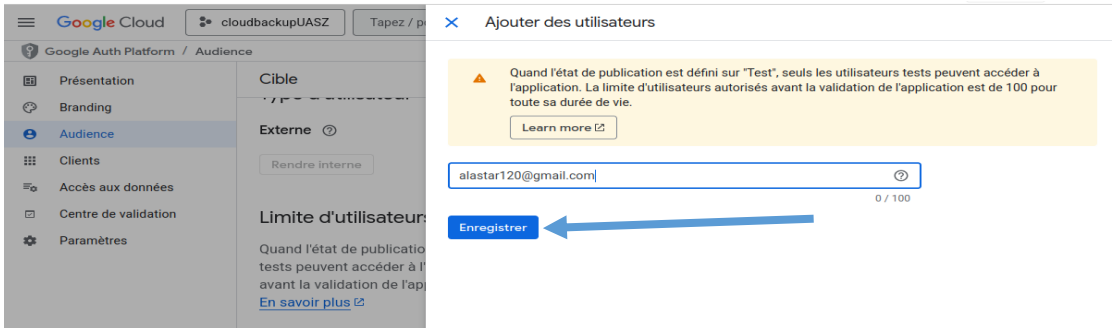
Ces identifiants sont utilisés par rclone pour établir une connexion sécurisée avec Google Drive lors de la synchronisation des données. Les jetons générés sont ensuite stockés localement sur le serveur backup, ce qui permet l'exécution automatique des synchronisations ultérieures sans intervention humaine.

L'utilisation d'un client OAuth constitue une étape essentielle de la solution de sauvegarde cloud proposée, en garantissant un accès sécurisé, contrôlé et conforme aux standards modernes de sécurité des données.

- **Ajouter un utilisateur de test**



Screenshot 28 : Ajouter un utilisateur de test

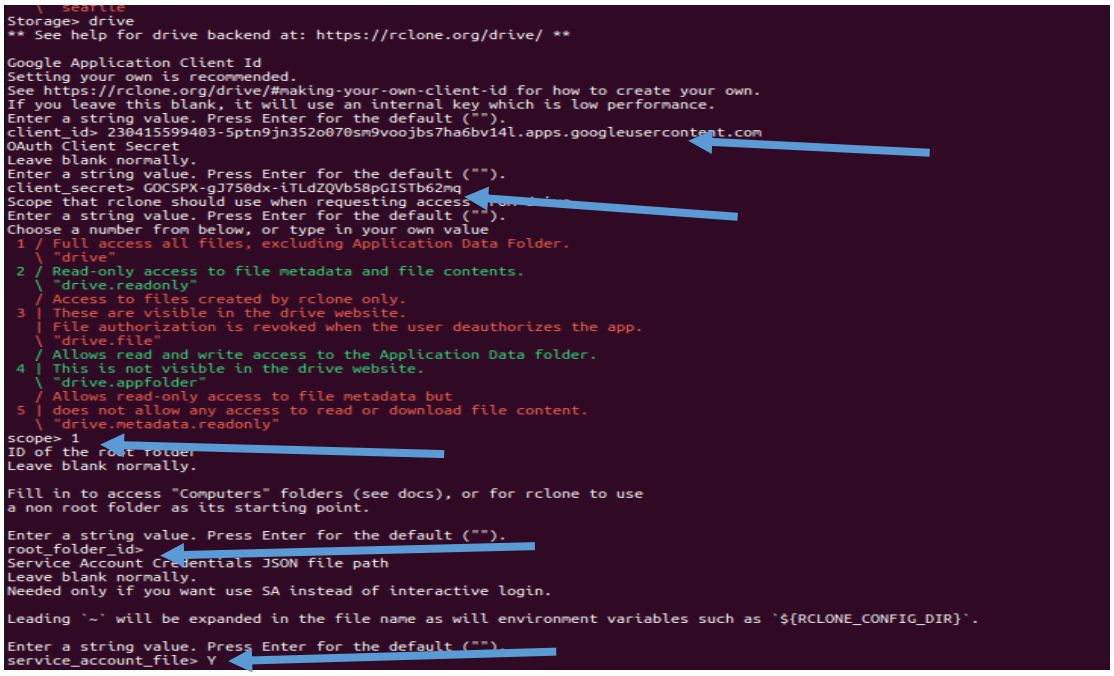


Screenshot 29 : Ajouter un utilisateur de test (suite et fin)

L'adresse e-mail utilisée pour la sauvegarde cloud (par exemple : *alastar120@gmail.com*) a été enregistrée comme utilisateur de test (voir **Screenshot 28 et 29**). Cette étape est indispensable pour autoriser l'authentification OAuth lors de la configuration de rclone et éviter les refus d'accès liés aux politiques de sécurité de Google.

Grâce à cette configuration, le serveur de sauvegarde peut établir une connexion OAuth valide avec Google Drive, tout en restant dans un environnement contrôlé et sécurisé. Cette approche garantit la conformité avec les exigences de Google tout en permettant la mise en œuvre efficace de la sauvegarde distante chiffrée des données critiques de l'Université Assane Seck de Ziguinchor.

La mise en place du client OAuth et la définition des utilisateurs de test constituent une étape essentielle dans la sécurisation de la synchronisation cloud, assurant à la fois l'automatisation des sauvegardes et le respect des bonnes pratiques de sécurité imposées par les fournisseurs de services cloud.

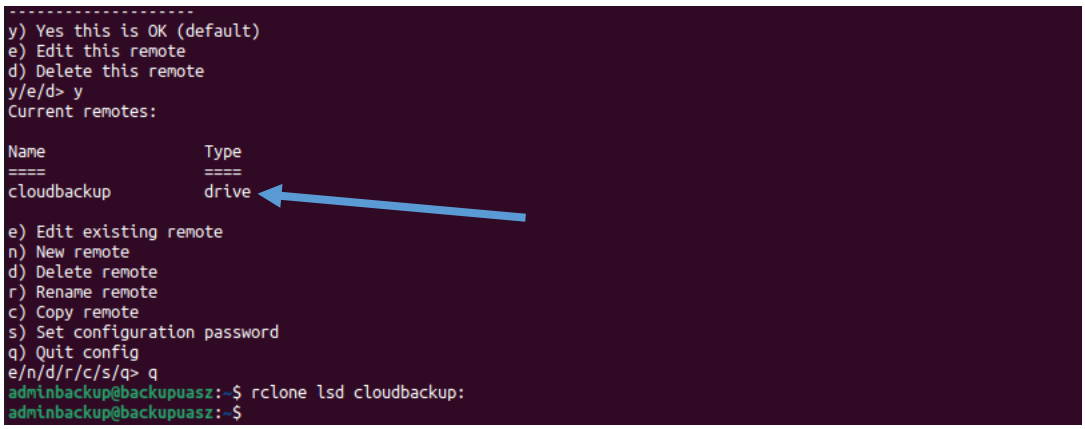


Screenshot 30 : Configuration de Rclone (Suite)

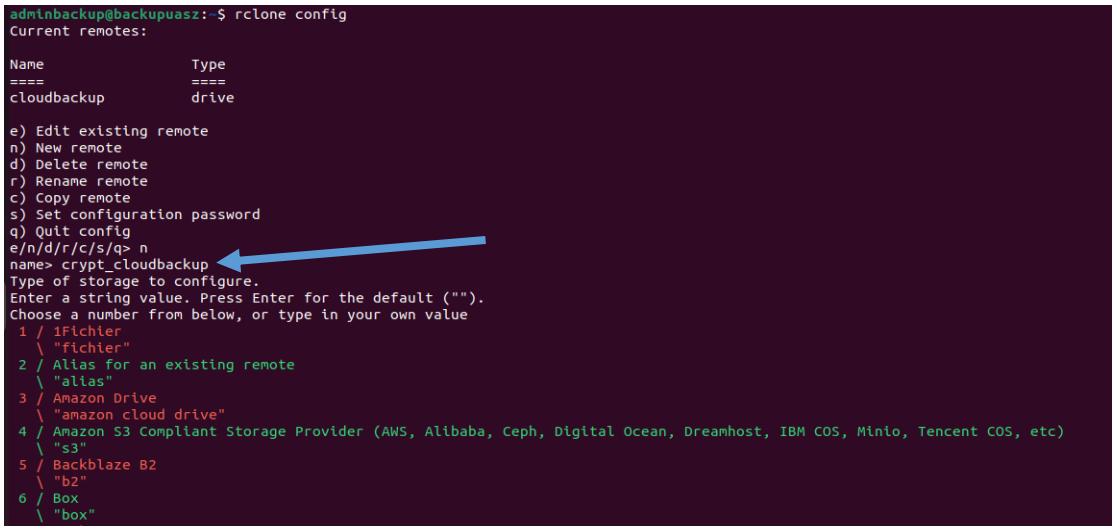


Screenshot 31 : Configuration de Rclone (Résumé)

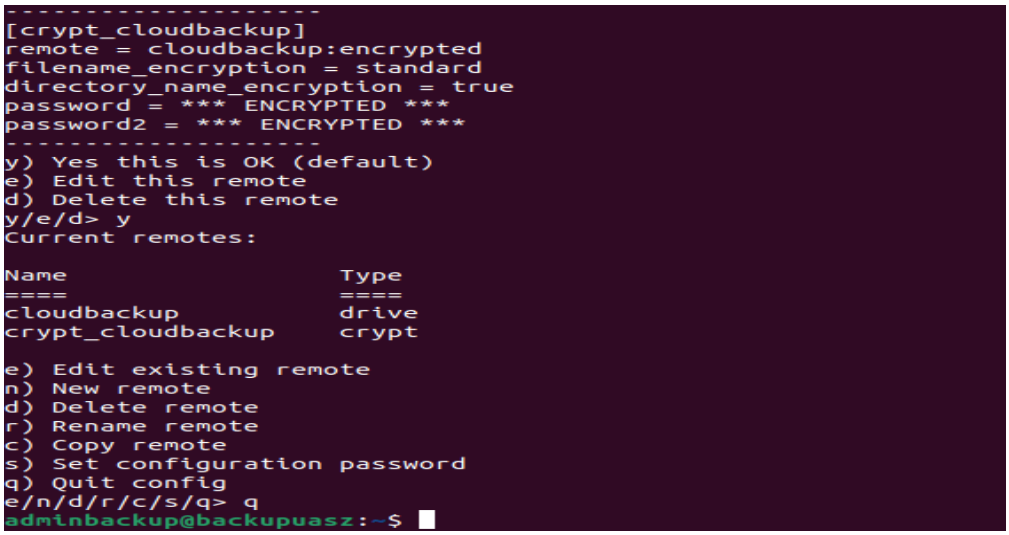
Durant cette configuration, plusieurs paramètres essentiels ont été définis : tout d'abord, l'identifiant client OAuth (client_id) et le secret client (client_secret) ont été renseignés, permettant l'authentification auprès de l'API Google Drive. Ensuite, le niveau d'accès (scope) a été configuré sur l'option 1, accordant un accès complet aux fichiers du Drive, incluant les permissions de lecture, écriture et modification des métadonnées. Cette configuration permet également l'accès au dossier "Application Data" de Google Drive. Le paramètre root_folder_id a été laissé vide, signifiant que l'accès s'effectue depuis le répertoire racine du Drive. Enfin, l'option service_account_file a également été omise, indiquant l'utilisation d'un compte utilisateur standard plutôt qu'un compte de service. Cette configuration constitue une étape fondamentale pour l'automatisation des opérations de sauvegarde et de synchronisation des données avec Google Drive dans le cadre de notre solution de gestion documentaire. (voir **Screenshot 30 et 31**)



Screenshot 32 : Configuration de Rclone (cloudbackup) Réussie



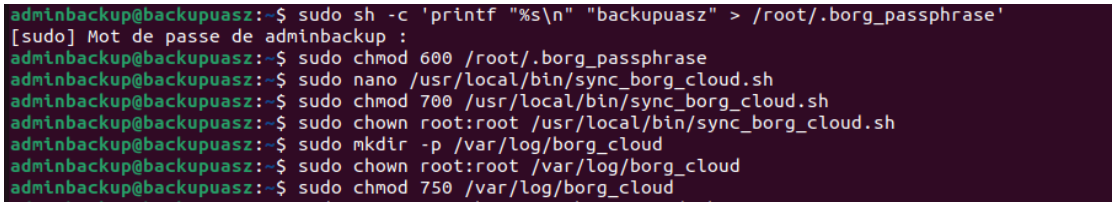
Screenshot 33 : Création de crypt_cloudbackup



Screenshot 34 : crypt_cloudbackup crée

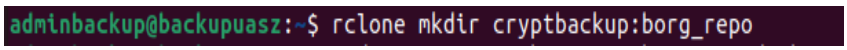
Le **Screenshot 33** montre la création d'un remote de type "**crypt**" nommé **crypt_cloudbackup**, qui s'appuie sur le remote préalablement configuré **cloudbackup** (voir **Screenshot 32**). Cette configuration utilise le mode de chiffrement "**encrypted**" pour le remote, avec un chiffrement standard pour les noms de fichiers (**filename_encryption = standard**) et un chiffrement des noms de répertoires activé (**directory_name_encryption = true**). Deux mots de passe ont été définis et chiffrés (indiqués par *** **ENCRYPTED** ***) : un mot de passe principal (**password**) pour le chiffrement des données, et un second mot de passe (**password2**) pour l'obscurcissement des noms de fichiers, renforçant ainsi la sécurité. Le **Screenshot 34** illustre également l'état final de la configuration Rclone, montrant les deux remotes actifs : **cloudbackup** de type "**drive**" pour l'accès direct à Google Drive, et **crypt_cloudbackup** de type "**crypt**" qui assure le chiffrement transparent des données avant leur transfert vers le cloud. Cette architecture en couches garantit que toutes les données sauvegardées sont chiffrées localement avant d'être transmises,

protégeant ainsi la confidentialité des informations même en cas de compromission du compte cloud.

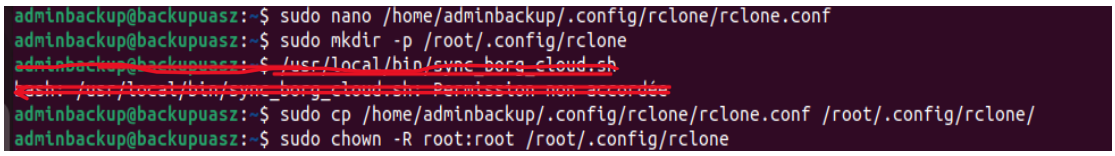


Screenshot 35 : Création du fichier .borg_passphrase et du script de synchronisation vers le cloud.

La séquence de commandes (**Screenshot 35)** montre plusieurs opérations critiques : premièrement, la création sécurisée du fichier /root/.borg_passphrase contenant le mot de passe de chiffrement des archives Borg, avec les permissions restrictives 600 (lecture/écriture uniquement pour root) afin de protéger cette information sensible. Ensuite, le script sync_borg_cloud.sh est créé dans /usr/local/bin/ et configuré avec les permissions d'exécution appropriées (700 pour le propriétaire uniquement). Les permissions du script et de son répertoire parent sont ensuite ajustées pour garantir que seul l'utilisateur root puisse modifier le script. Enfin, un répertoire de logs dédié /var/log/borg_cloud est créé avec les permissions 750 pour conserver l'historique des opérations de synchronisation. Cette configuration respecte les principes de sécurité en limitant l'accès aux fichiers critiques et en établissant une traçabilité des opérations de sauvegarde via les logs.



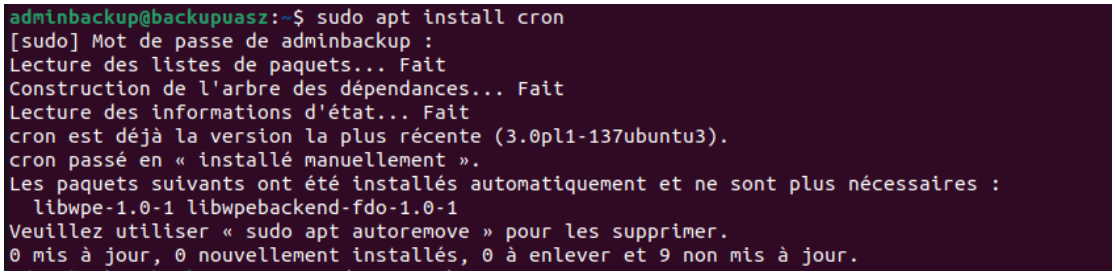
Screenshot 36 : création du dossier borg_repo dans crypt_cloudbackup.



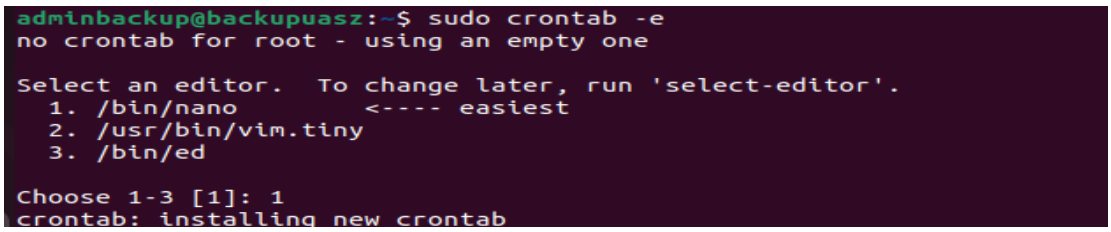
Screenshot 37 : Déploiement de la configuration Rclone dans l'environnement root

Le **Screenshot 37** montre l'édition du fichier de configuration Rclone situé dans le répertoire personnel de l'utilisateur (/home/adminbackup/.config/rclone/rclone.conf). Ensuite, un répertoire de configuration dédié est créé dans l'arborescence root (/root/.config/rclone/) pour centraliser les paramètres d'accès au cloud. Puis la copie du fichier de configuration vers le répertoire root nouvellement créé (/root/.config/rclone/). Enfin, les droits de propriété sont ajustés via la commande **chown -R root:root** pour garantir que seul le compte root possède les permissions appropriées sur ce répertoire et son contenu. Cette migration de la configuration depuis l'espace utilisateur vers l'espace système permet au script de synchronisation, exécuté en tant que root via des tâches planifiées, d'accéder aux credentials

nécessaires pour transférer les archives Borg chiffrées vers Google Drive de manière autonome et sécurisée.



Screenshot 38 : Installation CRON sur le serveur backup



Screenshot 39 : Edition du fichier CRONTAB pour l'automatisation

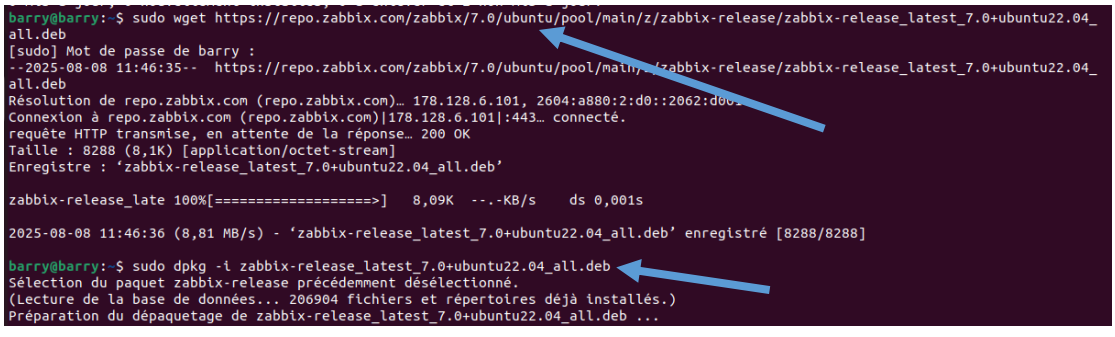
4.3 Implémentation du système de supervision

4.3.1 Installation du serveur Zabbix

La première étape a consisté à installer le serveur Zabbix sur une machine dédiée. Ce serveur joue le rôle de cœur du système de supervision et centralise la collecte, le traitement et l'affichage des métriques.

L'installation comprend :

- Le serveur Zabbix,
- La base de données (MySQL),
- L'interface web,
- Le serveur Apache et PHP



Screenshot 40 : Récupération du paquet du dépôt Zabbix

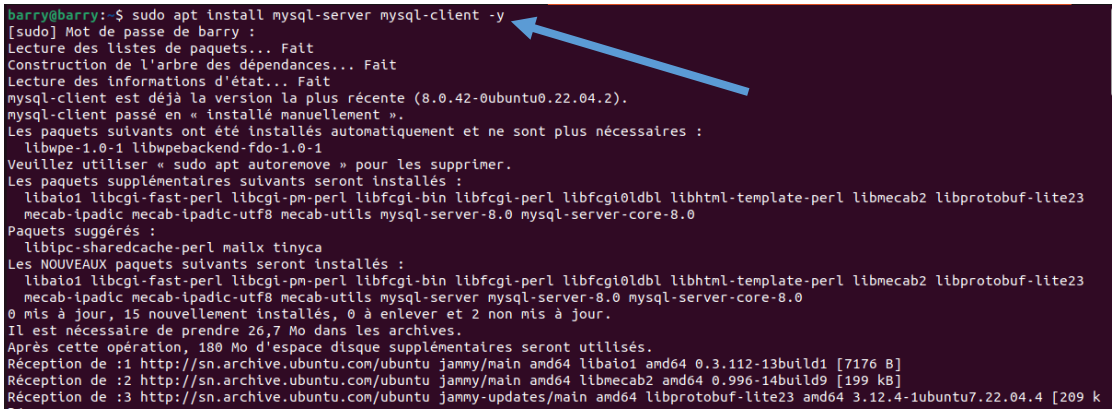
La première commande récupère le paquet du dépôt Zabbix destinée à Ubuntu 22.04.

La deuxième commande installe le paquet provenant du dépôt Zabbix précédemment téléchargé.

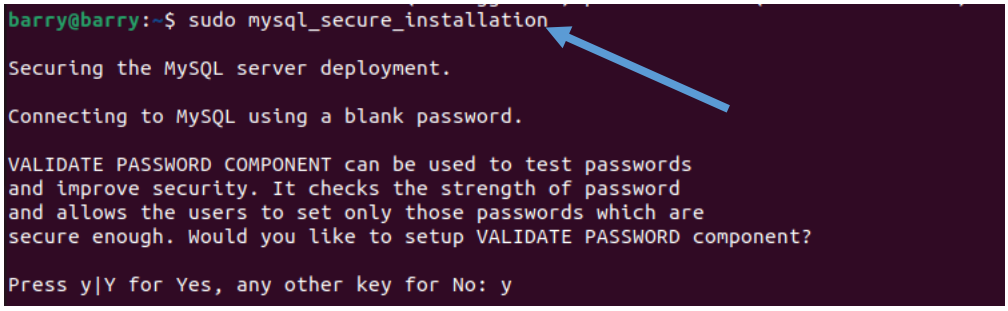


Screenshot 41 : Installation serveur Zabbix, le frontend et l'agent

Cette commande installe le serveur Zabbix, le frontend, l'agent Zabbix et d'autres paquets nécessaires.

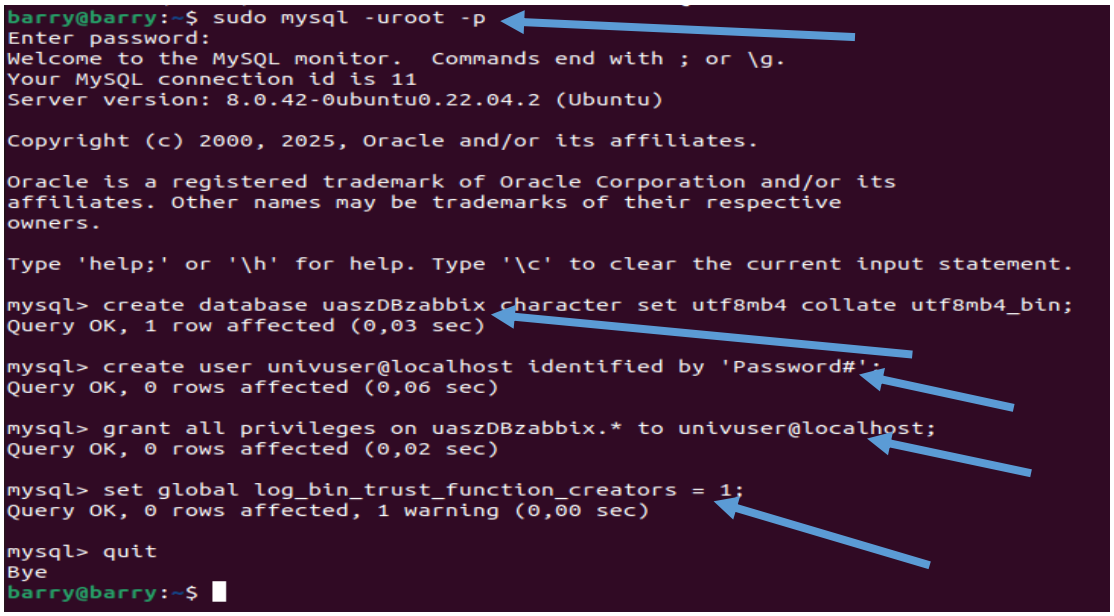


Screenshot 42 : Installation du serveur MySQL et le client MySQL



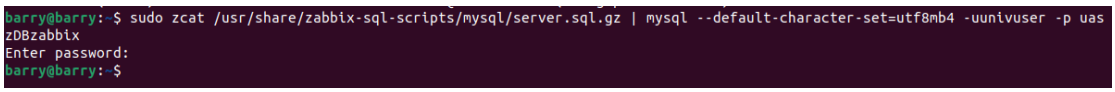
Screenshot 43 : Installation mysql_secure_installation

Mysql_secure nous permet de sécuriser notre base de données.



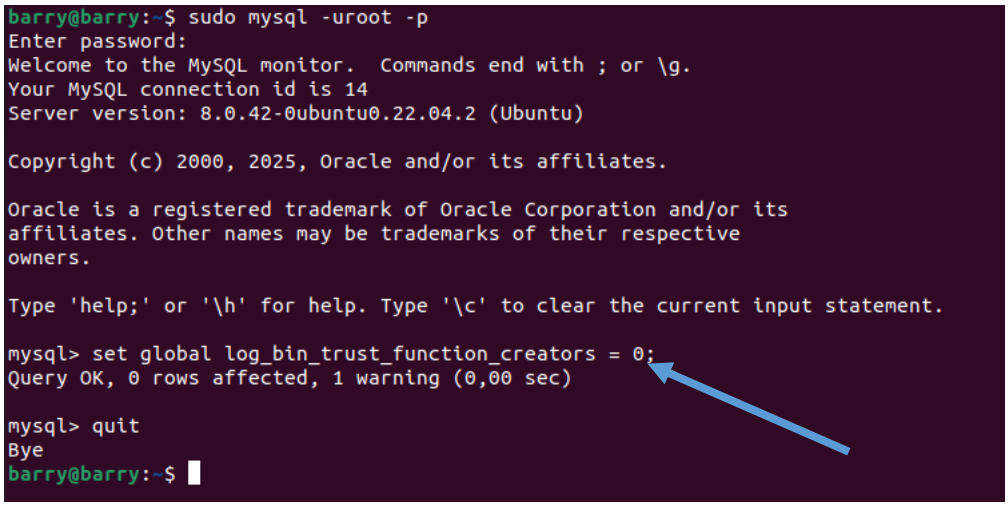
Screenshot 44 : Création de la base de données (uaszDBzabbix)

La première commande permet de se connecter à MySQL en tant que root. La deuxième crée la base de données **uaszDBzabbix** avec le jeu de caractères utf8mb4 et la collation utf8mb4_bin, garantissant ainsi la prise en charge complète des caractères Unicode et une gestion sensible à la casse, essentielles pour le stockage des données de monitoring. Ensuite, un utilisateur dédié **univuser@localhost** est créé avec son mot de passe associé (**Password**). Cet utilisateur se voit attribuer tous les privilèges (**GRANT ALL PRIVILEGES)** sur la base de données **uaszDBzabbix**, lui permettant d'effectuer toutes les opérations nécessaires (lecture, écriture, modification de structure). Enfin, le paramètre **log_bin_trust_function_creators** est activé pour autoriser la création de fonctions et de triggers sans restriction, une configuration souvent requise lors de l'initialisation du schéma Zabbix. Cette préparation de l'environnement de base de données constitue une étape fondamentale avant l'installation et la configuration du serveur Zabbix.



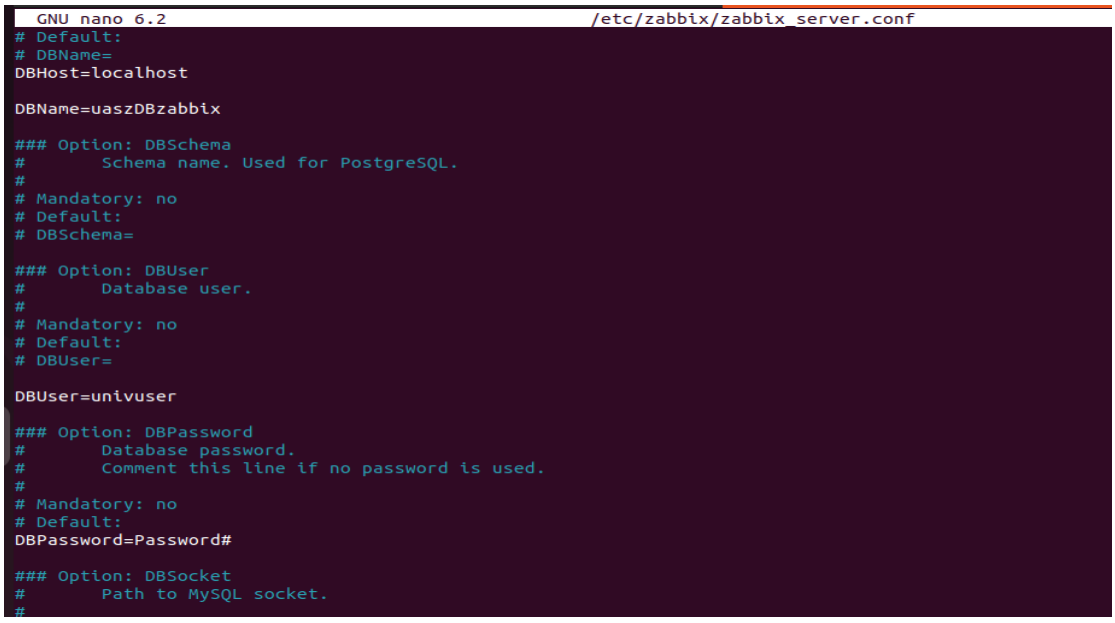
Screenshot 45 : Importation du schéma initial de la base de données Zabbix

La commande utilise **zcat** pour décompresser à la volée le fichier SQL compressé (server.sql.gz) situé dans **/usr/share/zabbix-sql-scripts/mysql/**, puis redirige le contenu via un pipe vers le client MySQL. Les paramètres spécifiés incluent **--default-character-set=utf8mb4** pour garantir la cohérence avec le jeu de caractères défini lors de la création de la base, **-uunivuser** pour se connecter avec l'utilisateur dédié créé précédemment, **-p** pour activer l'authentification par mot de passe (saisi de manière sécurisée après l'invite), et enfin **uaszDBzabbix** comme base de données cible. Cette opération importe l'ensemble de la structure de la base de données Zabbix, incluant les tables, index, contraintes et données initiales nécessaires au fonctionnement du système de monitoring.



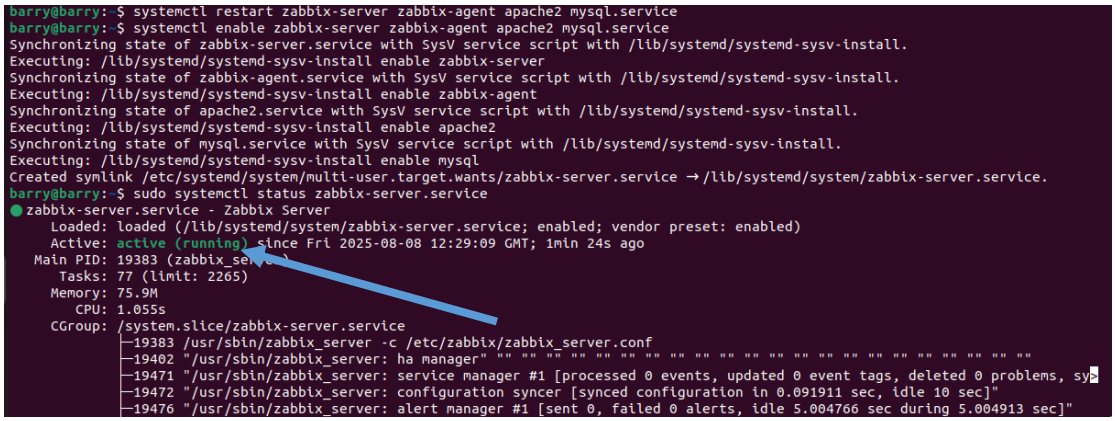
Screenshot 46 : Désactivation du paramètre log_bin_trust_function_creators après importation

Cette capture d'écran illustre la désactivation du paramètre **log_bin_trust_function_creators** après l'importation réussie du schéma Zabbix, conformément aux bonnes pratiques de sécurité MySQL. Une nouvelle connexion au serveur MySQL est établie en tant qu'utilisateur root, et la commande **SET GLOBAL log_bin_trust_function_creators = 0;** est exécutée pour rétablir la restriction sur la création de fonctions et de triggers. Ce paramètre avait été temporairement activé (**valeur = 1**) lors de la phase de configuration initiale pour permettre l'importation du schéma complet de Zabbix sans restrictions. Sa désactivation (**valeur = 0**) renforce la sécurité de la base de données en exigeant que les utilisateurs disposent du privilège CREATE ROUTINE pour créer des fonctions stockées et des triggers, empêchant ainsi les créations non autorisées qui pourraient compromettre l'intégrité ou la sécurité du système.



Screenshot 47 : Édition du fichier de configuration zabbix_server.conf

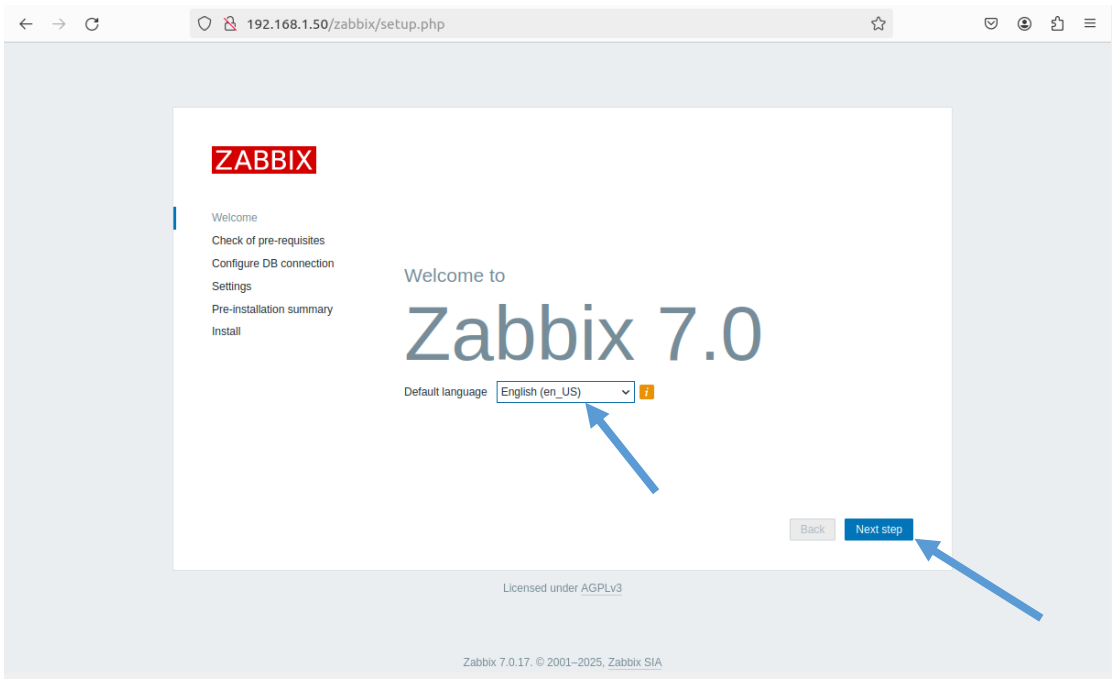
Les paramètres de connexion à la base de données MySQL sont configurés avec précision : **DBHost=localhost** indique que le serveur de base de données est hébergé localement sur la même machine, **DBName=uaszDBzabbix** spécifie le nom de la base de données créée précédemment, **DBUser=univuser** définit l'utilisateur MySQL dédié pour l'accès à la base, et **DBPassword=Password#** contient le mot de passe d'authentification. Cette configuration est essentielle car elle établit le lien entre le processus serveur Zabbix et sa base de données de stockage, permettant ainsi la persistance des données de monitoring, des configurations, des historiques d'événements et des alertes. Sans ces paramètres correctement configurés, le service Zabbix Server ne pourrait pas démarrer ni fonctionner.



Screenshot 48 : Démarrage et vérification des services Zabbix

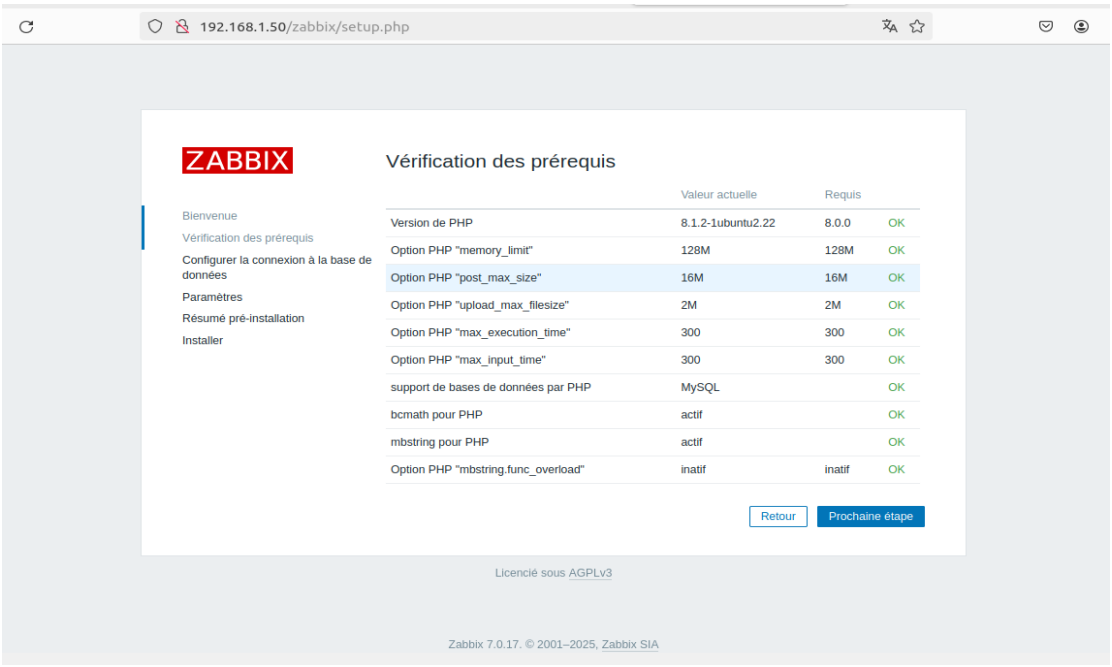
Cette capture montre le démarrage et l'activation des services Zabbix.

4.3.2 Configurations du serveur Zabbix



Screenshot 49 : Interface de configuration Zabbix

Cette capture d'écran présente l'interface web d'installation initiale de Zabbix 7.0, accessible via l'URL **http://192.168.1.50/zabbix/** L'écran d'accueil affiche le logo Zabbix et le message "**Welcome to Zabbix 7.0**", confirmant la version installée du système de monitoring. Un sélecteur de langue permet de choisir la langue par défaut de l'interface, ici on choisit la langue française. Le bouton "**Next step**" en bas à droite permet de progresser vers l'étape suivante de la configuration.



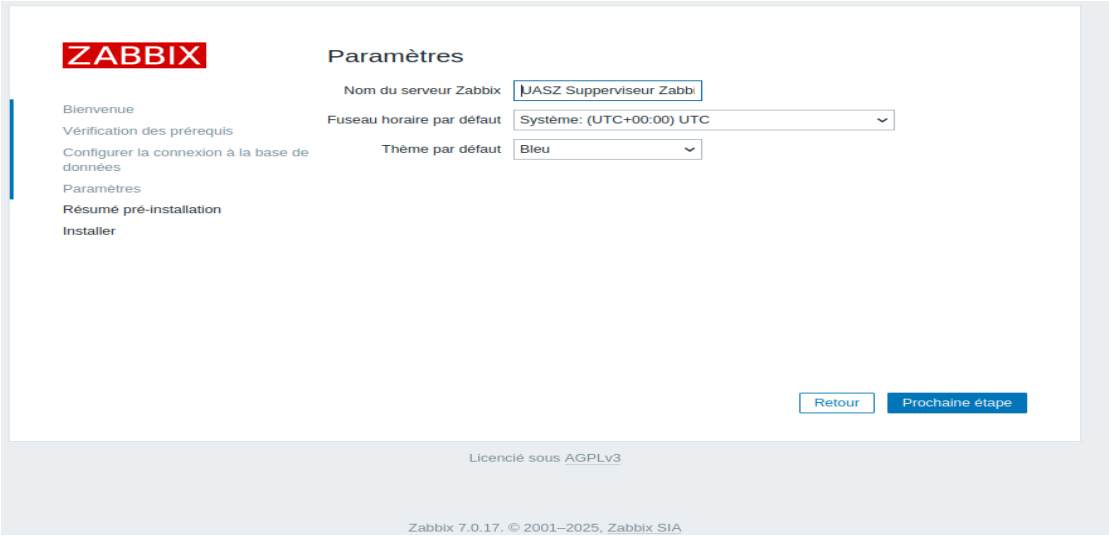
Screenshot 50 : Vérification des prérequis système PHP pour Zabbix

Cette capture d'écran illustre l'étape cruciale de vérification des prérequis techniques ("Vérification des prérequis") dans le processus d'installation de Zabbix. Tous les éléments affichent le statut "OK" en vert, indiquant que l'environnement serveur répond intégralement aux exigences de Zabbix. Cette validation automatique garantit que l'installation pourra se dérouler sans erreur et que le système disposera des ressources nécessaires pour fonctionner de manière optimale.



Screenshot 51 : Configuration des paramètres de connexion à la base de données MySQL

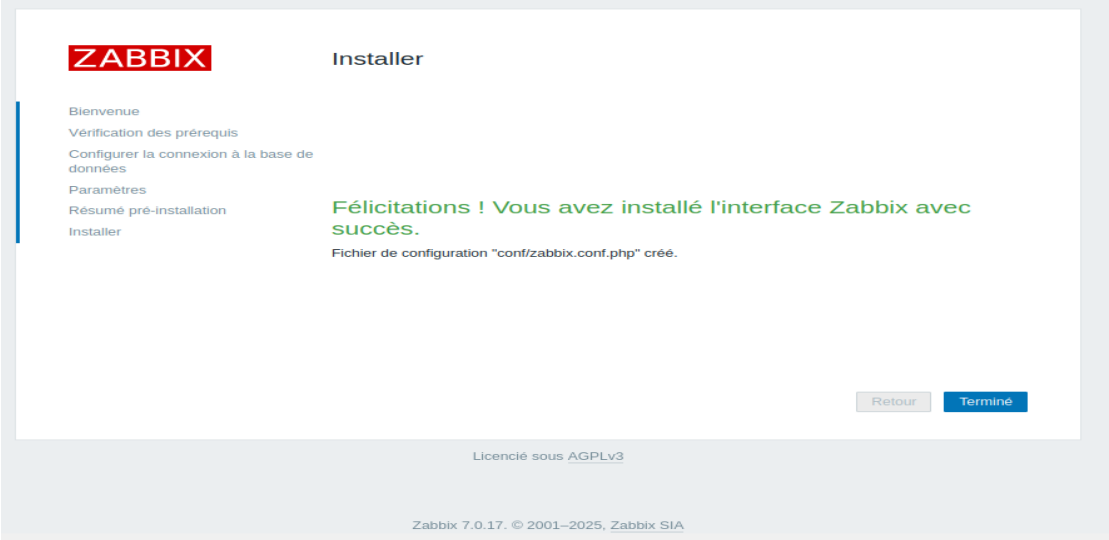
Cette capture d'écran présente l'étape de configuration de la connexion à la base de données dans l'assistant d'installation web de Zabbix. L'interface propose un formulaire détaillé permettant de spécifier les paramètres de connexion : le type de base de données sélectionné est "**MySQL**" (via un menu déroulant), l'hôte est configuré sur "**localhost**" indiquant que la base de données réside sur le même serveur, le port est laissé à "**0**" pour utiliser le port MySQL par défaut (**3306**), et le nom de la base de données est renseigné comme "**uaszDBzabbix**", correspondant à la base créée précédemment. Les identifiants d'authentification incluent l'utilisateur "**univuser**" et son mot de passe masqué par des points pour des raisons de sécurité.



Screenshot 52 : Configuration des paramètres généraux du serveur Zabbix

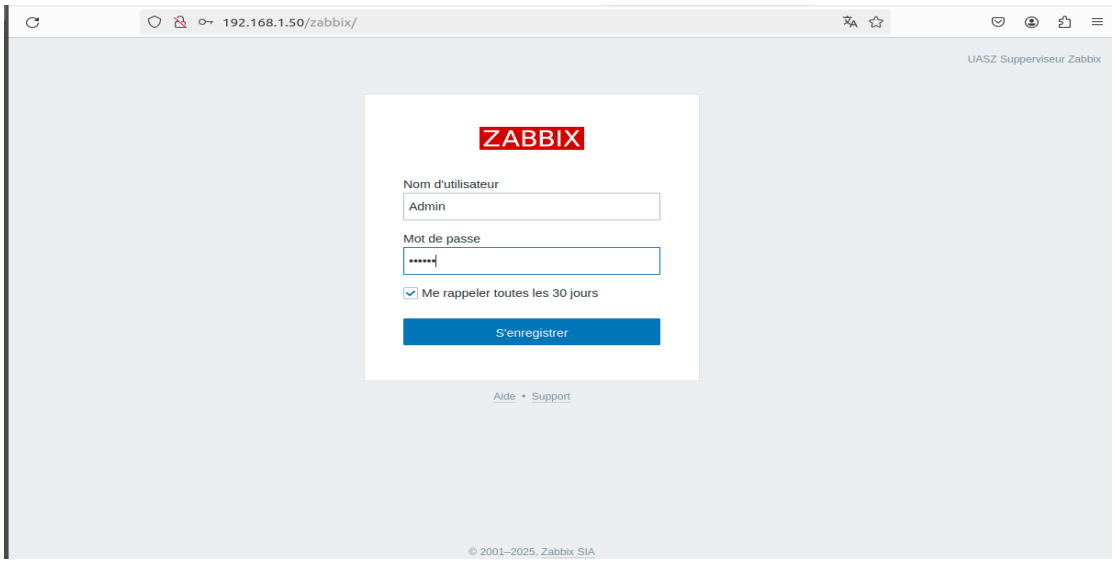
Cette capture d'écran illustre l'étape de configuration des paramètres généraux ("**Paramètres**") de l'installation Zabbix. Trois paramètres principaux sont configurés : premièrement, le nom du serveur Zabbix est défini comme "**UASZ Superviseur Zabbix**", permettant d'identifier clairement cette instance de monitoring dans le

contexte de l'Université Assane Seck de Ziguinchor ; deuxièmement, le fuseau horaire par défaut est configuré sur "**Système: (UTC+00:00) UTC**", ce qui détermine l'horodatage des événements et des données collectées; troisièmement, le thème visuel par défaut de l'interface est réglé sur "**Bleu**", offrant ainsi une apparence cohérente et professionnelle à l'interface web. Le nom du serveur apparaîtra notamment dans les titres de pages, les notifications par email et les rapports, facilitant l'identification de la source des alertes dans un environnement multi-serveurs.



Screenshot 53 : Confirmation de l'installation réussie de l'interface Zabbix

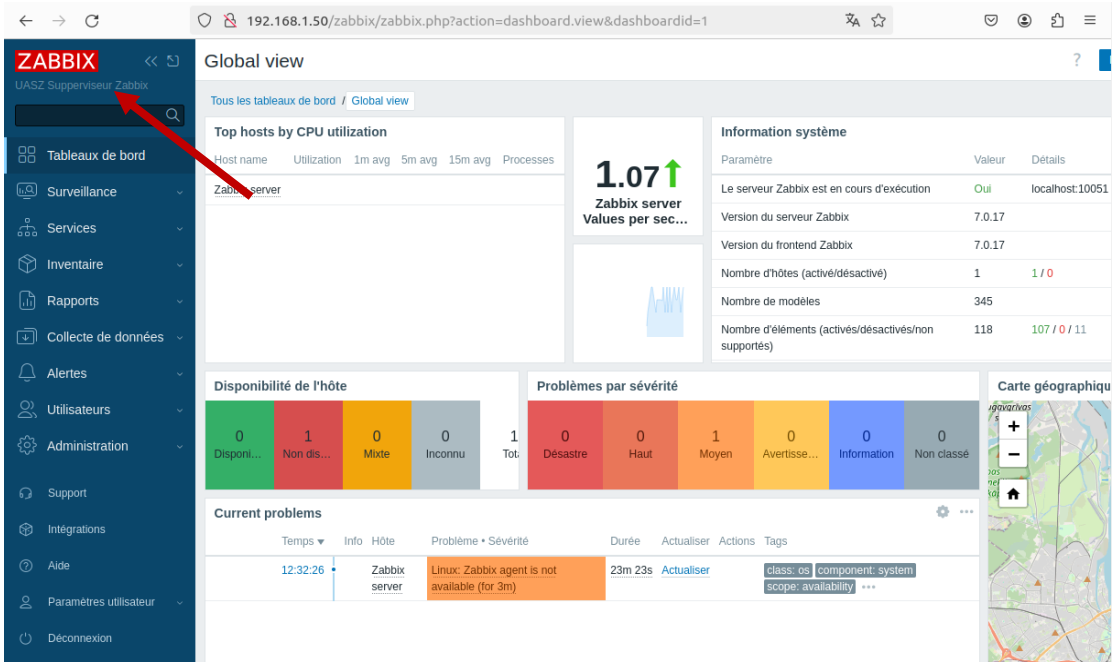
Cette capture d'écran confirme la réussite de l'installation de l'interface web Zabbix. Le message de confirmation affiché en vert, "Félicitations ! Vous avez installé l'interface Zabbix avec succès", indique que toutes les étapes de configuration ont été complétées sans erreur. Le bouton "**Terminé**" permet de finaliser le processus d'installation et d'accéder à l'écran de connexion de Zabbix.



Screenshot 54 : Page d'authentification à la plateforme Zabbix

À partir de ce moment, l'administrateur pourra se connecter à l'interface Zabbix avec les identifiants par défaut (généralement "**Admin**" avec le mot de passe "**zabbix**") pour commencer à configurer les hôtes à superviser, définir les templates de monitoring, créer des triggers d'alerte et personnaliser les tableaux de bord de visualisation.

NB : 192.168.1.50 est l'adresse IP du serveur hébergeant Zabbix-server



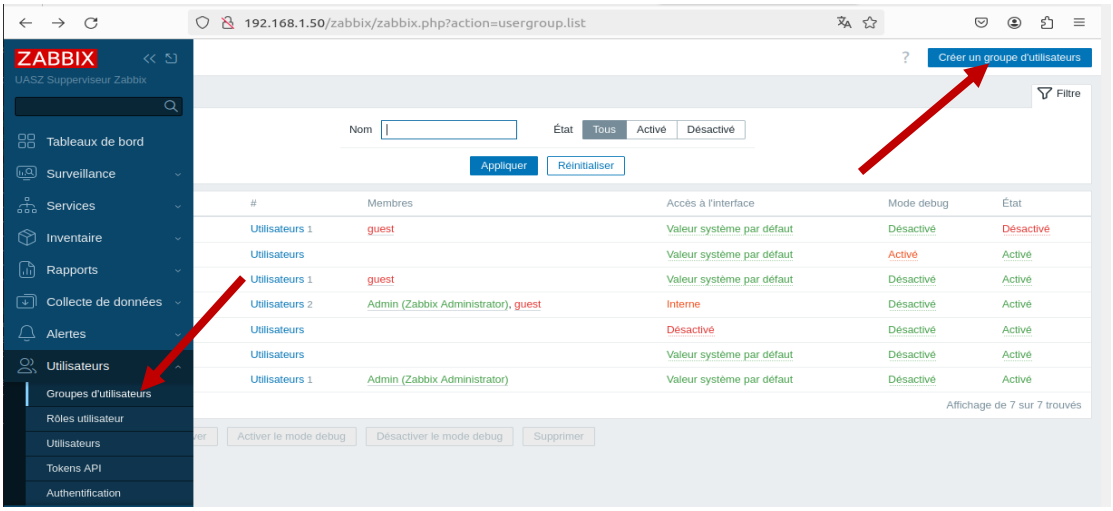
Screenshot 55 : Interface de monitoring Zabbix après authentification

L'interface affiche le nom personnalisé du serveur "**UASZ Superviseur Zabbix**" en haut à gauche, confirmant la configuration effectuée lors de l'installation. Le menu latéral gauche donne accès aux différentes sections fonctionnelles : Tableaux de bord, Surveillance, Services, Inventaire, Rapports, Collecte de données, Alertes, Utilisateurs, Administration, Support, Intégrations, Aide et Paramètres utilisateur.

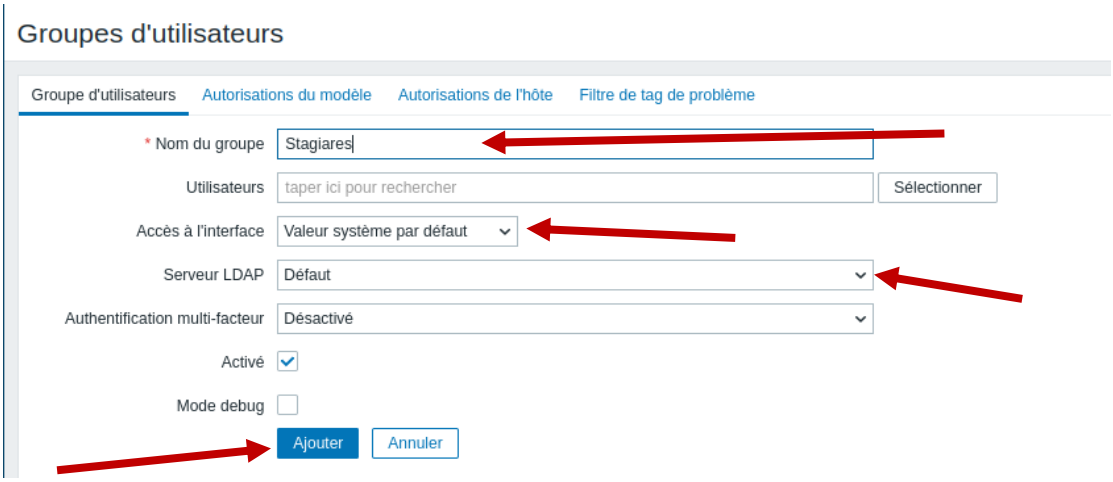
Une fois l'installation terminée, l'interface web de Zabbix a été configurée afin de permettre l'accès sécurisé à la plateforme de supervision et la gestion des hôtes, des templates et des alertes.

4.3.3 Ajout des utilisateurs Zabbix

- **Création du groupe Stagiaires**



Screenshot 56 : Accès à la gestion des groupes



Screenshot 57 : Création du groupe Stagiaires

Étape 1 : Accès à la gestion des groupes Depuis le tableau de bord principal de Zabbix, naviguer dans le menu latéral gauche vers **Utilisateurs** > **Groupes d'utilisateurs**. L'interface affiche alors la liste des groupes existants avec leurs paramètres (membres, accès à l'interface, mode debug, état). Cliquer sur le bouton bleu **"Créer un groupe d'utilisateurs"** situé en haut à droite de l'écran pour initier le processus de création.

Étape 2 : Configuration du nom du groupe Dans le formulaire qui s'affiche, saisir **"Stagiaires"** dans le champ obligatoire **"Nom du groupe"** (marqué par un astérisque rouge). Ce nom identifiera le groupe dans l'ensemble du système et apparaîtra dans les listes déroulantes de sélection de groupes lors de la création d'utilisateurs ou l'attribution de permissions.

Étape 3 : Configuration de l'accès à l'interface Dans le menu déroulant **"Accès à l'interface"**, conserver la valeur **"Valeur système par défaut"**. Cette option permet aux membres du groupe d'hériter automatiquement des paramètres d'interface définis

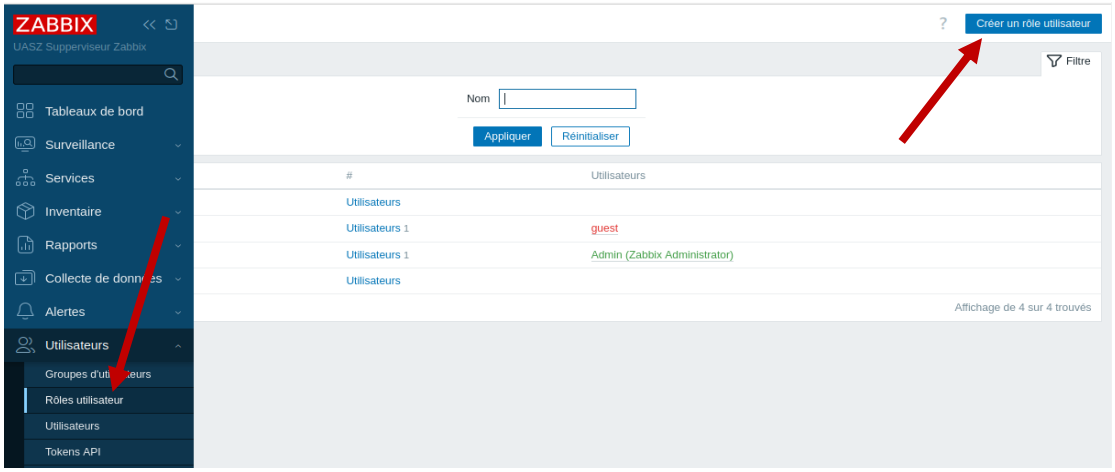
au niveau système, facilitant ainsi la gestion centralisée des préférences d'affichage.

Étape 4 : Configuration du serveur LDAP Le paramètre **"Serveur LDAP"** doit être configuré sur **"Défaut"**. Bien que non utilisé dans une configuration locale standard, ce paramètre prépare le système pour une éventuelle intégration future avec un annuaire LDAP centralisé, permettant l'authentification unifiée des stagiaires via Active Directory ou OpenLDAP.

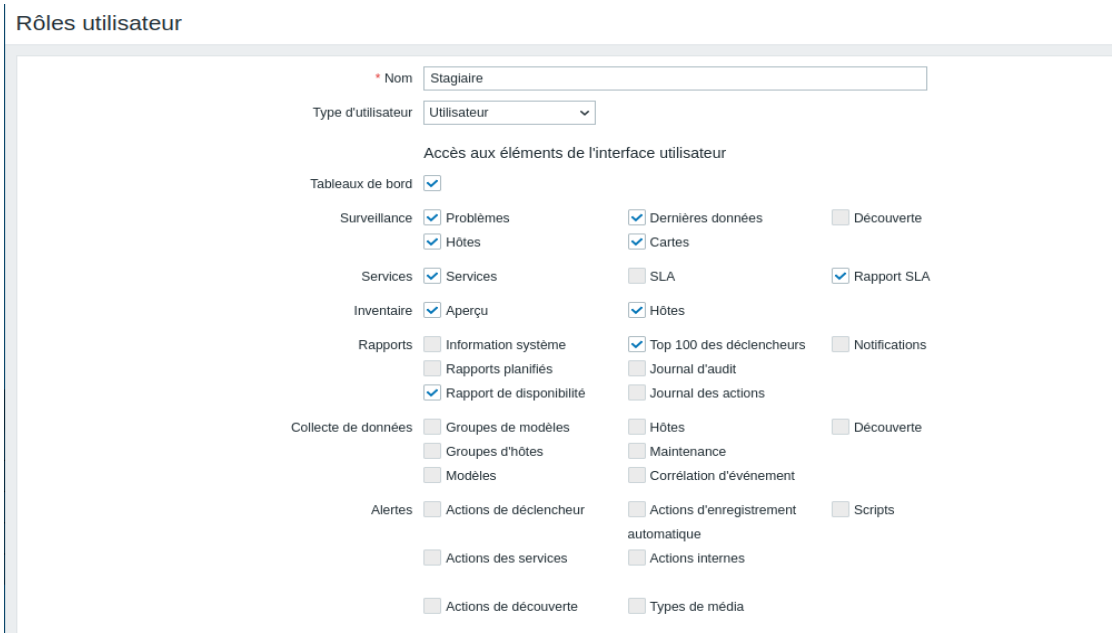
Étape 5 : Activation du groupe Cocher la case **"Activé"** pour rendre le groupe immédiatement opérationnel. Un groupe désactivé reste présent dans la base de données mais ses membres ne peuvent pas se connecter au système.

Étape 6 : Validation et enregistrement Cliquer sur le bouton bleu **"Ajouter"** en bas du formulaire pour créer le groupe. Le système enregistre la configuration et redirige vers la liste des groupes d'utilisateurs où le nouveau groupe "Stagiaires" apparaîtra avec le statut "Activé". Le bouton **"Annuler"** permet d'abandonner la création sans sauvegarder les modifications.

- **Création des rôles**



Screenshot 58 : Accès à la gestion des rôles



Screenshot 59 : Création du rôle Stagiaire

Étape 1 : Accès à la gestion des rôles Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Utilisateurs** > **Rôles utilisateur**. La page affiche la liste des rôles existants avec leurs utilisateurs associés (Utilisateurs, Utilisateurs 1 avec guest, Utilisateurs 1 avec Admin). Cliquer sur le bouton bleu **"Créer un rôle utilisateur"** situé en haut à droite de l'écran pour accéder au formulaire de création.

Étape 2 : Configuration du nom du rôle Dans le formulaire "Rôles utilisateur", saisir **"Stagiaire"** dans le champ obligatoire **"Nom"** (marqué par un astérisque rouge). Ce nom identifiera le profil de permissions qui sera attribué aux membres du groupe Stagiaires.

Étape 3 : Définition du type d'utilisateur Dans le menu déroulant **"Type d'utilisateur"**, sélectionner **"Utilisateur"**. Ce type offre un niveau de privilèges intermédiaire, permettant la consultation et le monitoring sans accès aux fonctions d'administration ou de configuration avancée du système. Les autres options disponibles sont généralement "Super Admin" et "Admin".

Étape 4 : Configuration de l'accès aux tableaux de bord Dans la section **"Accès aux éléments de l'interface utilisateur"**, cocher la case **"Tableaux de bord"**. Cette permission permet aux stagiaires de visualiser les tableaux de bord de monitoring, élément essentiel pour suivre l'état des systèmes supervisés.

Étape 5 : Configuration de l'accès à la surveillance Pour la section **"Surveillance"**, cocher les cases suivantes :

- **"Problèmes"** : permet de consulter la liste des incidents et alertes actifs
- **"Hôtes"** : donne accès à la visualisation de la liste des serveurs et équipements monitorés
- **"Dernières données"** : autorise la consultation des valeurs collectées récemment
- **"Cartes"** : permet de visualiser les cartes réseau et les topologies d'infrastructure.

Étape 6 : Configuration de l'accès aux services Cocher les cases suivantes dans la section **"Services"** :

- **"Services"** : permet de consulter la liste des services métiers définis
- **"Rapport SLA"** : autorise la visualisation des rapports de niveau de service

Laisser **"SLA"** décochée pour restreindre la configuration des objectifs de niveau de service aux administrateurs.

Étape 7 : Configuration de l'accès à l'inventaire Dans la section **"Inventaire"**, cocher :

- **"Aperçu"** : permet de visualiser une vue synthétique de l'inventaire matériel
- **"Hôtes"** : donne accès aux informations détaillées d'inventaire des équipements

Étape 8 : Configuration de l'accès aux rapports Pour la section **"Rapports"**, cocher les permissions suivantes :

- **"Rapport de disponibilité"** : permet de consulter les statistiques de disponibilité des hôtes
- **"Top 100 des déclencheurs"** : autorise la visualisation des triggers les plus fréquemment activés

Laisser décoché **"Information système"**, **"Notifications"**, **"Journal d'audit"**, et **"Journal des actions"** car ces éléments relèvent de l'administration système.

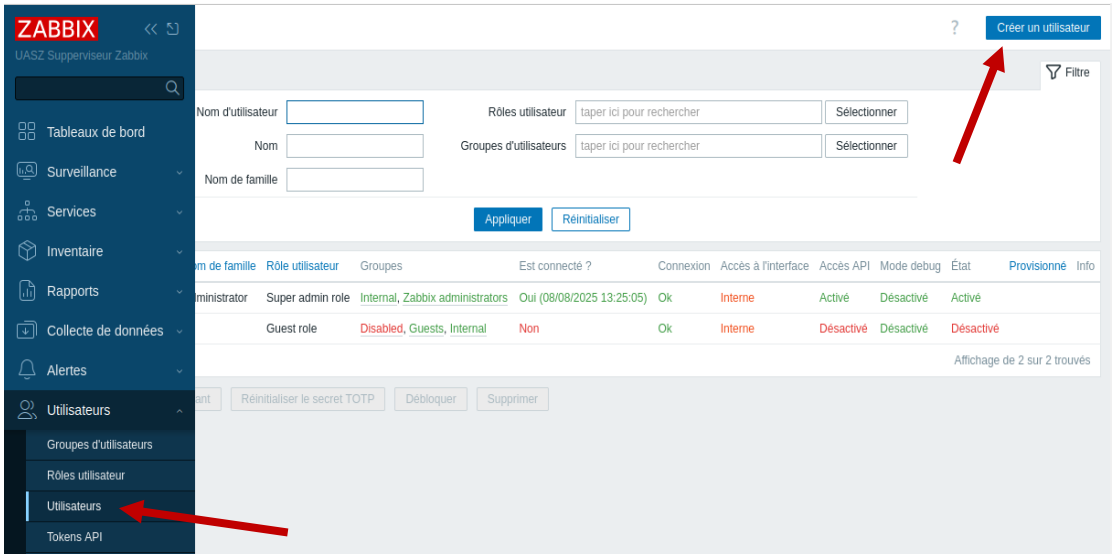
Étape 9 : Restriction de l'accès à la collecte de données Dans la section **"Collecte de données"**, laisser **toutes les cases décochées** : Groupes de modèles, Groupes d'hôtes, Modèles, Hôtes, Maintenance, Corrélation d'événement, Découverte.
Ces fonctions de configuration ne doivent pas être accessibles aux stagiaires qui ont uniquement un rôle de consultation.

Étape 10 : Restriction de l'accès aux alertes Dans la section **"Alertes"**, laisser **toutes les cases décochées** : Actions de déclencheur, Actions d'enregistrement automatique, Actions internes, Actions des services, Actions de découverte, Types de média, Scripts.
La gestion des alertes et des actions automatisées est réservée aux administrateurs.

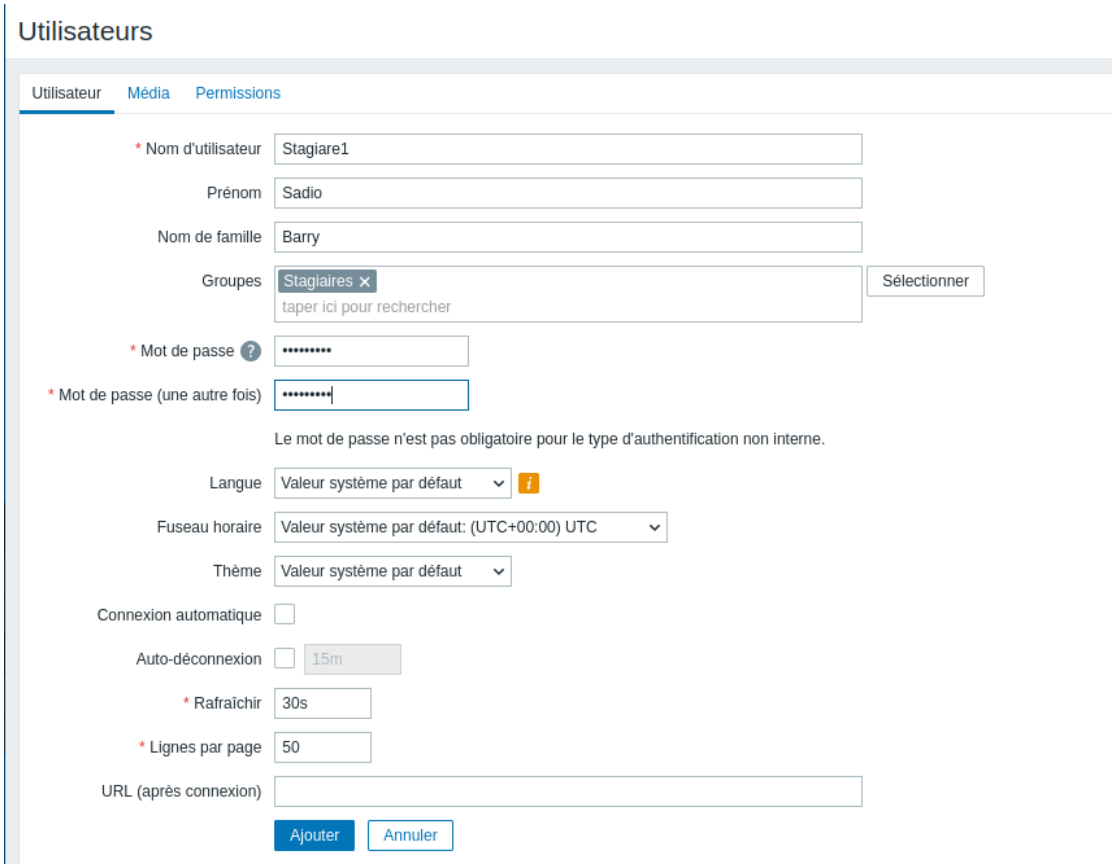
Étape 11 : Validation et enregistrement Faire défiler le formulaire jusqu'en bas et cliquer sur le bouton **"Ajouter"** pour créer le rôle. Le système enregistre la

configuration et redirige vers la liste des rôles utilisateurs où le nouveau rôle "**Stagiaire**" apparaîtra dans le tableau.

- **Création de l'utilisateur Stagiaire**



Screenshot 60 : Accès à la gestion des utilisateurs



Screenshot 61 : Création de l'utilisateur Stagiaire

Étape 1 : Accès à la gestion des utilisateurs Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Utilisateurs** > **Utilisateurs**. Cliquer sur le bouton bleu **"Créer un utilisateur"** situé en haut à droite de l'écran pour accéder au formulaire de création.

Étape 2 : Saisie du nom d'utilisateur Dans l'onglet **"Utilisateur"** (premier onglet actif parmi Utilisateur, Média, Permissions), renseigner le champ obligatoire **"Nom d'utilisateur"** avec la valeur **"Stagiaire1"**. Ce nom servira d'identifiant de connexion unique dans le système Zabbix.

Étape 3 : Dans le champ **"Prénom"**, on saisit **"Sadio"**. Dans le champ **"Nom de famille"**, on saisit **"Barry"**. Ces informations constituent l'identité complète de l'utilisateur dans le système, permettent d'identifier personnellement l'utilisateur et apparaîtra dans les logs d'audit et les notifications.

Étape 4 : Attribution au groupe d'utilisateurs Dans le champ **"Groupes"**, cliquer sur le bouton **"Sélectionner"** pour parcourir la liste des groupes disponibles et sélectionner "Stagiaires". Cette affectation détermine les permissions de base héritées du groupe.

Étape 6 : Définition du mot de passe, Dans le champ obligatoire **"Mot de passe"**, saisir un mot de passe sécurisé. **Confirmation du mot de passe** Dans le champ obligatoire **"Mot de passe (une autre fois)"**, ressaisir exactement le même mot de passe pour validation.

Étape 8 : Configuration de la langue Dans le menu déroulant **"Langue"**, conserver **"Valeur système par défaut"** pour que l'utilisateur hérite de la langue définie au niveau système, ou sélectionner une langue spécifique (français, anglais, etc.).

Étape 9 : Configuration du fuseau horaire Dans le menu déroulant **"Fuseau horaire"**, laisser **"Valeur système par défaut : (UTC+00 :00) UTC"** ou sélectionner le fuseau horaire approprié pour l'utilisateur (par exemple, "Africa/Dakar" pour le Sénégal, UTC+0). Ce paramètre affectera l'affichage des horodatages dans l'interface.

Étape 10 : Configuration du thème visuel Dans le menu déroulant **"Thème"**, conserver **"Valeur système par défaut"** pour utiliser le thème défini globalement (Bleu dans ce cas), ou permettre à l'utilisateur de choisir son propre thème parmi les options disponibles (Bleu, Sombre, Clair).

Étape 11 : Validation et enregistrement Faire défiler jusqu'en bas du formulaire et cliquer sur le bouton bleu **"Ajouter"** pour créer le compte utilisateur. Le bouton **"Annuler"** permet d'abandonner la création sans sauvegarder. Le système enregistre toutes les informations et redirige vers la liste des utilisateurs.

4.3.4 Déploiement des agents Zabbix sur les serveurs supervisés

Après la mise en place du serveur Zabbix, les **agents Zabbix** ont été installés sur :

- Le **serveur Zabbix**.
- Le **serveur Moodle** (serveur web et base de données),
- Le **serveur Backup**.

L'agent Zabbix permet de collecter localement les métriques système (CPU, mémoire, disque, processus, services, etc.) et de les transmettre au serveur Zabbix.
Chaque agent a été configuré avec : l'adresse IP du serveur Zabbix, Un nom d'hôte unique et les paramètres de sécurité nécessaires (ports, droits d'accès).

a) Le serveur Moodle

```
alassane@alassane:/$ sudo apt install zabbix-agent
[sudo] Mot de passe de alassane :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libevent-core-2.1-7 libevent-pthreads-2.1-7 libmecab2 libprotobuf-lite23
  libsnappy1v5 libwpe-1.0-1 libwpebackend-fdo-1.0-1 mecab-ipadic
  mecab-ipadic-utf8 mecab-utils
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
Les NOUVEAUX paquets suivants seront installés :
  zabbix-agent
0 mis à jour, 1 nouvellement installés, 0 à enlever et 39 non mis à jour.
Il est nécessaire de prendre 298 ko dans les archives.
Après cette opération, 945 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 zabbix-agent amd64 1:5.0.17+dfsg-1 [298 kB]
298 ko réceptionnés en 1s (239 ko/s)
```

Screenshot 62 : Installation de l'agent Zabbix sur le serveur Web et Base de données

```
  GNU nano 6.2                                  /etc/zabbix/zabbix_agentd.conf
# Default:
# Server=

Server=192.168.1.50
ListenPort=10050
ServerActive=192.168.1.50
Hostname=moodle-agent
### Option: ListenPort
#       Agent will listen on this port for connections from the server.
#
```

Screenshot 63 : Configuration agent Zabbix sur Moodle

Cette capture d'écran montre la configuration du fichier **/etc/zabbix/zabbix_agentd.conf** sur le serveur hébergeant la plateforme Moodle. L'agent Zabbix est un processus léger qui s'exécute sur les machines à superviser et collecte les métriques système pour les transmettre au serveur Zabbix central. Quatre paramètres essentiels ont été configurés :

- **Server=192.168.1.50** définit l'adresse IP du serveur Zabbix autorisé à se connecter à cet agent pour récupérer les données de monitoring (mode passif) ;
- **ListenPort=10050** spécifie le port TCP sur lequel l'agent écoute les connexions entrantes du serveur Zabbix, utilisant le port standard par défaut ;
- **ServerActive=192.168.1.50** indique l'adresse du serveur Zabbix vers lequel l'agent enverra proactivement les données collectées (mode actif), permettant ainsi une communication bidirectionnelle ;

- **Hostname=moodle-agent** définit le nom d'hôte unique qui identifiera cette machine dans l'interface Zabbix, facilitant son repérage parmi l'ensemble des équipements supervisés.

```
alassane@alassane:~$ sudo cat /etc/zabbix/psk.key
[sudo] Mot de passe de alassane :
ea918c023351dc6630e0aa94325dbf3e6cca89325828b93d50000cb2960abb85
alassane@alassane:~$
```

Screenshot 64 : Clé PSK générée

La clé PSK est utilisée pour sécuriser les communications entre l'agent Zabbix installé sur le serveur hébergeant Moodle et MySQL et le serveur Zabbix central. Le contenu constitue un secret partagé qui permet de chiffrer l'ensemble des échanges de données entre l'agent et le serveur, garantissant ainsi la confidentialité et l'intégrité des métriques transmises sur le réseau. Cette clé est générée via la commande **openssl rand -hex 32 | sudo tee /etc/zabbix/psk.key.** Le fichier PSK doit être protégé avec des permissions restrictives (**sudo chmod 600 /etc/zabbix/psk.key**) accessibles uniquement par l'utilisateur Zabbix (**sudo chown zabbix :zabbix /etc/zabbix/psk.key**), empêchant tout accès non autorisé à cette information critique de sécurité.

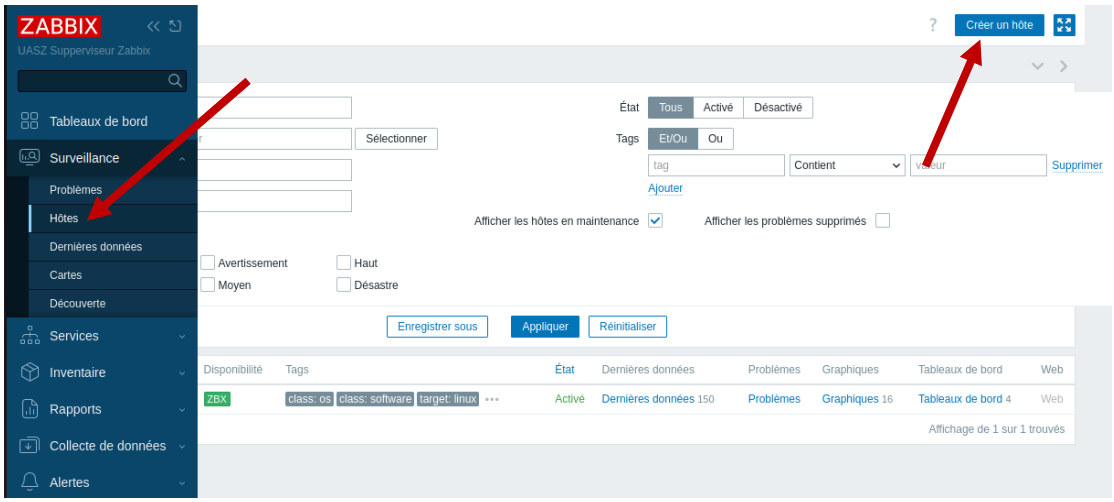
```
# Mandatory: yes, if TLS certificate or PSK parameters are defined (even for 'unencrypted' connection)
# Default:
TLSConnect=psk
TLSAccept=psk
TLSPSKIdentity=moodle@123
TLSPSKFile=/etc/zabbix/psk.key


### Option: TLSAccept
```

Screenshot 65 : Configuration TLS/PSK sur le serveur Web et base et de données

Quatre paramètres sont configurés pour activer et sécuriser les communications :

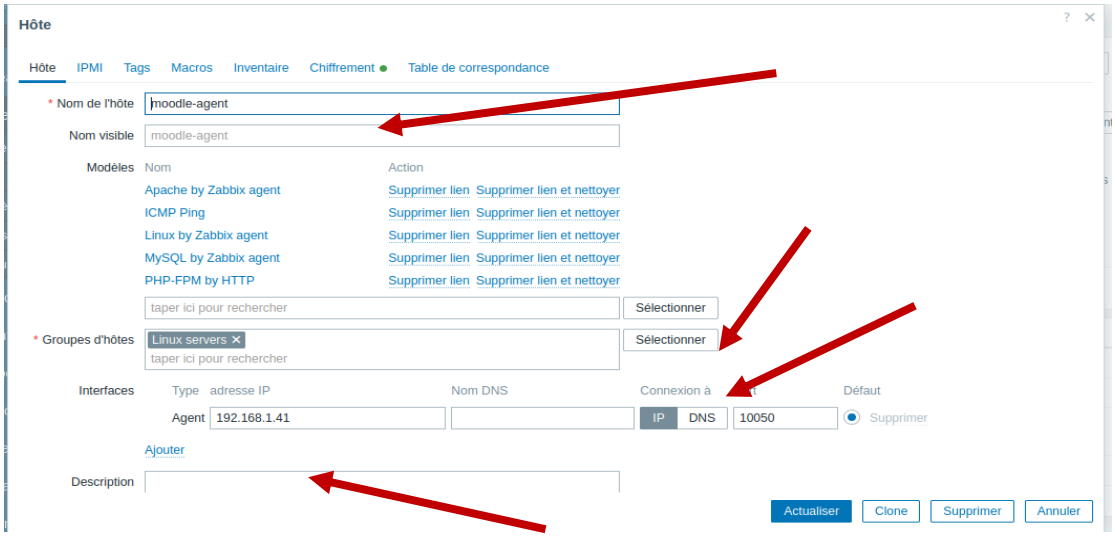
- **TLSConnect=psk** spécifie que l'agent utilisera le mode PSK pour les connexions sortantes actives vers le serveur Zabbix, garantissant que toutes les données envoyées proactivement sont chiffrées ;
- **TLSAccept=psk** indique que l'agent n'acceptera que les connexions entrantes (mode passif) utilisant également le chiffrement PSK, refusant ainsi toute tentative de communication non sécurisée ;
- **TLSPSKIdentity=moodle@123** définit un identifiant unique associé à cette clé PSK, permettant au serveur Zabbix de reconnaître et d'authentifier cet agent spécifique parmi tous les agents supervisés (cet identifiant doit correspondre exactement à celui configuré dans l'interface web Zabbix);
- **TLSPSKFile=/etc/zabbix/psk.key** indique le chemin absolu vers le fichier contenant la clé pré-partagée hexadécimale.



Screenshot 66 : Accès gestion des hôtes

Étape 1 : Accès à la gestion des hôtes

Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Surveillance** > **Hôtes**. La page affiche la liste des hôtes supervisés existants. Cliquer sur le bouton bleu **"Créer un hôte"** situé en haut à droite de l'écran pour ouvrir le formulaire de création.



Screenshot 67 : Ajout de l'hôte agent Moodle

Étape 2 : Configuration du nom de l'hôte

Dans le champ obligatoire **"Nom de l'hôte"**, saisir **"moodle-agent"**. Ce nom doit correspondre exactement au paramètre Hostname défini dans le fichier de configuration de l'agent Zabbix (**/etc/zabbix/zabbix_agentd.conf)** pour assurer la reconnaissance mutuelle entre le serveur et l'agent.

Étape 3 : Sélection des modèles (templates)

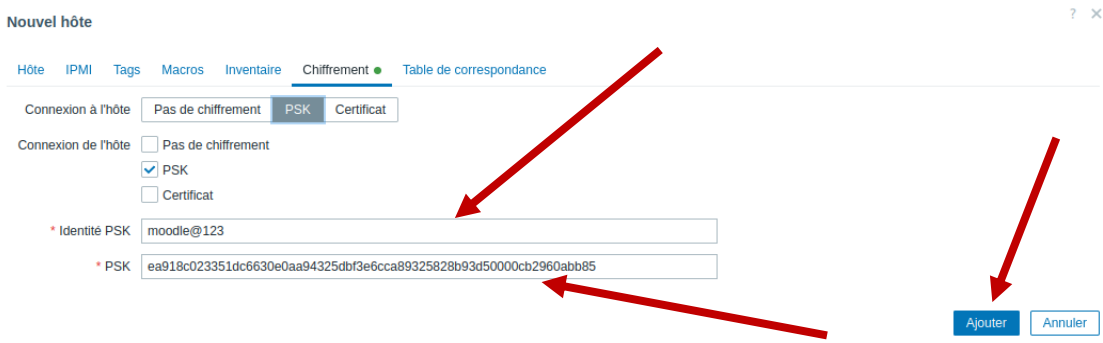
Dans le champ **"Modèles"**, les templates **Linux by Zabbix agent, Apache by Zabbix agent, ICMP ping, MySQL by Zabbix agent, PHP-FPM by HTTP** sont

sélectionnés. Pour ajouter d'autres templates, cliquer sur le bouton **"Sélectionner"**.

Étape 4 : Attribution aux groupes d'hôtes
Dans le champ obligatoire **"Groupes d'hôtes"**, le groupe **"Linux servers"** est sélectionné. Ce regroupement logique facilite l'organisation et le filtrage des hôtes dans l'interface. Pour ajouter d'autres groupes utiliser la barre de recherche ou le bouton **"Sélectionner"**.

Étape 5 : Configuration de l'interface Agent
Dans la section **"Interfaces"**, l'agent est configuré avec les paramètres suivants :

- **Type** : Agent (par défaut pour les agents Zabbix)
- **Adresse IP** : **192.168.1.41** (l'adresse IP du serveur Moodle où l'agent est installé)
- **Connexion à** : **IP** sélectionné (indique que Zabbix utilisera l'adresse IP pour se connecter plutôt que le nom DNS)
- **Port** : **10050** (port standard d'écoute de l'agent Zabbix, doit correspondre au paramètre ListenPort dans zabbix_agentd.conf)



Screenshot 68 : Configuration du chiffrement sur l'agent Moodle

Étape 6 : Accès à l'onglet Chiffrement
Cliquer sur l'onglet **"Chiffrement"** dans la barre d'onglets en haut du formulaire. Cet onglet contient les paramètres de sécurisation des communications entre le serveur Zabbix et l'agent installé sur le serveur Moodle.

Étape 7 : Configuration de la connexion à l'hôte (chiffrement sortant)
Dans la section **"Connexion à l'hôte"**, trois options sont disponibles :

- **Pas de chiffrement** : communications en clair (non sécurisé)
- **PSK** : chiffrement par clé pré-partagée (recommandé)
- **Certificat** : chiffrement par certificat TLS/SSL

Cliquer sur le bouton **"PSK"** pour activer le mode de chiffrement par clé pré-partagée. Ce choix doit correspondre au paramètre **TLSConnect=psk** configuré dans le fichier zabbix_agentd.conf de l'agent.

Étape 8 : Configuration de la connexion de l'hôte (chiffrement entrant)
Dans la section **"Connexion de l'hôte"** juste en dessous, trois cases à cocher sont disponibles. Cocher la case **"PSK"** pour autoriser l'agent à établir des connexions chiffrées vers le serveur en mode actif. Cette configuration correspond au paramètre **TLSAccept=psk** dans **zabbix_agentd.conf.**

Étape 9 : Saisie de l'identité PSK et de la clé PSK
Dans le champ obligatoire **"Identité PSK"** saisir exactement **"moodle@123"**. Cette valeur doit être strictement identique au paramètre **TLSPSKIdentity=moodle@123** configuré dans le fichier **zabbix_agentd.conf** de l'agent. Cet identifiant permet au serveur Zabbix de sélectionner la bonne clé PSK parmi celles enregistrées pour authentifier cet agent spécifique. Dans le champ obligatoire **"PSK"** copier/coller la clé hexadécimale générée précédemment. Cette clé doit être exactement la même que celle stockée dans le fichier **/etc/zabbix/psk.key** sur le serveur Moodle.

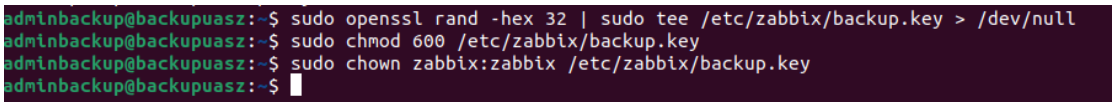
Étape 10 : Validation et ajout de l'hôte
Après avoir vérifié que tous les paramètres sont correctement configurés (nom d'hôte, adresse IP, port, template, groupe, chiffrement PSK avec identité et clé), faire défiler jusqu'en bas du formulaire et cliquer sur le bouton bleu **"Ajouter"** pour créer l'hôte.

Le serveur Moodle est maintenant entièrement intégré dans le système de monitoring Zabbix avec des communications sécurisées par chiffrement PSK, permettant une supervision continue de ses performances système.

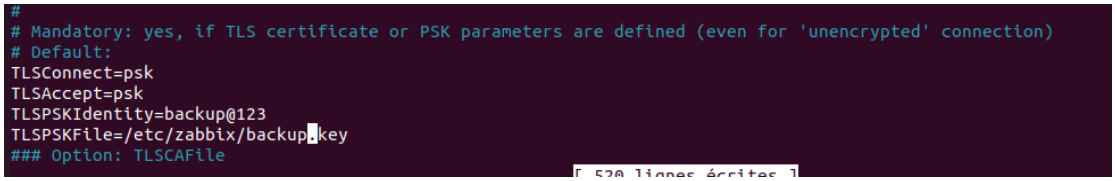
b) Le serveur Backup(sauvegarde)



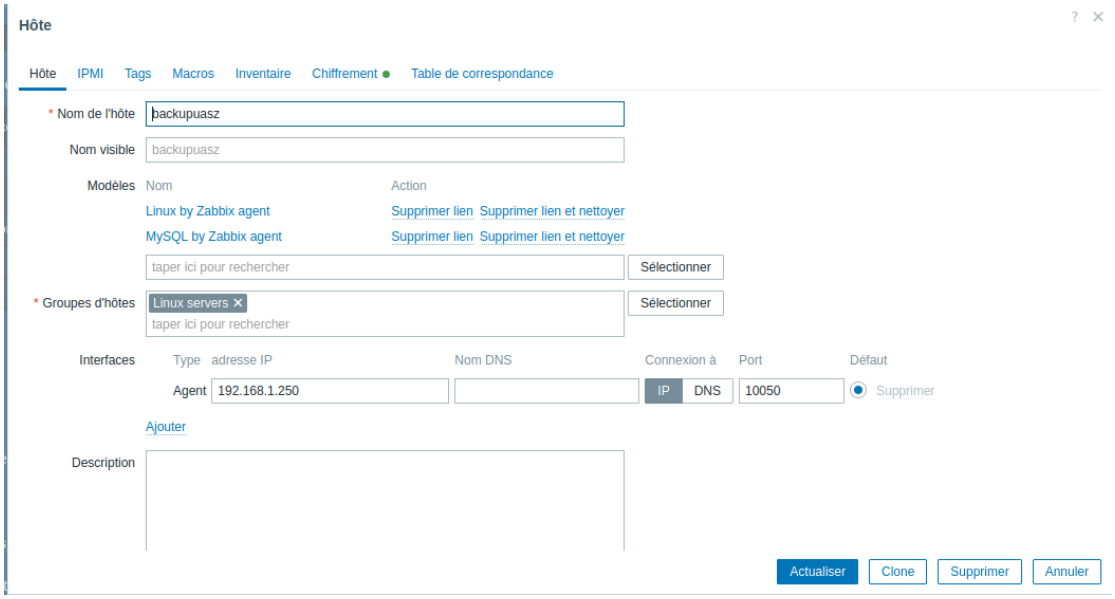
Screenshot 69 : Configuration de l'agent Zabbix sur le serveur backup



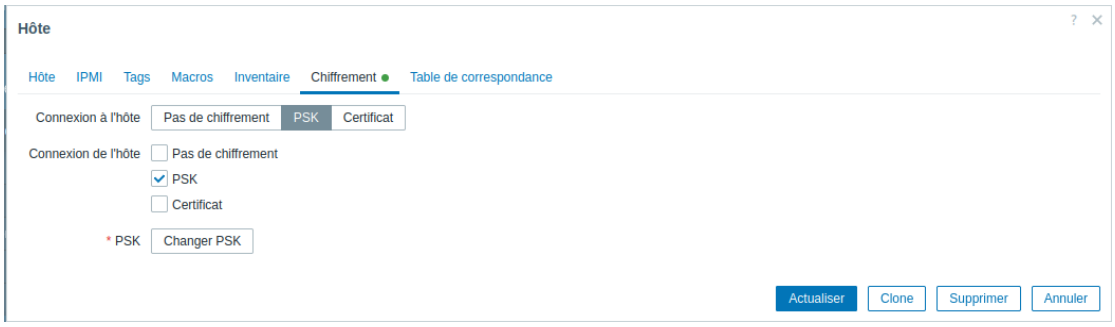
Screenshot 70 : Génération et protection de la clé PSK



Screenshot 71 : Configuration TLS/PSK sur le serveur backup



Screenshot 72 : Ajout de l'hôte backupuasz



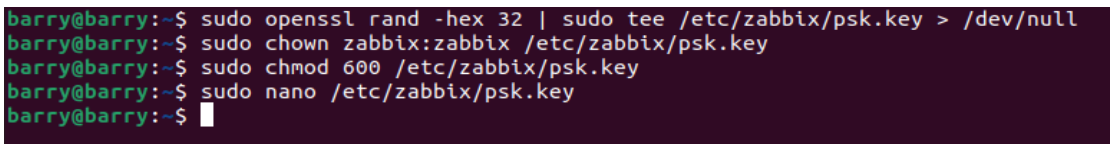
Screenshot 73 : Configuration du chiffrement sur l'agent backupuasz

On procède les mêmes étapes (**Voir les étapes a)**).

c) Le serveur Zabbix



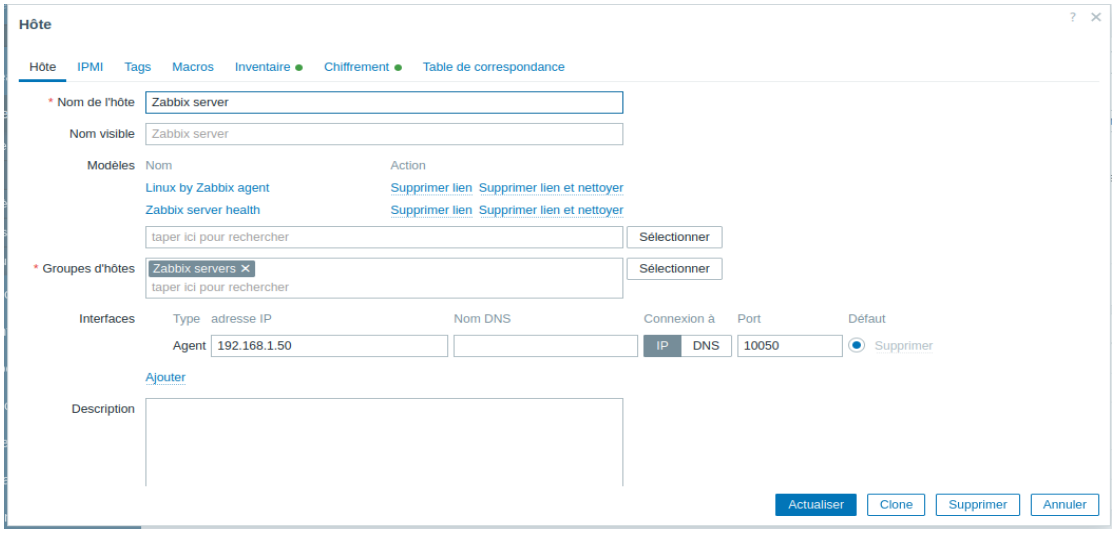
Screenshot 74 : Configuration de l'agent Zabbix sur le serveur Zabbix



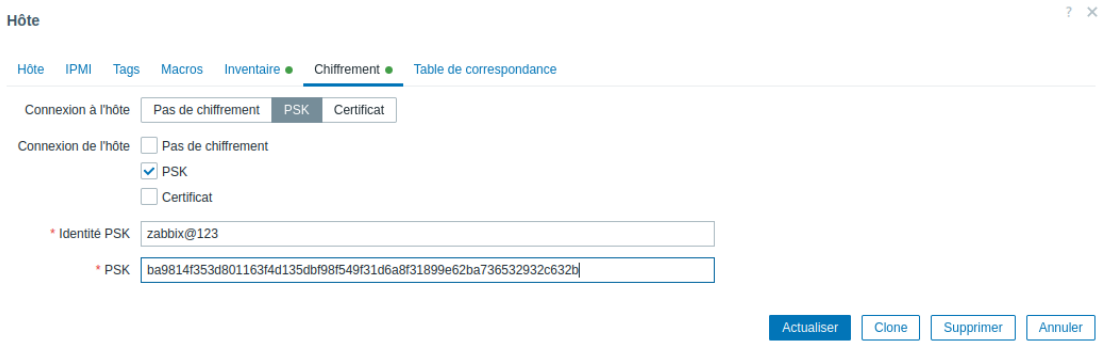
Screenshot 75 : Génération et protection de la clé PSK sur Zabbix

```
 GNU nano 6.2                                    /etc/zabbix/zabbix_agentd.conf
# Mandatory: yes, if TLS certificate or PSK parameters are defined (even for 'unencrypted' connection)
# Default:
TLSConnect=psk
TLSAccept=psk
TLSPSKIdentity=zabbix@123
TLSPSKFile=/etc/zabbix/psk.key
### Option: TLSAccept
```

Screenshot 76 : Configuration TLS/PSK sur le serveur Zabbix



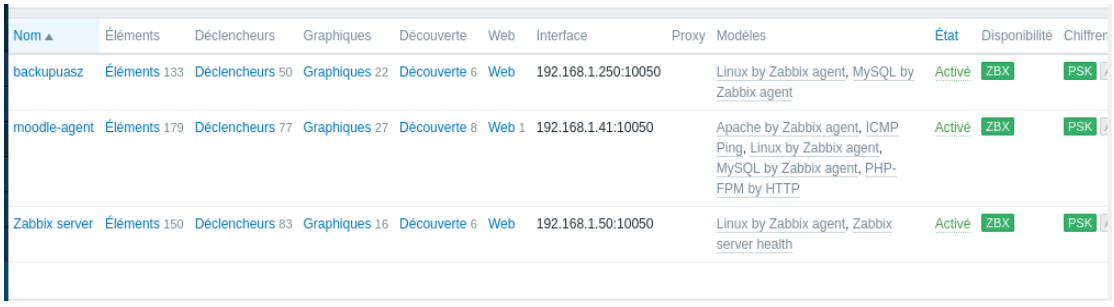
Screenshot 77 : Ajout de l'hôte Zabbix server



Screenshot 78 : Configuration du chiffrement sur l'agent Zabbix server

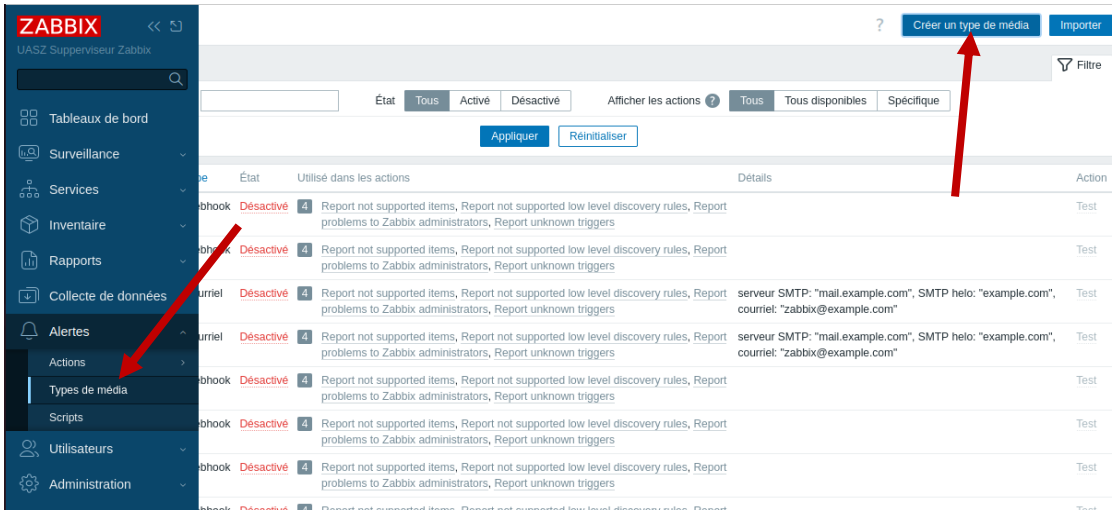
On procède les mêmes étapes que précèdemment **(Voir les étapes a))**

Résultats : Les trois agents sont maintenant disponibles et sécurisés



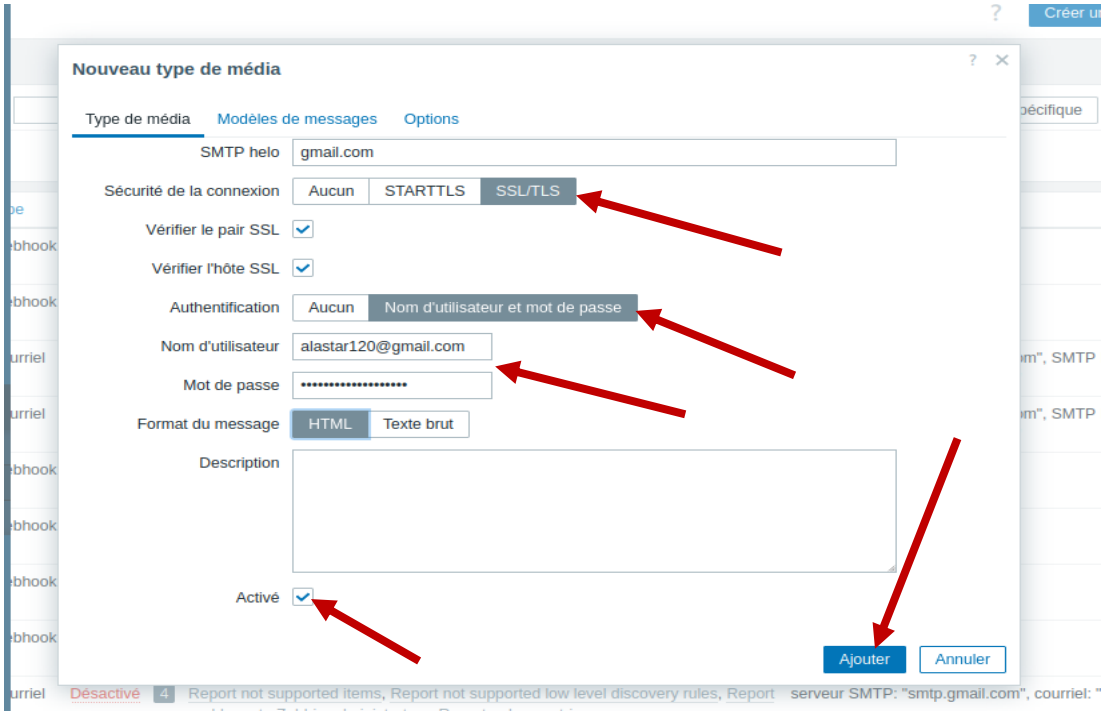
Screenshot 79 : Listes des agents disponibles

4.3.5 Ajout d'un type de Media (Gmail uasz)



Screenshot 80 : Accès à la gestion des types de médias

Étape 1 : Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Alertes** > **Types de média**. La page affiche la liste des types de médias existants. Cliquer sur le bouton bleu **"Créer un type de média"** situé en haut à droite de l'écran pour accéder au formulaire de configuration.



Screenshot 81 : Accès à l'onglet « Type de média »

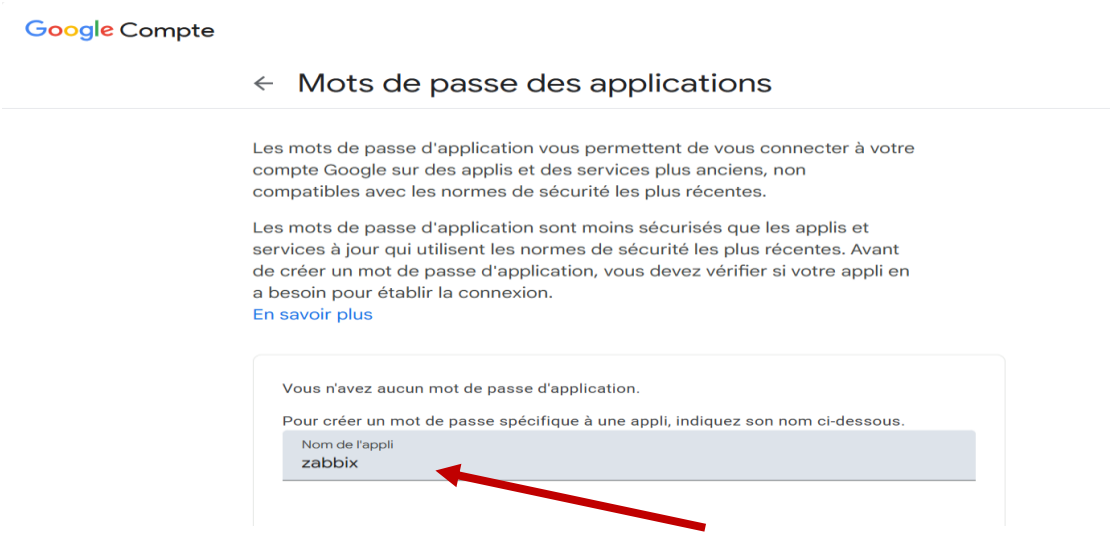
Etape 2 : le premier champ du formulaire est généralement **Nom.** Saisir **Gmail uasz** comme nom pour identifier ce type de média. Dans le champ **Type**, sélectionner **Email** dans le menu déroulant. Dans le champ **SMTP helo,** saisir **gmail.com.** Dans la

section **Sécurité de la connexion**, Sélectionner **SSL/TLS** pour établir une connexion sécurisée directe avec le serveur Gmail. Cette option utilise généralement le port **465** et offre le meilleur niveau de sécurité pour les communications email. Dans la section **Authentification**, sélectionner **Nom d'utilisateur et mot de passe**. Gmail exige une authentification pour envoyer des emails, rendant cette option obligatoire. Dans le champ **Nom d'utilisateur**, saisir l'adresse email complète du compte Gmail UASZ, Cette adresse servira à la fois d'identifiant de connexion SMTP et d'adresse d'expédition par défaut pour les notifications. Dans le champ **Mot de passe**, saisir le mot de passe du compte Gmail.

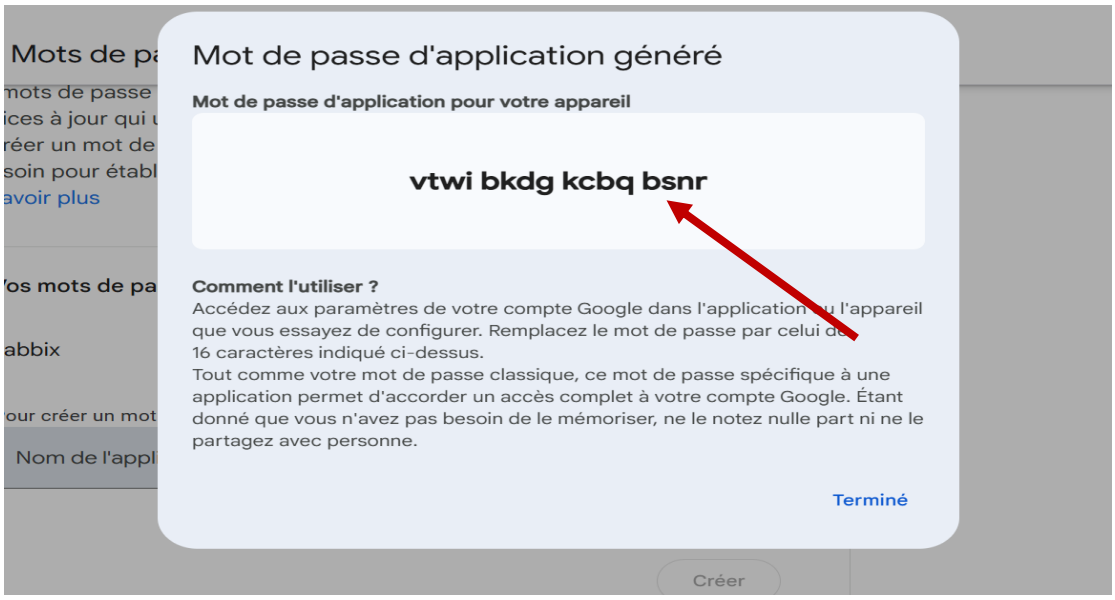
IMPORTANT : Si l'authentification à deux facteurs (2FA) est activée sur le compte Gmail (fortement recommandé), il faut utiliser un **mot de passe d'application** spécifique généré depuis les paramètres de sécurité Google (https://myaccount.google.com/apppasswords), et non le mot de passe principal du compte (**Screenshot 82, 83, 84 ci-dessous**). Cocher la case **Activé** pour rendre ce type de média immédiatement opérationnel.



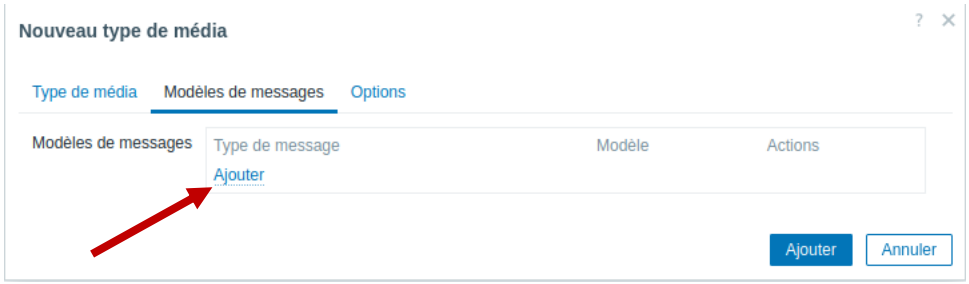
Screenshot 82 : Accès à la page sécurité Google



Screenshot 83 : Saisi du nom de l'application pour générer un mot de passe

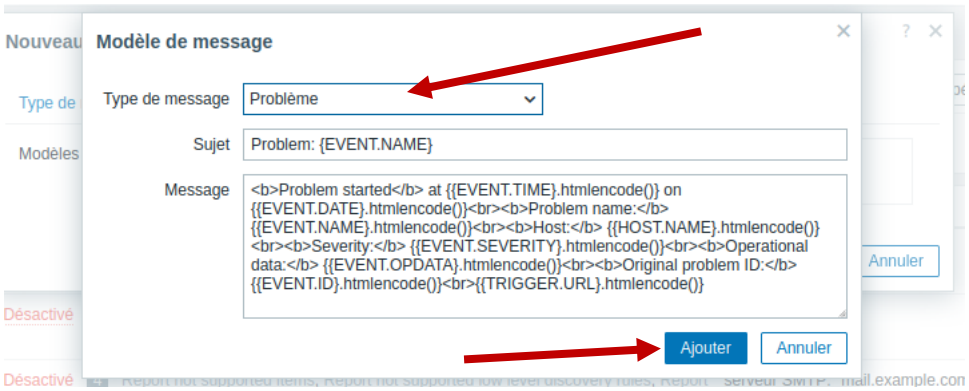


Screenshot 84 : Mot de passe d'application généré



Screenshot 85 : Passage à l'onglet "Modèles de messages"

Etape 3 : Cliquer sur l'onglet **Modèles de messages** en haut de la fenêtre. Cette section permet de personnaliser le contenu des notifications pour différents types d'événements. Un lien **Ajouter** permet de créer un nouveau modèle de message.

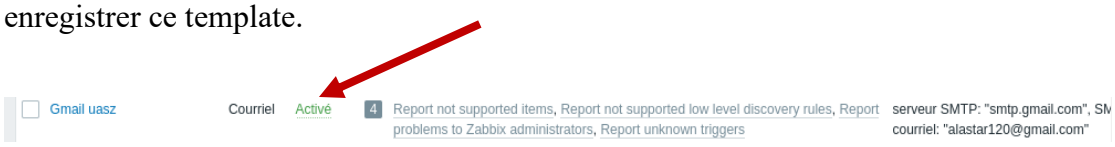


Screenshot 86 : Création d'un modèle de message pour les problèmes

Etape 4 : Dans le menu déroulant **Type de message**, sélectionner **Problème** Les autres types disponibles incluent généralement :

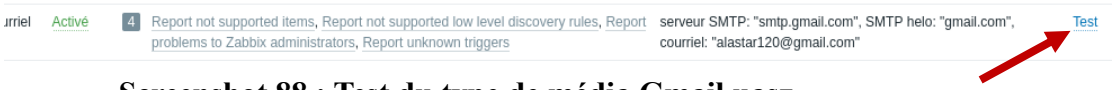
- Problème : notification lors de la détection d'un problème
- Rétablissement du problème : notification lors de la résolution
- Mise à jour du problème : notification lors de la modification d'un problème existant
- Découverte : notification de découverte d'équipement
- Auto-enregistrement : notification d'auto-enregistrement d'agent

Cliquer sur le bouton **Ajouter** en bas de la fenêtre **Modèle de message** pour enregistrer ce template.

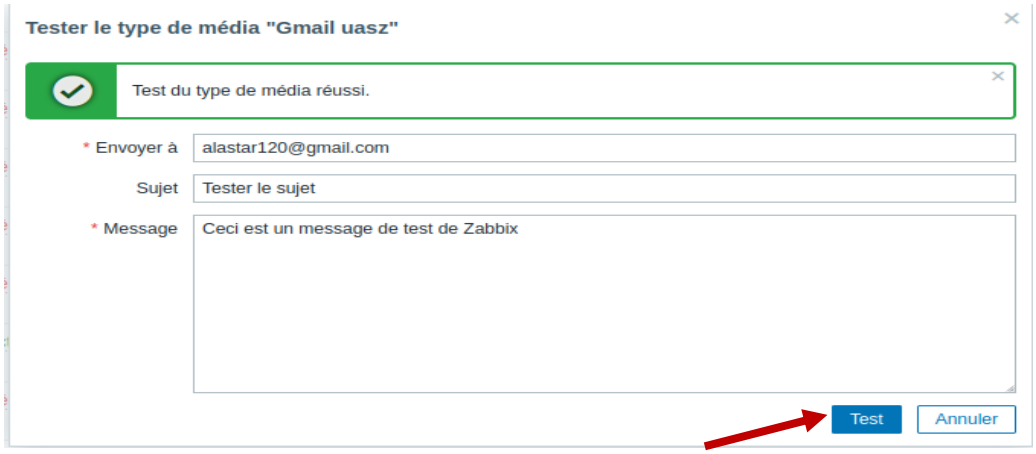


Screenshot 87 : Vérification dans la liste

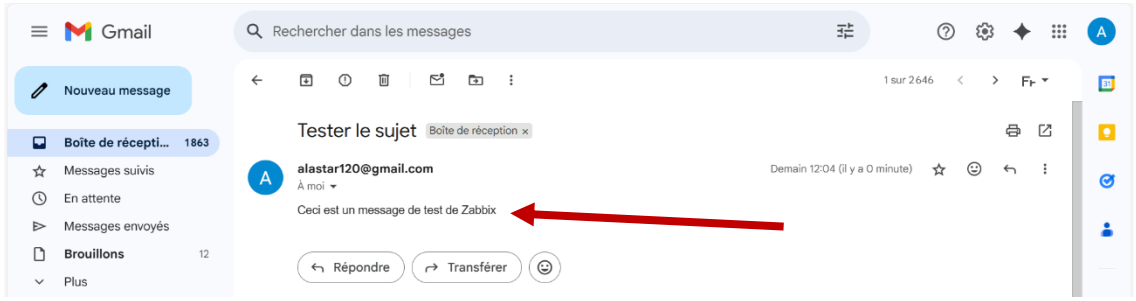
L'interface montre bel et bien que le type de média Gmail uasz est bien activé.



Screenshot 88 : Test du type de média Gmail uasz



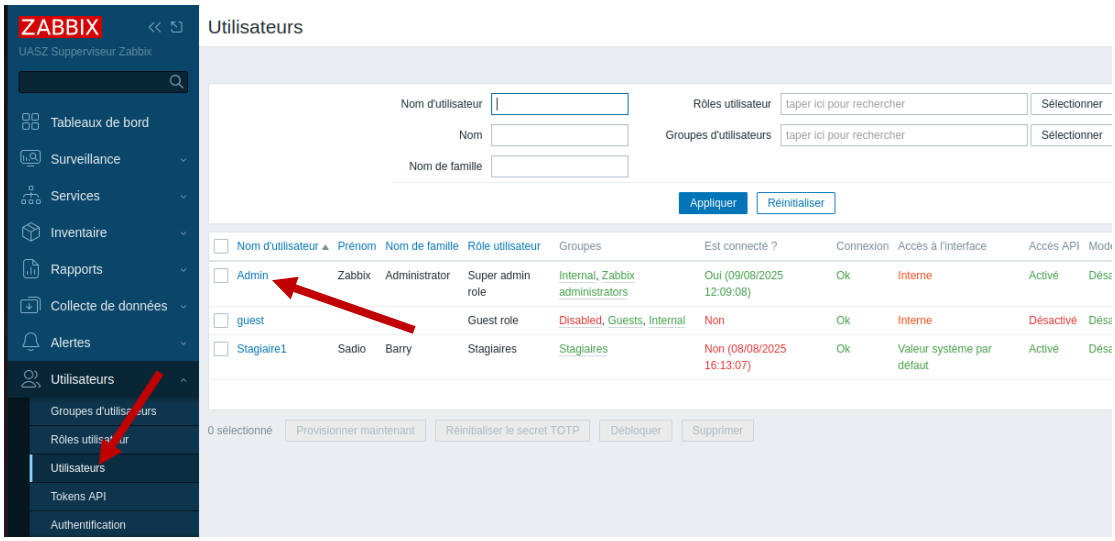
Screenshot 89 : Envoie de message pour le test du type de média Gmail uasz



Screenshot 90 : Réception du message de test

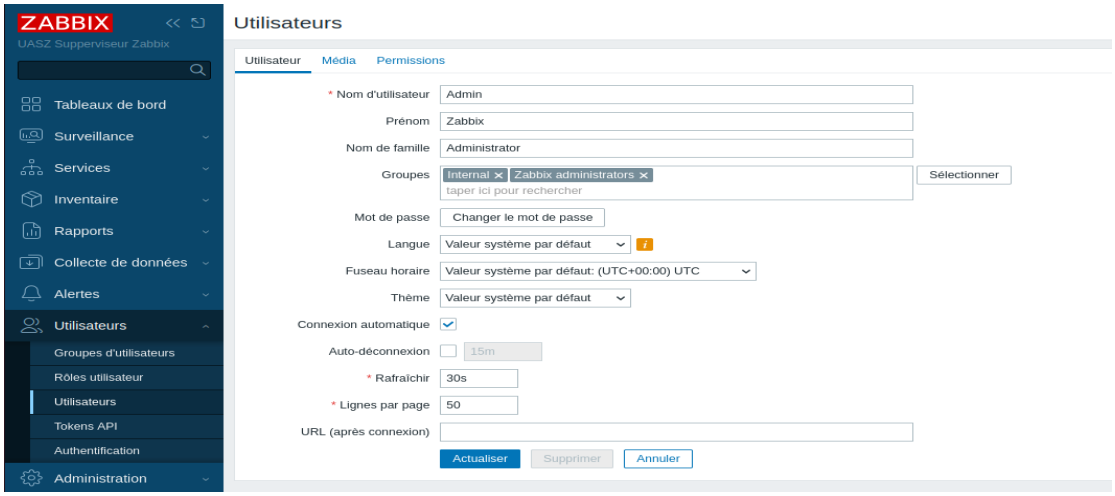
Etape 5 : Utiliser la fonction **Test** disponible dans l'interface pour envoyer un email de test **(Screenshot 88)**. Ecrire un petit message de test dans le champ message **(Screenshot 89)**. Puis cliquer sur **Test**. Vérifier la réception de l'email et son formatage **(Screenshot 90).**

4.3.6 Ajout de Mail pour l'administrateur

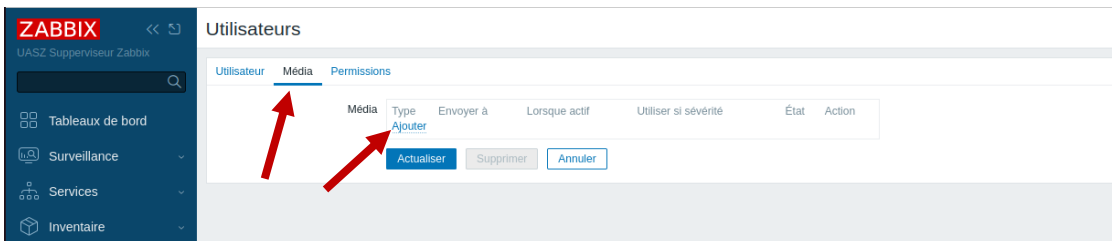


Screenshot 90 : Accès à la liste des utilisateurs

Étape 1 : Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Utilisateurs** > **Utilisateurs**. Dans le tableau des utilisateurs, cliquer sur le lien **"Admin"** dans la colonne "Nom d'utilisateur". Cette action ouvre la page de configuration détaillée de l'utilisateur Admin.

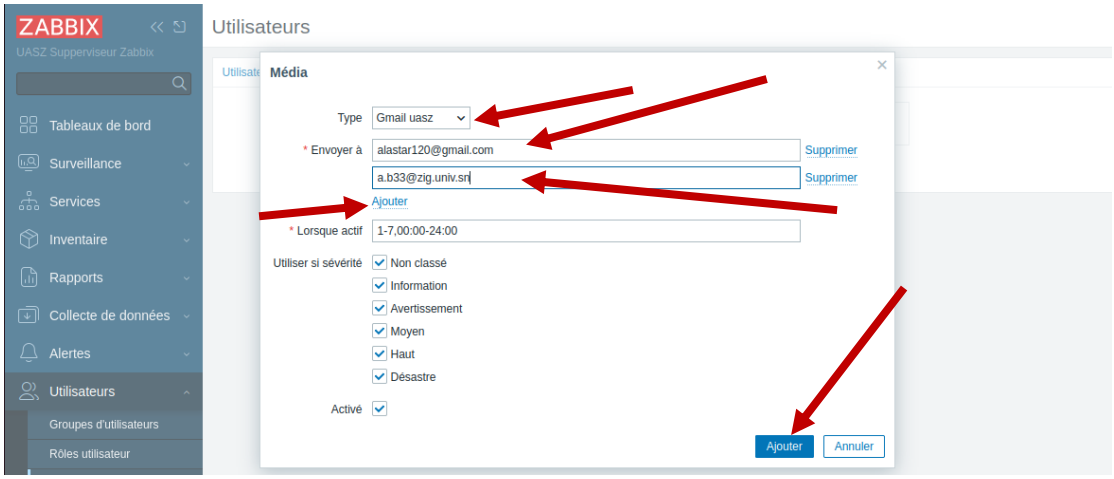


Screenshot 91 : Page de configuration détaillée de l'utilisateur Admin



Screenshot 92 : Accès à l'onglet Média

Étape 3 : Cliquer sur l'onglet **Média** pour accéder à la gestion des canaux de notification. Un lien **Ajouter** sous le tableau permet de créer un nouveau média.

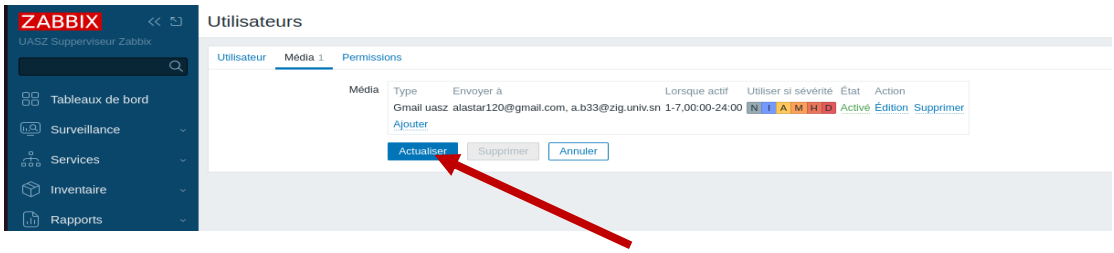


Screenshot 93 : La section ajouter Média

Étape 4 : Dans le menu déroulant **Type**, sélectionner **Gmail uasz** (le type de média Gmail créé précédemment). Dans le champ obligatoire **Envoyer à**, saisir la première adresse email de destination : **alastar120@gmail.com**. Cette adresse recevra toutes les notifications Zabbix envoyées via ce média. Cliquer sur le lien **Ajouter** pour ajouter un mail supplémentaire. Dans le nouveau champ **Envoyer à**, saisir la deuxième adresse : **a.b33@zig.univ.sn.**

Pour un administrateur système, il est recommandé de cocher toutes les cases (**Utiliser si sévérité**) afin de recevoir l'ensemble des notifications, quel que soit le niveau de sévérité. Pour d'autres profils utilisateurs (stagiaires), on peut ne cocher que certains niveaux (par exemple, uniquement "Haut" et "Désastre") pour éviter la surcharge d'informations.

Cocher la case **Activé** pour rendre ce média immédiatement opérationnel. Enfin Cliquer sur le bouton bleu **Ajouter** en bas de la fenêtre pour enregistrer ce média.



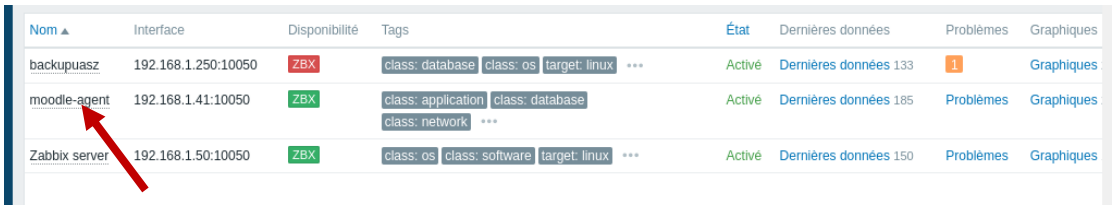
Screenshot 94 : Ajout média réussi

Étape 5 : il faut maintenant sauvegarder les modifications de l'utilisateur. Cliquer sur le bouton bleu **Actualiser** en bas de la page.

L'utilisateur **Admin** est maintenant configuré pour recevoir toutes les alertes Zabbix par email via le type de média **Gmail uasz**, sur deux adresses différentes, garantissant ainsi qu'aucune alerte critique ne soit manquée et permettant une réponse rapide aux incidents affectant l'infrastructure surveillée.

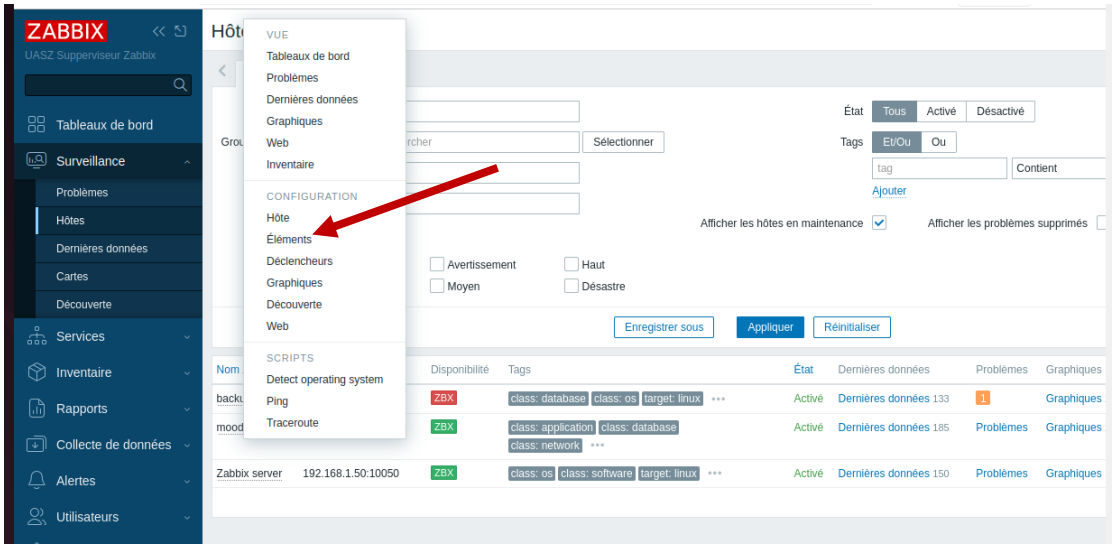
4.3.6 Supervision de la disponibilité du serveur web et base de données

a) Création de l'élément disponibilité de ssh sur le port 22



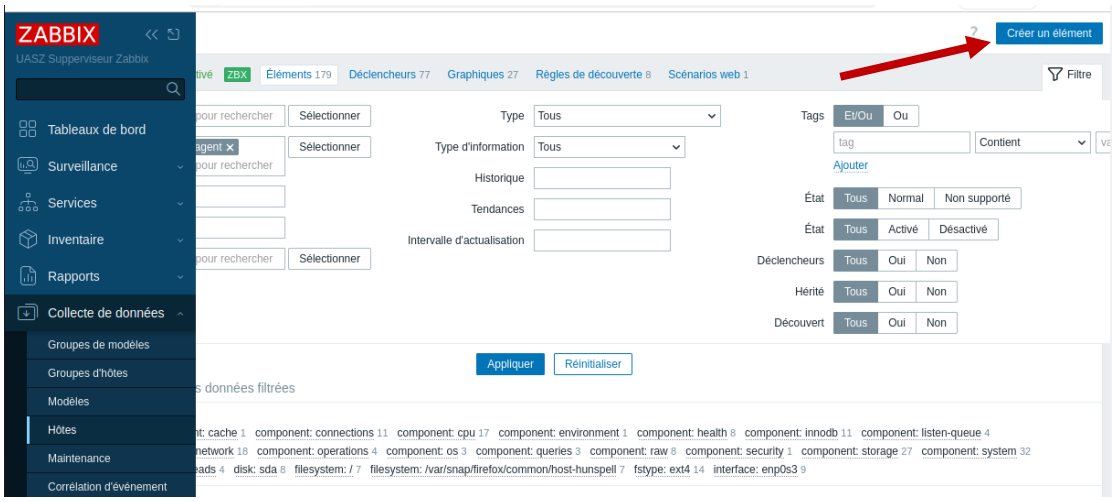
Screenshot 95 : Accès à la liste des hôtes

Étape 1 : Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Surveillance** > **Hôtes**. Sélectionner **moodle-agent** qui héberge le serveur Moodle nécessitant une surveillance de la disponibilité SSH pour les connexions d'administration.



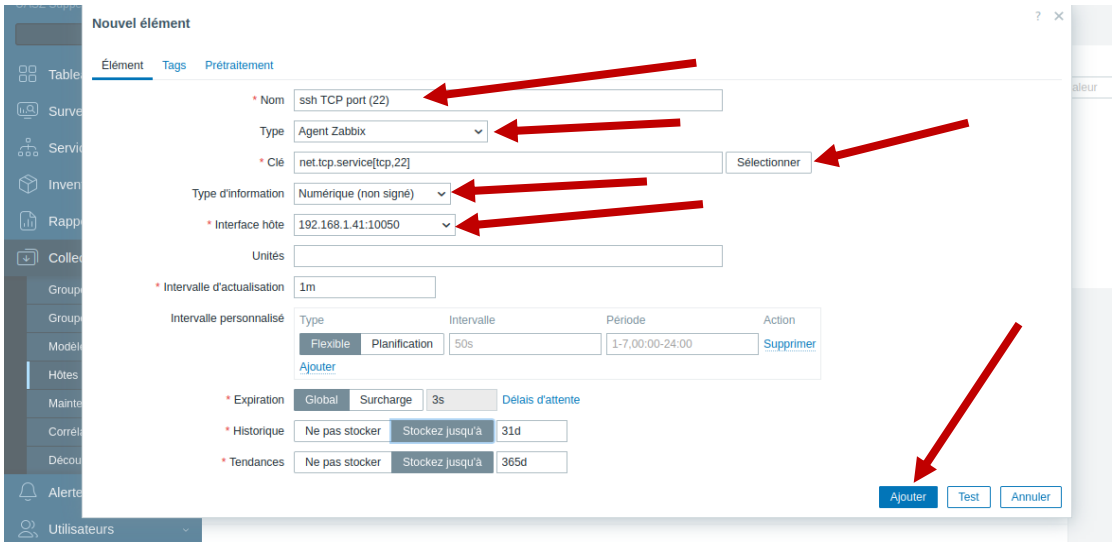
Screenshot 96 : Accès à la configuration des éléments

Etape 2 : Dans le menu contextuel, sous la section **CONFIGURATION**, cliquer sur **Éléments**.



Screenshot 97 : Création d'un nouvel élément

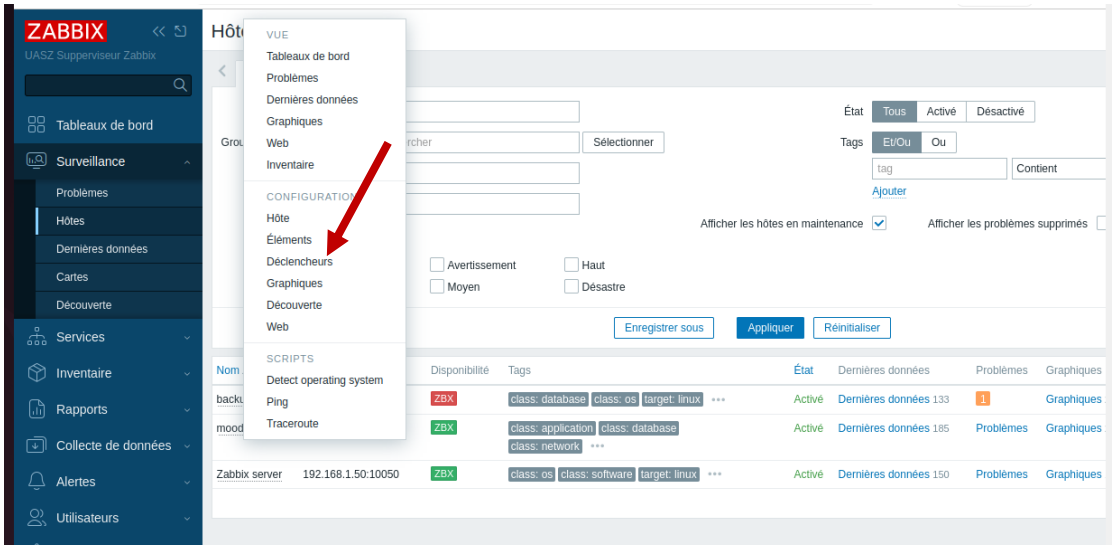
Etape 3 : Cliquer sur le bouton bleu **Créer élément** situé en haut à droite de la page.



Screenshot 98 : Configuration du nouveau de l'élément

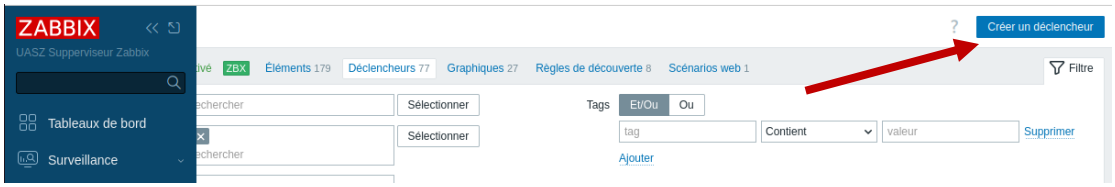
Etape 4 : Dans le champ **Nom,** saisir un nom : **ssh TCP port (22)**. Dans le menu déroulant **Type**, sélectionner **Agent Zabbix** pour collecter via l'agent Zabbix installé sur l'hôte. Dans le champ **Clé**, saisir : **net.tcp.service[tcp,22]**. Cette clé est une fonction Zabbix qui vérifie la disponibilité d'un service TCP sur un port spécifique. Un bouton **Sélectionner** à droite du champ permet de parcourir les clés disponibles. Dans le menu déroulant **Type d'information**, sélectionner **Numérique (non signé).** Dans le menu déroulant **Interface hôte**, sélectionner **192.168.1.41:10050** qui correspond à l'interface de l'agent Zabbix sur le serveur moodle-agent. Faire défiler jusqu'en bas de la fenêtre et cliquer sur le bouton bleu **Ajouter** pour créer l'élément.

b) Création des déclencheurs de l'indisponibilité de ssh sur le port 22 et Ping (ICMP)



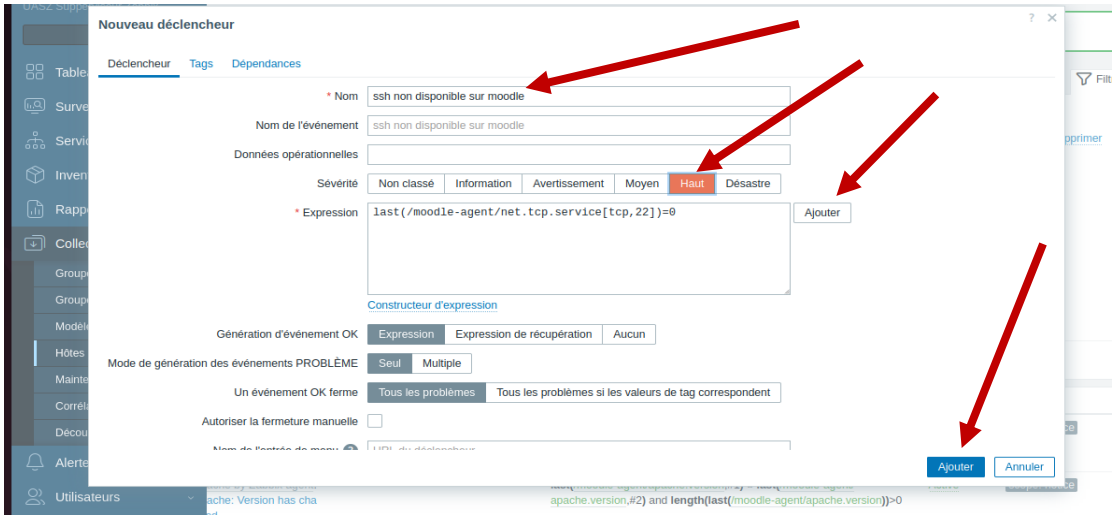
Screenshot 99 : Accès à la configuration des déclencheurs

Etape 1 : Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Surveillance** > **Hôtes > moodle-agent > Déclencheurs.**



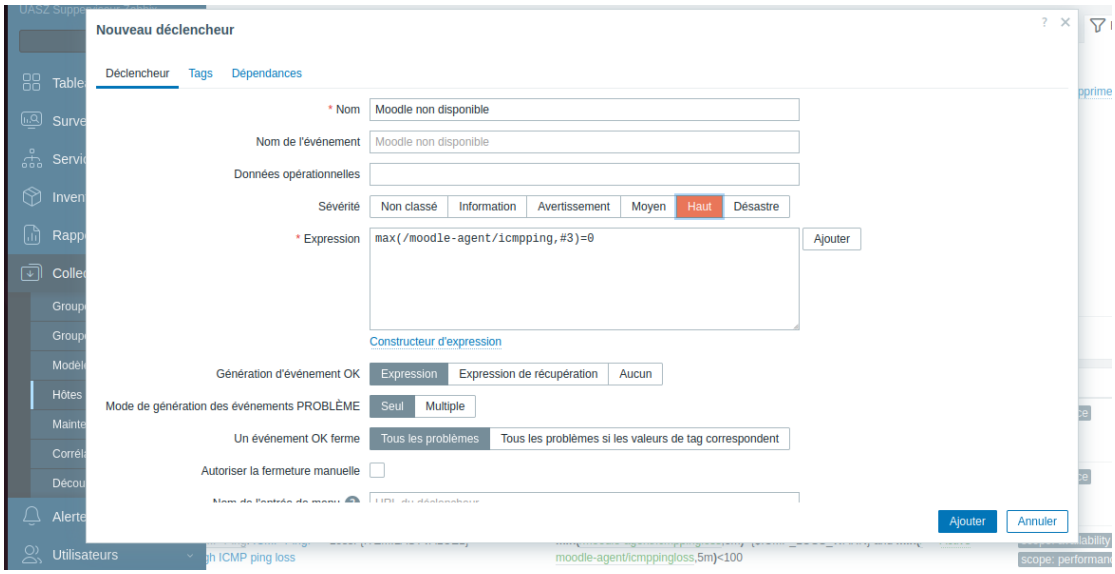
Screenshot 100 : Création d'un nouvel déclencheur

Etape 2 : Cliquer sur **Créer un déclencheur** situé en haut à droite de la page.



Screenshot 101 : Configuration du déclencheur ssh non disponible sur moodle

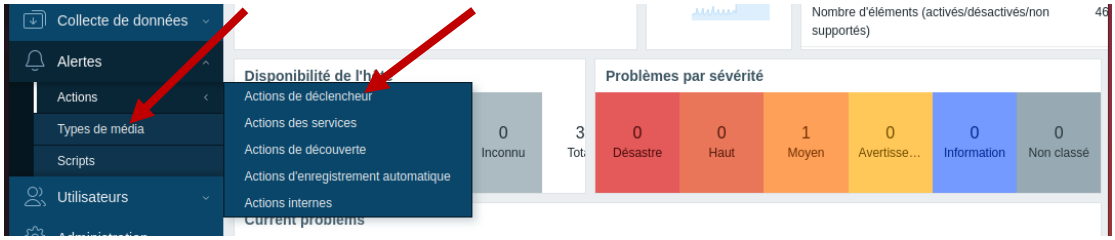
Etape 3 : Dans le champ Nom, saisir le nom du déclencheur : **ssh non disponible sur moodle**. Pour la sévérité, sélectionner **Haut** pour incident grave. Cliquer sur le bouton **ajouter** qui se trouve à coté du champ **expression**. Sélectionner l'hôte : **moodle-agent.** Sélectionner l'élément **: net.tcp.service[tcp,22]** précédemment crée. Choisir la fonction : **last().** Sélectionner l'opérateur : **=.** Saisir la valeur : **0**.
Cliquer sur **Insérer.** L'expression correcte est automatiquement générée dans le champ. Faire défiler jusqu'en bas de la fenêtre et cliquer sur le bouton bleu **Ajouter** pour créer le trigger.



Screenshot 102 : Configuration du déclencheur moodle non disponible (ping)

(voir **Etape 1 &2**).
Dans le champ Nom, saisir le nom du déclencheur : **moodle non disponible**. Pour la sévérité, sélectionner **Haut** pour incident grave. Cliquer sur le bouton **ajouter** qui se trouve à côté du champ expression. Sélectionner l'hôte : moodle-agent. Sélectionner l'élément : **icmpping**. Choisir la fonction : **max()**. Sélectionner l'opérateur : =. Saisir la valeur : **0**. Cliquer sur Insérer. Faire défiler jusqu'en bas de la fenêtre modale et cliquer sur le bouton bleu **Ajouter** pour créer le trigger.

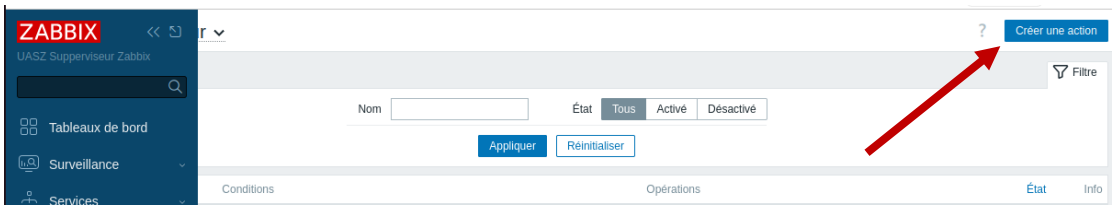
c) Création des actions Moodle (Ping et SSH) non disponible



Screenshot 103 : Accès au menu des actions

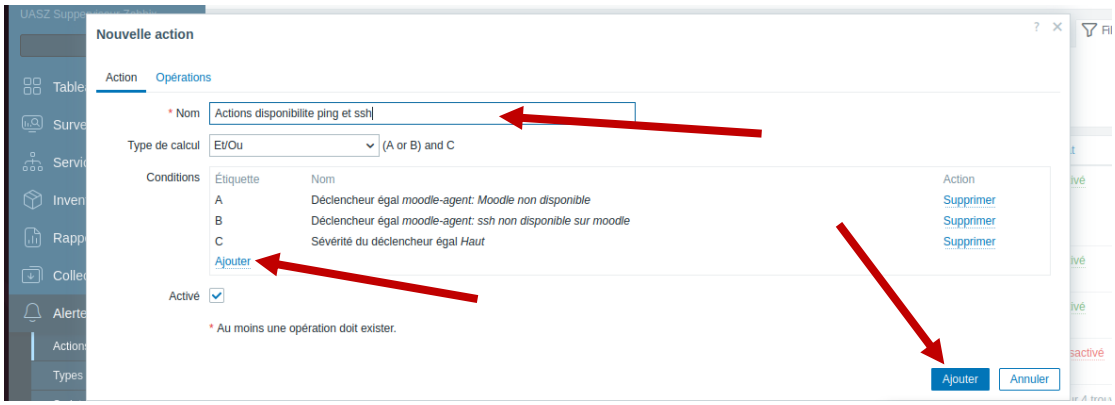
Étape 1 : Depuis l'interface principale de Zabbix, naviguer dans le menu latéral

gauche vers **Alertes** > **Actions** > **Action de déclencheur**.



Screenshot 104 : Création d'une nouvelle action

Étape 2 : Cliquer sur le bouton bleu **Créer une action** située en haut à droite de la page.

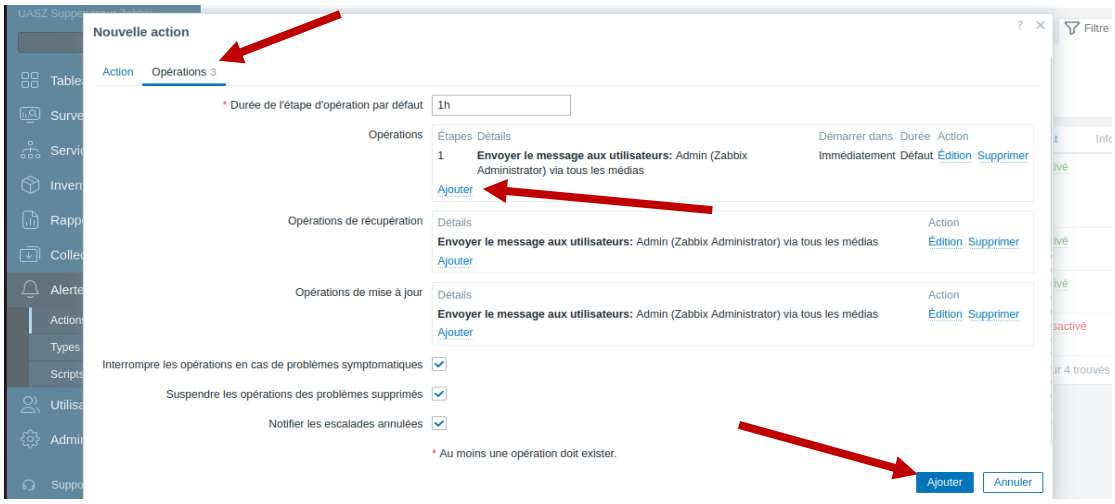


Screenshot 105 : Configuration de l'action disponibilité du serveur Web

Etape 3 : Saisir dans le champ **Nom**: **Actions disponibilite ping et ssh**. Pour ajouter cette condition :

- Cliquer sur le lien **"Ajouter"** sous le tableau des conditions
- Une nouvelle ligne de configuration apparaît avec des menus déroulants
- Configurer :
  - **Type de condition** : Déclencheur
  - **Opérateur** : égal
  - **Valeur** : sélectionner **moodle-agent: ssh non disponible sur moodle** déclencheur précédemment crée.

Procéder les mêmes étapes pour ajouter la condition pour **moodle non disponible** (ping). Cocher la case **Activé** pour que l'action soit immédiatement opérationnelle après création.



Screenshot 106 : l'onglet Opérations

Etape 4 : Cliquer sur l'onglet **Opérations** en haut de la fenêtre modale. Cliquer sur le lien **Ajouter** sous le tableau des opérations. Une section de configuration d'opération s'ouvre. Configurer :

- **Étapes** : 1-1 (de l'étape 1 à l'étape 1, donc une seule fois)
- **Type d'opération** : Envoyer un message
- **Envoyer à** :
    - Sélectionner **Utilisateurs**
    - Choisir **Admin (Zabbix Administrator)** dans la liste
- **Envoyer uniquement à** : **tous les médias** (ou sélectionner **Gmail uasz** spécifiquement)
- **Défaut message** : utiliser le template de message par défaut
- Cliquer sur **ajouter** pour créer l'opération

La section **Opérations de récupération** définit les actions à exécuter lorsque le problème est résolu.

La section **Opérations de mise à jour** définit les actions lors de la mise à jour d'un problème existant.

Faire défiler jusqu'en bas de la fenêtre modale et cliquer sur le bouton bleu **Ajouter** pour créer l'action.

Les **Screenshot 105 & 106** montrent la configuration d'une action nommée **Actions disponibilité ping et ssh** qui enverra des notifications aux administrateurs lorsque des problèmes de disponibilité réseau (ping) ou SSH sont détectés sur le serveur web et base de données**.**

La disponibilité des serveurs constitue le premier niveau de supervision. Elle a été mise en œuvre à l'aide :

- De tests **ICMP (Ping)** pour vérifier si le serveur est accessible,

- De contrôles des ports critiques (SSH).

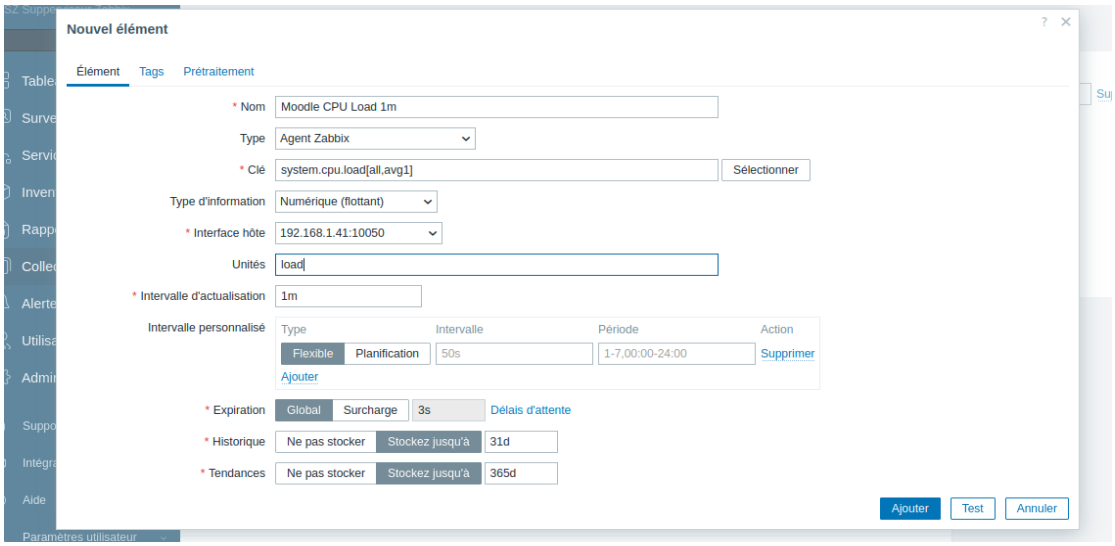
Ces vérifications permettent de détecter rapidement :

- Une panne totale du serveur,
- Une coupure réseau,
- Un service essentiel indisponible.

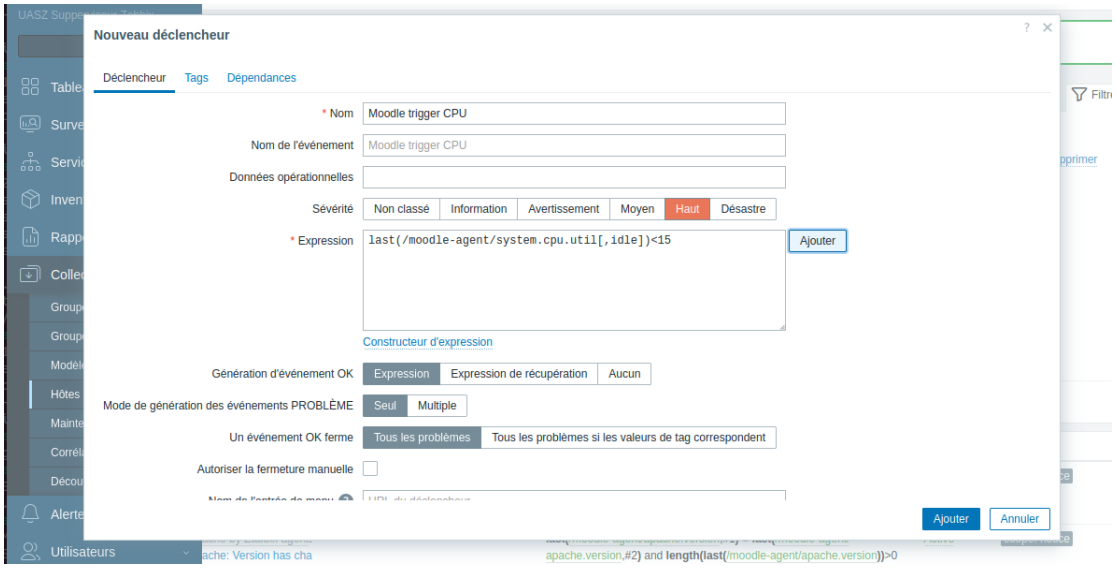
Des déclencheurs critiques ont été définis afin de générer des alertes immédiates en cas d'indisponibilité.

4.3.7 Supervision des ressources système du serveur web et base de données

a) Surveillance de l'utilisation du Processeur (CPU)



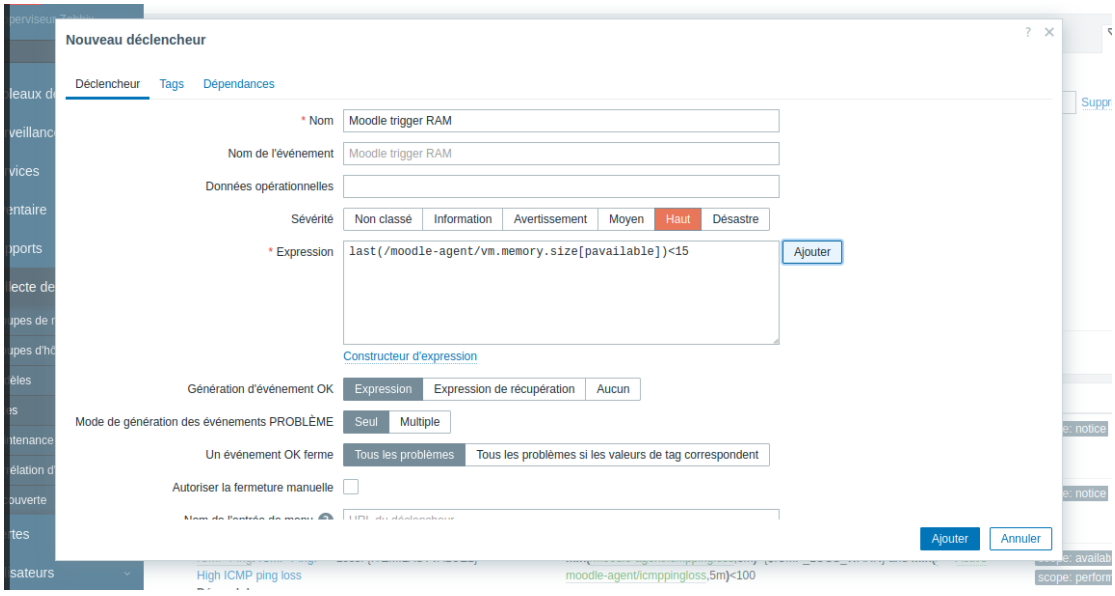
Screenshot 107 : Création de l'élément Moodle CPU Load 1m



Screenshot 107 : Création du déclencheur pour le processeur CPU

Ce déclencheur surveille l'utilisation du processeur (CPU) sur le serveur moodle-agent et génère une alerte lorsque que moins de 15% du CPU est disponible, donc plus de 85% est utilisé.

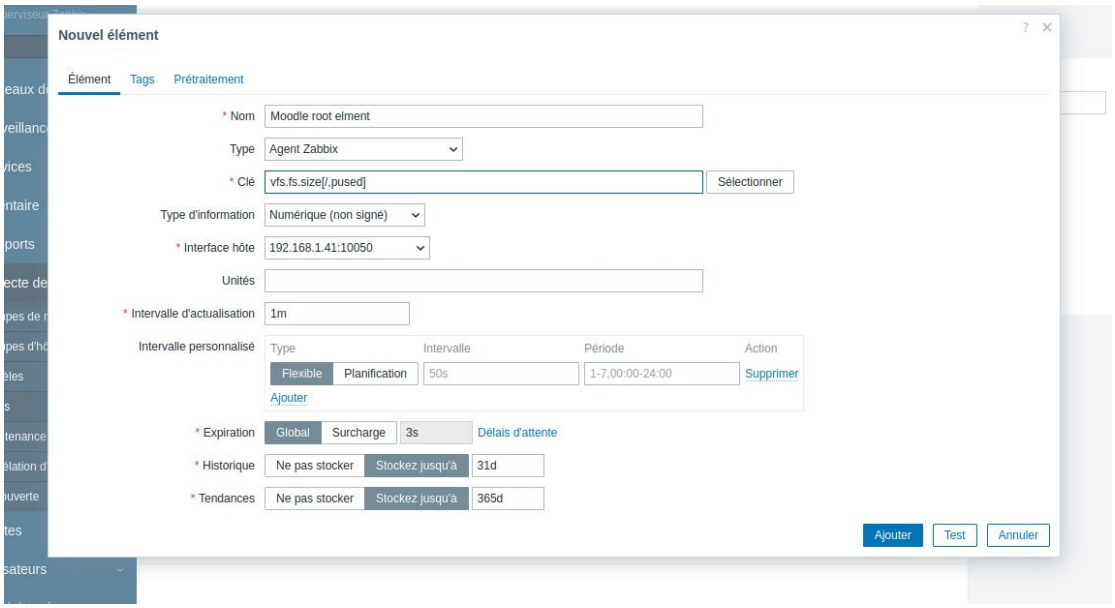
b) Surveillance de la disponibilité de la RAM



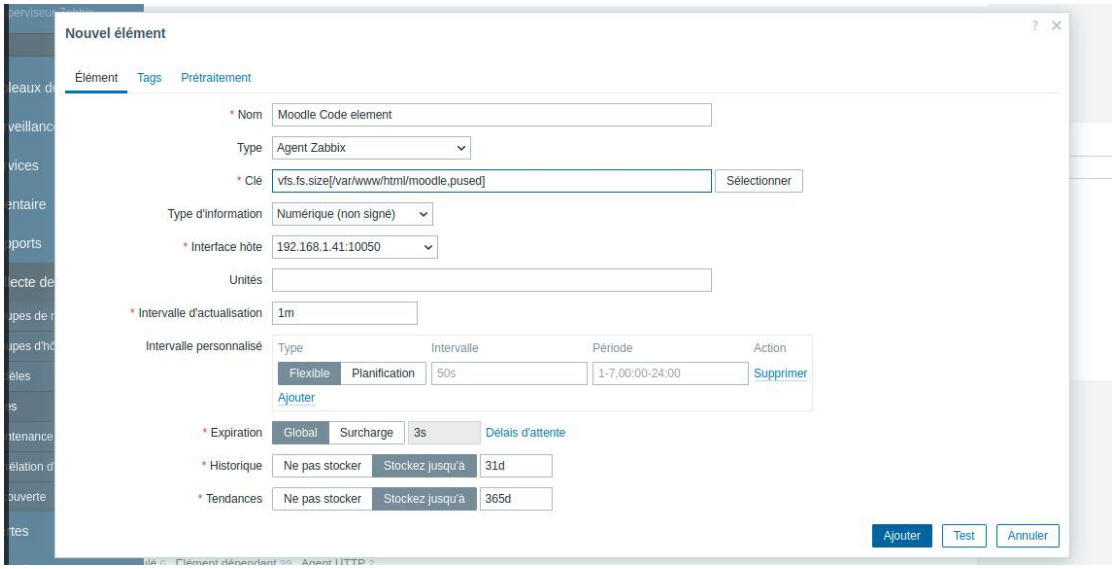
Screenshot 108 : Création du déclencheur pour la RAM

Ce déclencheur surveille la mémoire vive (RAM) disponible sur le serveur moodle-agent et génère une alerte lorsque moins de 15% de la mémoire est disponible, indiquant une saturation critique de la RAM pouvant entraîner des ralentissements.

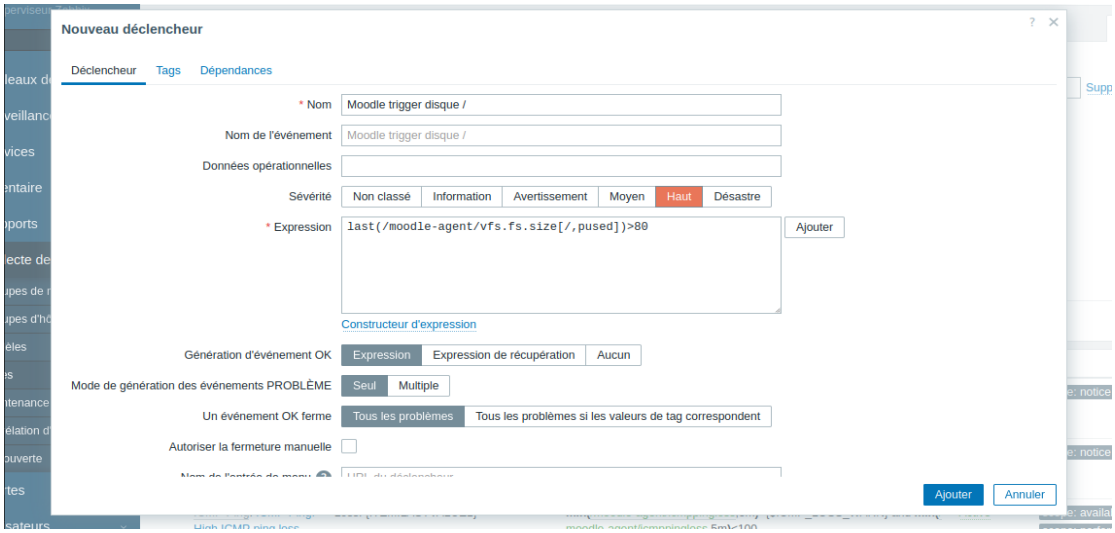
c) La surveillance des espaces disques



Screenshot 109 : Création de l'élément pour le répertoire racine (/)

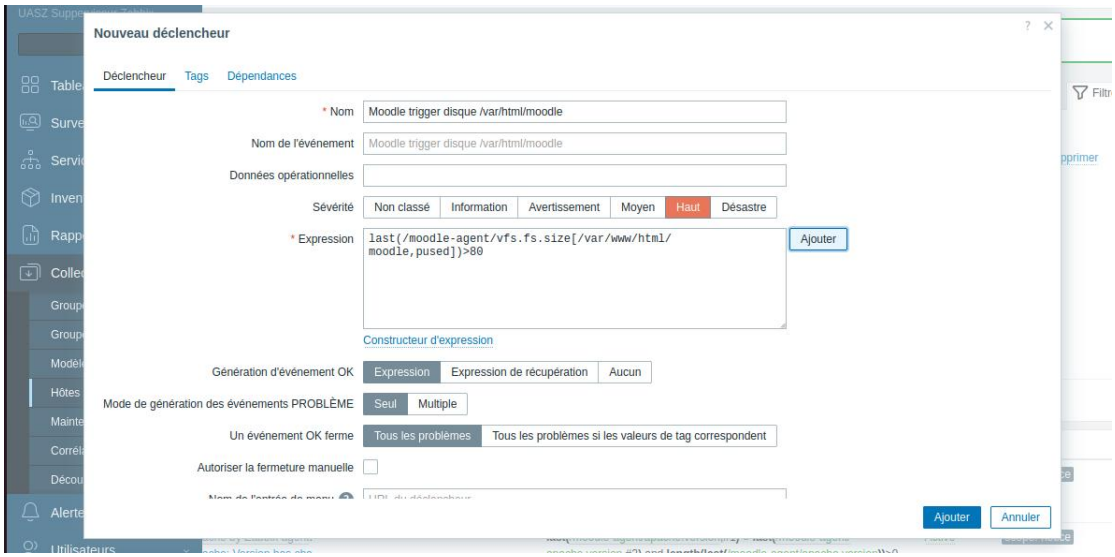


Screenshot 110 : Création de l'élément pour le disque /var/www/html/moodle



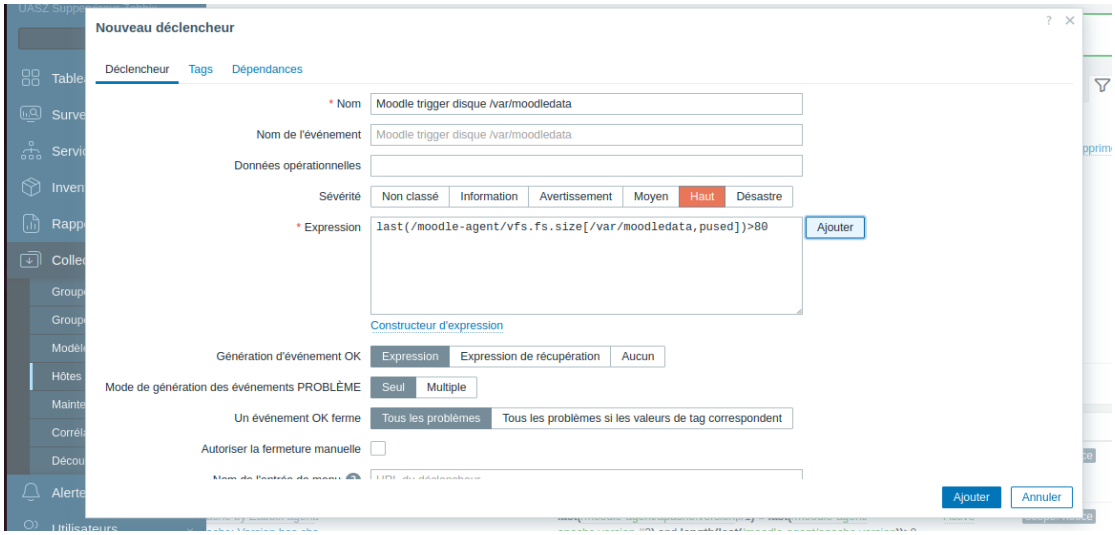
Screenshot 111 : Création du déclencheur pour le disque racine (/)

Ce déclencheur surveille l'utilisation de l'espace disque de la partition racine / **(root)** sur le serveur moodle-agent et génère une alerte lorsque plus de 80% de l'espace disque est utilisé. La partition racine est critique car elle contient l'ensemble du système d'exploitation Linux, les programmes système, les bibliothèques, les logs système, et potentiellement des données temporaires. Sa saturation peut entraîner un dysfonctionnement complet du serveur.



Screenshot 112 : Création du déclencheur pour le répertoire Code Moodle

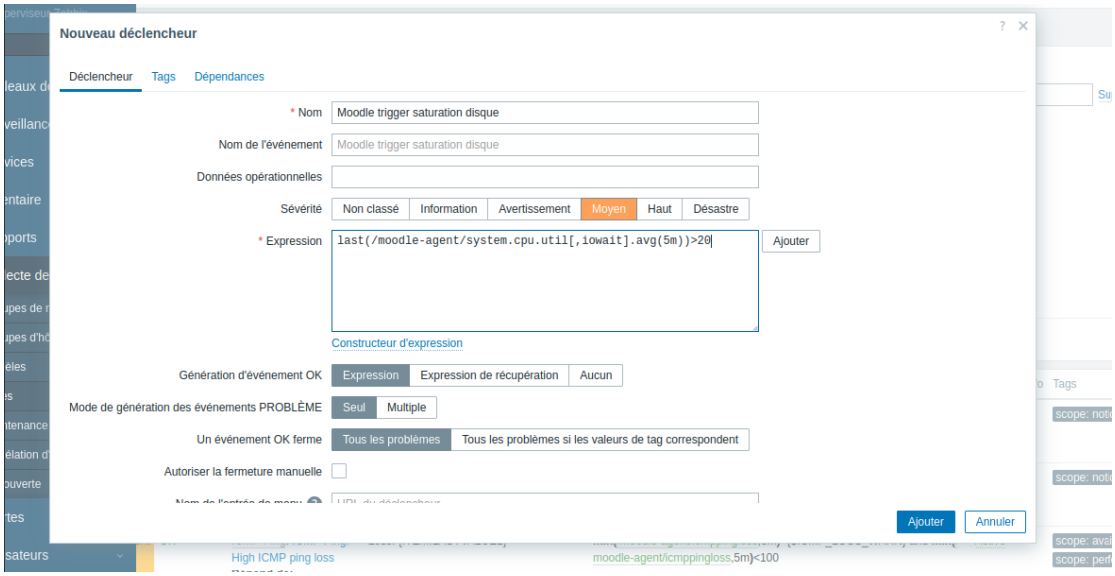
Ce déclencheur surveille l'utilisation de l'espace disque du répertoire **/var/www/html/moodle** sur le serveur moodle-agent et génère une alerte lorsque plus de 80% de l'espace disque est utilisé, indiquant une saturation critique du système de fichiers hébergeant l'application Moodle. Ce trigger est particulièrement important car le répertoire **/var/www/html/moodle** contient principalement du code PHP de la plateforme Moodle.



Screenshot 113 : Création du déclencheur pour le répertoire de données moodle

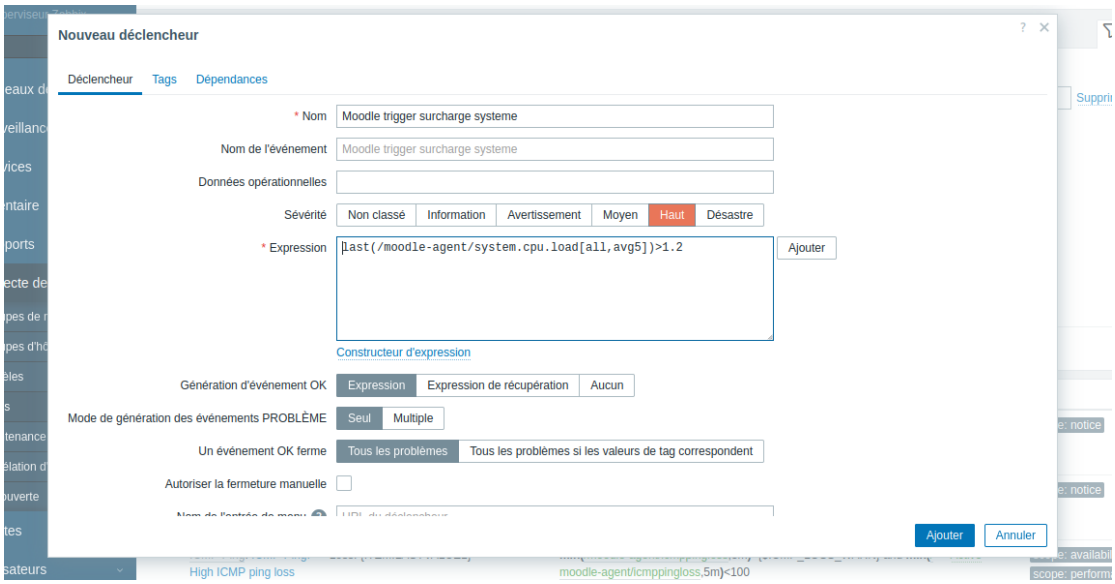
Ce déclencheur surveille l'utilisation de l'espace disque du répertoire **/var/moodledata** sur le serveur moodle-agent et génère une alerte lorsque plus de 80% de l'espace disque est utilisé. Le répertoire moodledata est **le répertoire le plus critique** de toute installation Moodle car il contient exclusivement les **données utilisateurs** : tous les fichiers ajoutés par étudiants et enseignants, le cache applicatif,

les sessions, les fichiers temporaires, et les sauvegardes de cours.



Screenshot 114 : Création du déclencheur saturation disque

Ce déclencheur surveille le temps d'attente CPU lié aux opérations d'entrée/sortie disque (I/O wait) sur le serveur moodle-agent et génère une alerte lorsque la moyenne d'I/O wait sur 5 minutes dépasse 20%. Contrairement aux triggers précédents qui surveillent l'espace disque (combien d'espace est utilisé), ce trigger surveille la performance disque (combien de temps le CPU attend que le disque réponde). Un I/O wait élevé indique une saturation des performances du sous-système disque, même si l'espace disponible est suffisant. C'est un indicateur critique de goulot d'étranglement qui peut rendre Moodle extrêmement lent même avec des ressources CPU et RAM disponibles.



Screenshot 115 : Création du déclencheur surcharge système

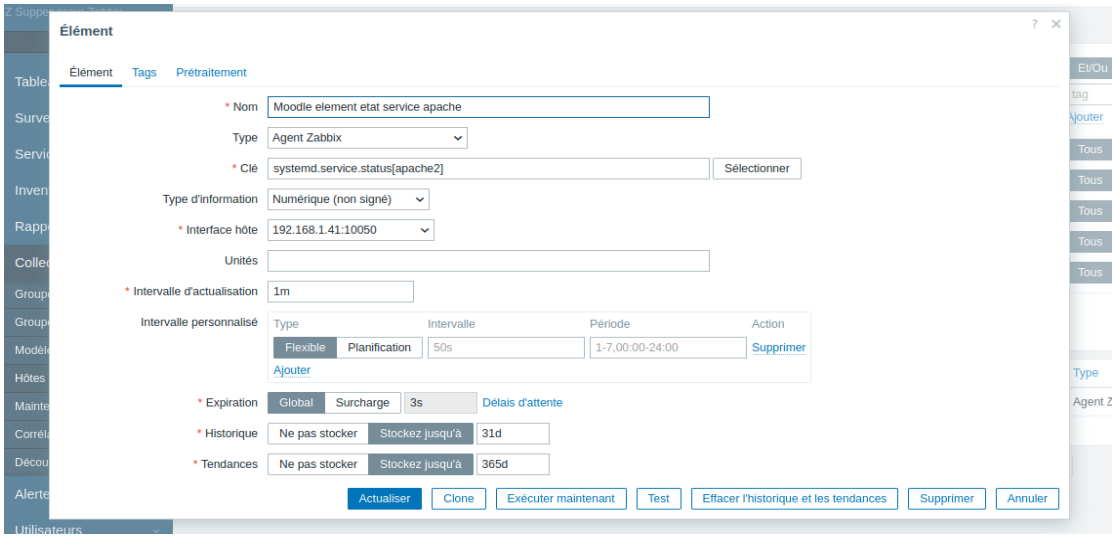
Ce déclencheur permet de surveiller la charge système (**load average**) sur le serveur moodle-agent et génère une alerte lorsque la moyenne de charge sur 5 minutes dépasse 1.2 fois le nombre de cœurs CPU disponibles. Le **load average** est une métrique système fondamentale qui représente le nombre moyen de processus **en attente d'exécution** ou **en cours d'exécution** sur le CPU. C'est l'un des premiers indicateurs qu'un administrateur système consulte pour évaluer si un serveur est surchargé.

La supervision des ressources système vise à anticiper les dégradations de performance et les risques de saturation. Des seuils d'alerte ont été définis (par exemple **CPU > 85 %, disque > 80 %)** afin de prévenir les administrateurs avant qu'une panne ne survienne.

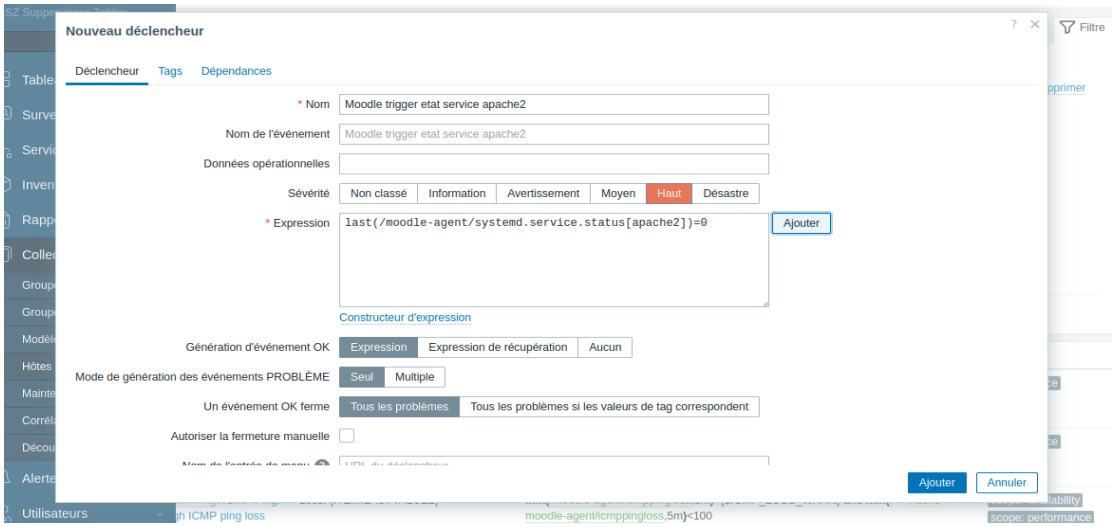
4.3.8 Supervision des services applicatifs Moodle

a) Le service Apache

Apache2 est le **serveur web** qui héberge l'application Moodle sur le serveur moodle-agent. Il constitue le **point d'entrée** de tous les utilisateurs (étudiants, enseignants, administrateurs) vers la plateforme d'apprentissage. Sans Apache2 actif et fonctionnel, Moodle devient totalement inaccessible, indépendamment de l'état de MySQL, PHP-FPM ou du système d'exploitation. La surveillance de l'état du service Apache2 est donc **absolument critique** pour garantir la disponibilité de la plateforme pédagogique.



Screenshot 116 : Création de l'élément pour le service apache



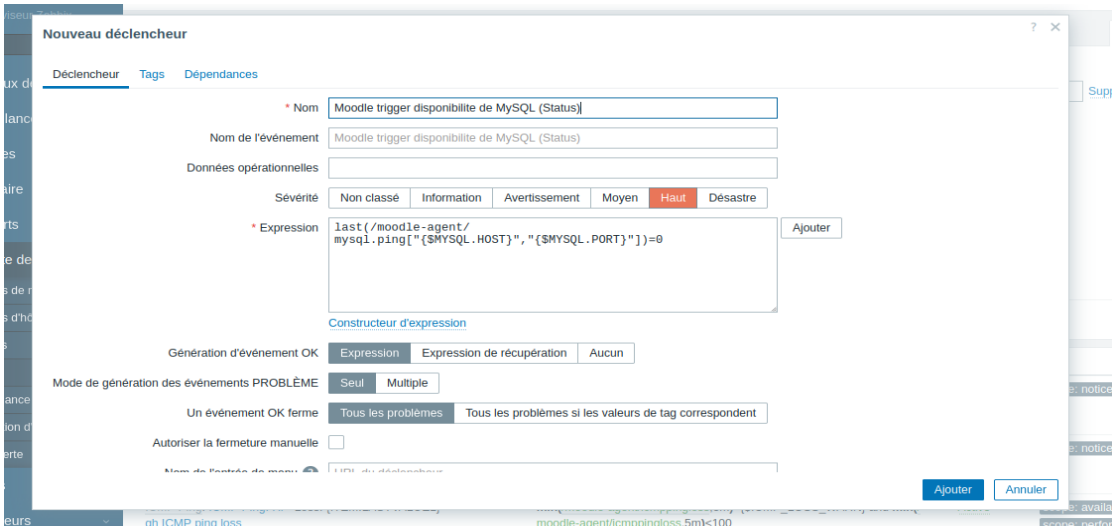
Screenshot 117 : Création du déclencheur pour le service apache

L'élément **Moodle élément état service apache** interroge régulièrement systemd (le gestionnaire de services Linux) pour déterminer si le service apache2 est actif, arrêté ou dans un état problématique.

Le déclencheur **Moodle trigger état service apache2** génère une alerte critique lorsque le service Apache2 n'est plus en état d'exécution, indiquant que le serveur web est arrêté ou en échec et que Moodle est par conséquent inaccessible.

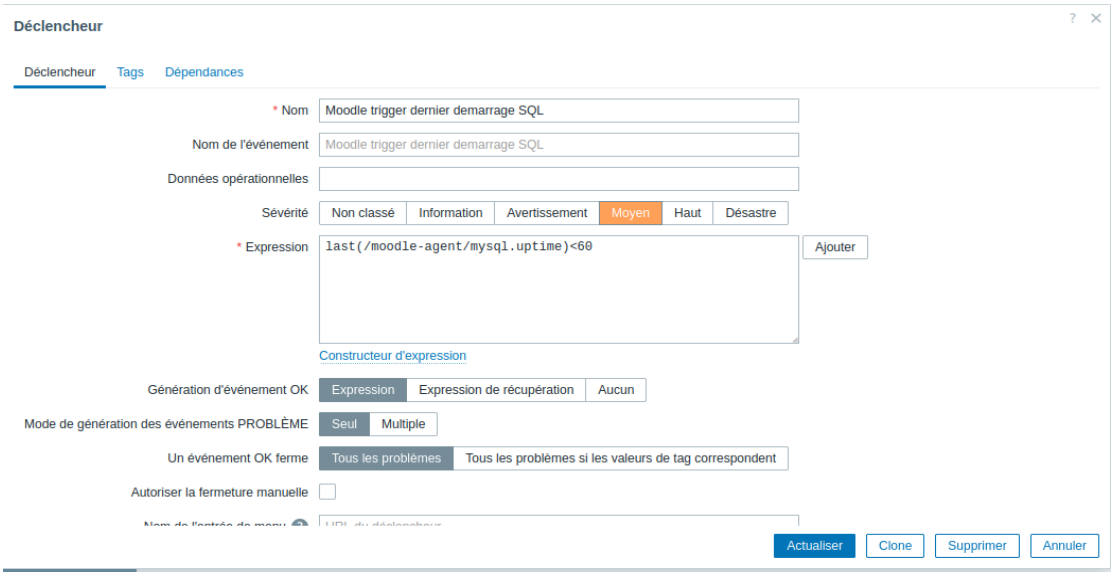
b) Le service MySQL

Sans MySQL fonctionnel, Moodle affiche des **erreurs de connexion base de données** et devient totalement inutilisable, même si Apache et PHP fonctionnent parfaitement. La surveillance de MySQL est donc **aussi critique** que celle d'Apache, constituant le second pilier de la disponibilité applicative.



Screenshot 118 : Création du déclencheur de la disponibilité de MySQL

Ce déclencheur permet de surveiller la **disponibilité** du service MySQL en testant la capacité à établir une connexion réseau au port MySQL (3306 par défaut). Il utilise une vérification de type **ping** réseau pour déterminer si le serveur MySQL répond aux requêtes de connexion. C'est un test de disponibilité de **bas niveau** (couche TCP/IP) qui détecte si MySQL est en écoute et accepte des connexions.

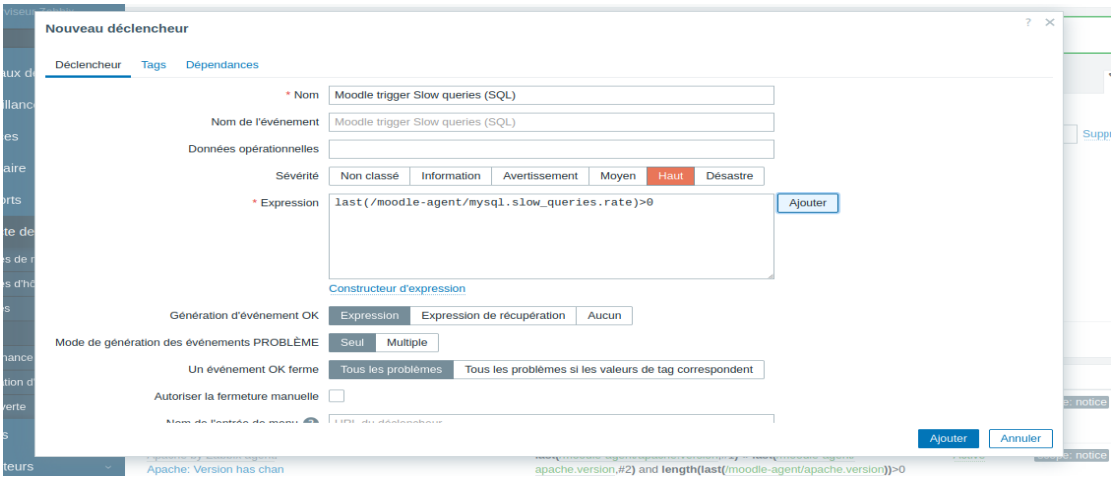


Screenshot 119 : Création du déclencheur de dernier démarrage de MySQL

Ce déclencheur surveille la performance de MySQL en mesurant le temps écoulé depuis le dernier démarrage (uptime). Plus précisément, il détecte quand MySQL a été récemment redémarré (uptime < 1 minute), ce qui peut indiquer :

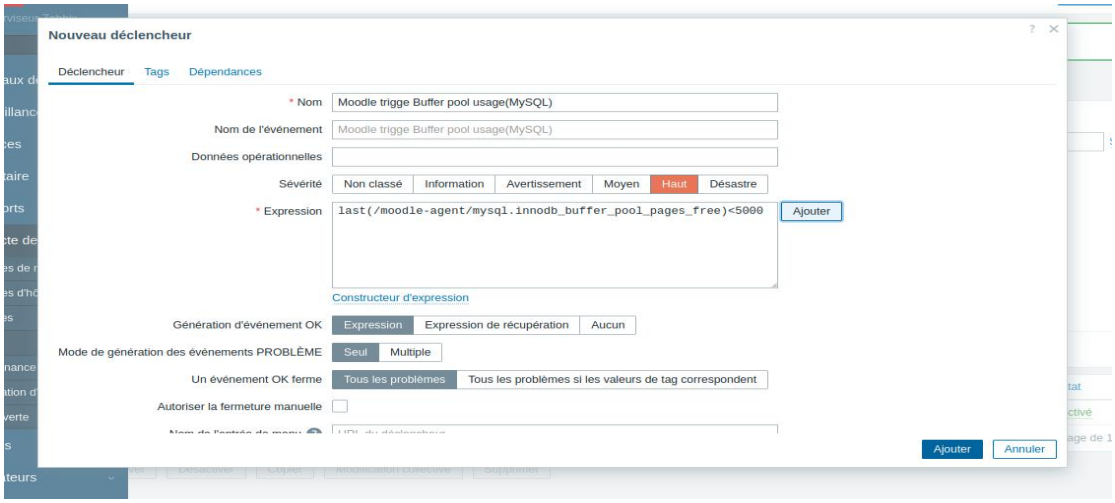
- Crash et redémarrage automatique
- Redémarrage manuel d'urgence
- Instabilité du service C'est un indicateur de disponibilité continue.

Un uptime très bas signale une interruption récente du service, même si MySQL est actuellement fonctionnel.



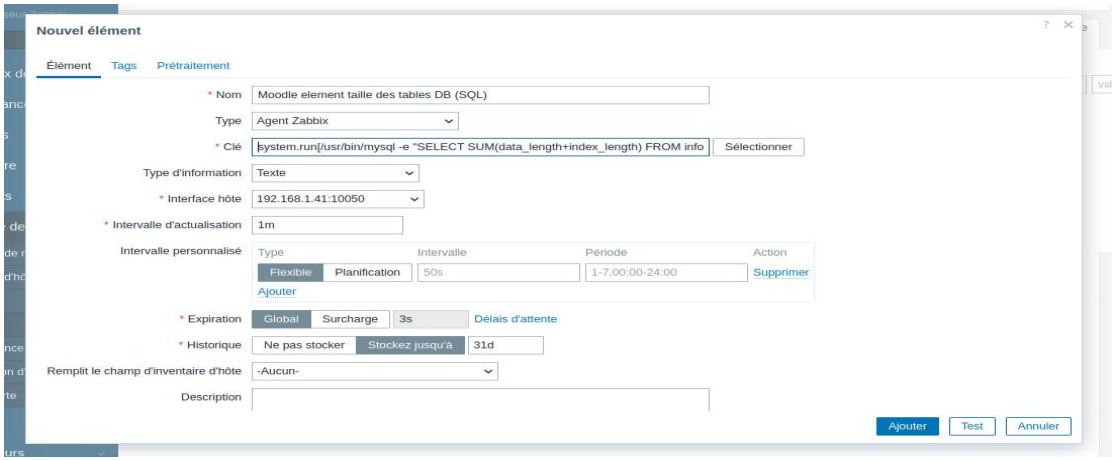
Screenshot 120 : Surveillance des requêtes lentes

Ce déclencheur surveille les **requêtes SQL lentes** (**slow queries**) dans MySQL en mesurant le **taux d'augmentation** du compteur de requêtes lentes par seconde. Les slow queries sont des requêtes qui dépassent un seuil de temps d'exécution défini (par défaut 10 secondes dans MySQL, mais configurable). L'accumulation de requêtes lentes est un indicateur critique de **dégradation des performances de la base de données**, pouvant causer des ralentissements majeurs de Moodle même si MySQL reste techniquement disponible. Ce trigger est particulièrement important car il détecte les problèmes de **performance qualitative** plutôt que les pannes absolues.

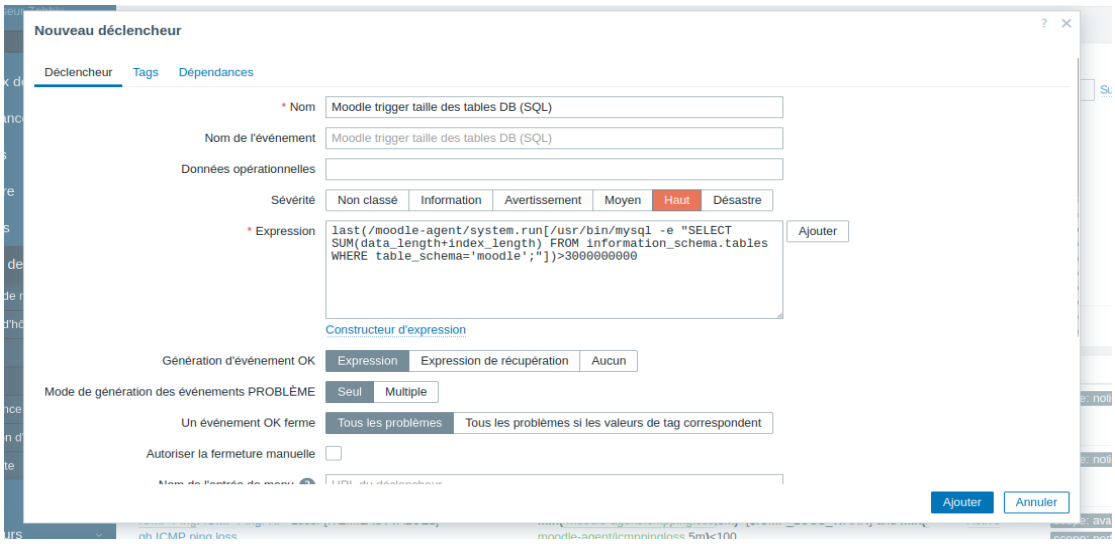


Screenshot 121 : Surveillance de l'efficacité du cache InnoDB

Ce déclencheur surveille l'utilisation du **InnoDB Buffer Pool**, le cache mémoire le plus critique de MySQL. Le buffer pool stocke en RAM les pages de données et d'index fréquemment accédées, évitant des lectures disque coûteuses. Lorsque le buffer pool est saturé (insuffisamment de pages libres disponibles), MySQL doit constamment **libérer des pages anciennes** (flush) pour charger de nouvelles données, causant une **dégradation majeure des performances**. Ce trigger détecte quand le nombre de pages libres dans le buffer pool devient critique, signal d'un cache sous-dimensionné ou d'une charge de travail dépassant la capacité mémoire allouée.



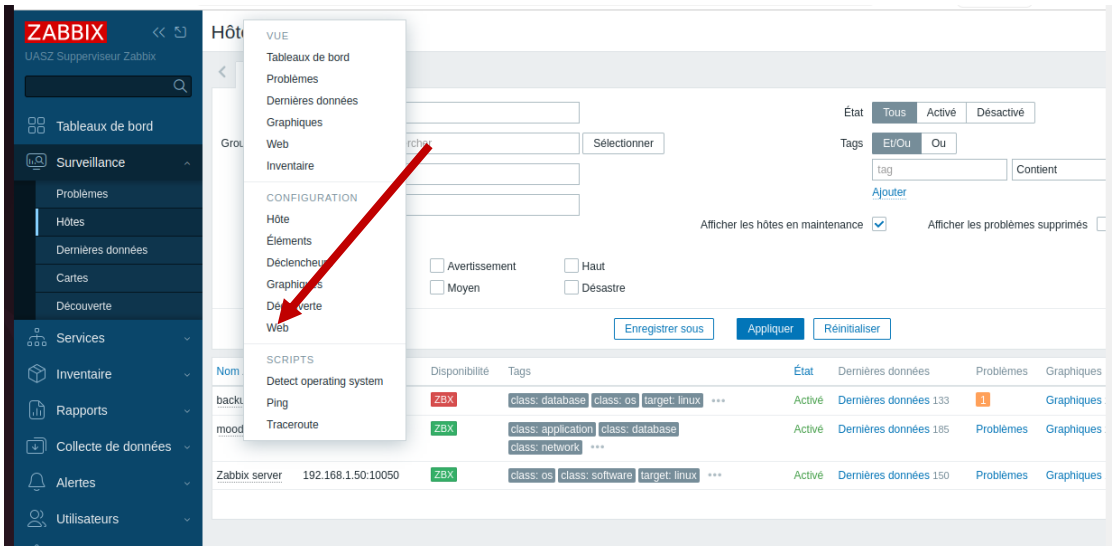
Screenshot 122 : Elément surveillance de la croissance de la base de données



Screenshot 123 : Déclencheur surveillance de la croissance de la base de données

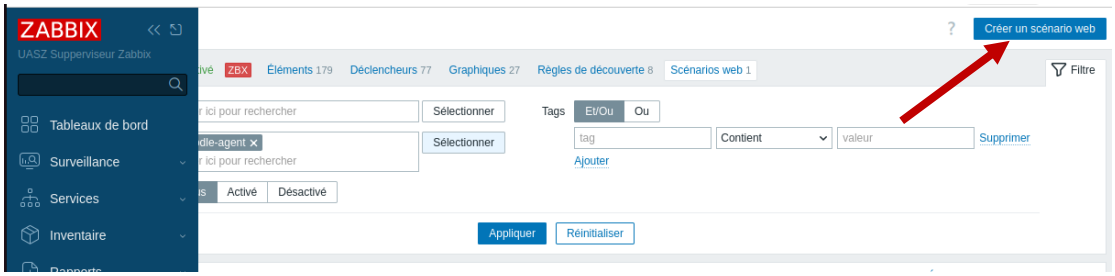
Ce déclencheur surveille la **taille totale de la base de données Moodle** en calculant la somme des tailles de toutes les tables (données + index) et génère une alerte lorsque cette taille dépasse 30 GB. Contrairement aux triggers surveillant l'espace disque disponible (**partitions /, /var/www, /moodledata**), ce trigger se concentre spécifiquement sur la **croissance logique de la base de données** elle-même. Une base de données volumineuse et en croissance rapide peut indiquer des problèmes de gestion des données (logs non purgés, tables non optimisées, accumulation de données obsolètes) et impacter directement les performances MySQL, la durée des backups, et les besoins en stockage.

c) Surveillance web



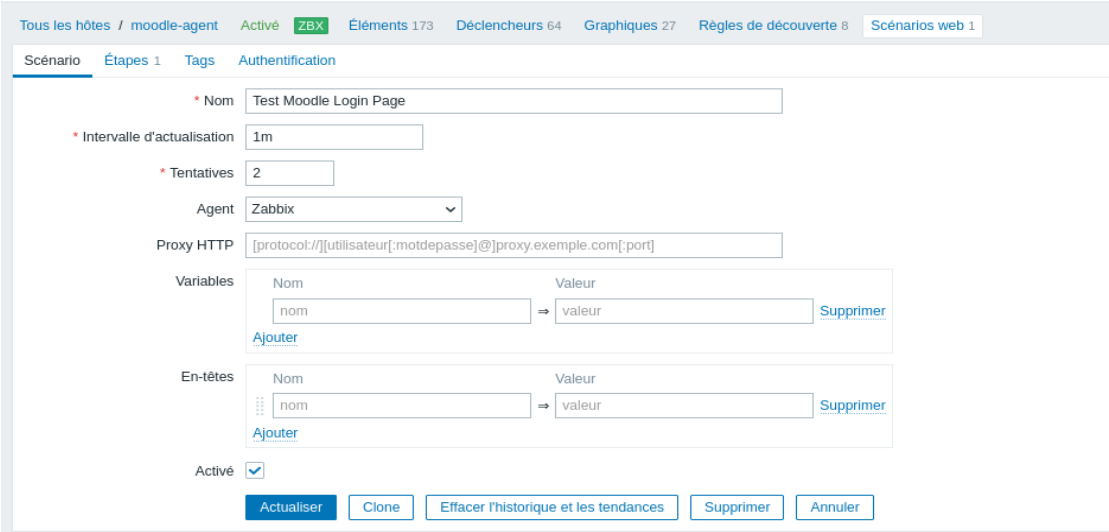
Screenshot 124 : Accès à la configuration d'un web scenarios

Etape 1 : Depuis l'interface principale de Zabbix, naviguer dans le menu latéral gauche vers **Surveillance** > **Hôtes > moodle-agent > web.**



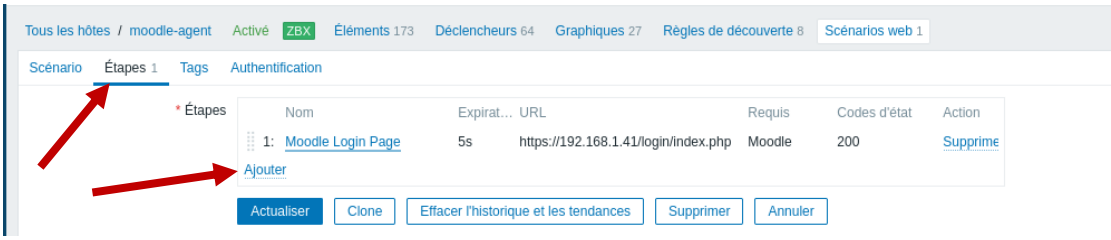
Screenshot 125 : Création d'un scenario web

Etape 2 : Cliquer sur **créer un scénario web** situé en haut à droit de la page.



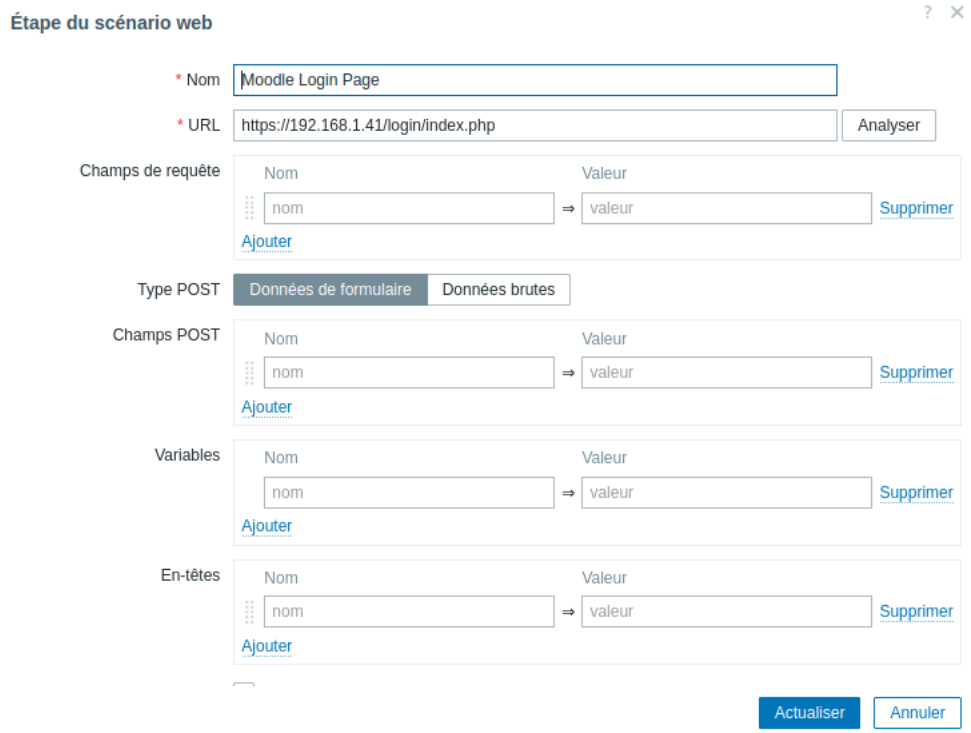
Screenshot 126 : Configuration du scénario web

Etape 3 : Dans le champ **Nom** indiquer le nom du scénario (**Test Moodle Login Page**). **Intervalle d'actualisation** (**1m**) cela signifie que le scénario s'exécute tous les 60s. Dans le champ **Tentatives**, définir le nombre de tentatives que le scenario s'exécute avant déclarer échec définitif.



Screenshot 127 : ajout d'une étape de scenario web

Etape 4 : Cliquer sur l'onglet Etapes, puis sur le lien **ajouter en bleu** sous le tableau.



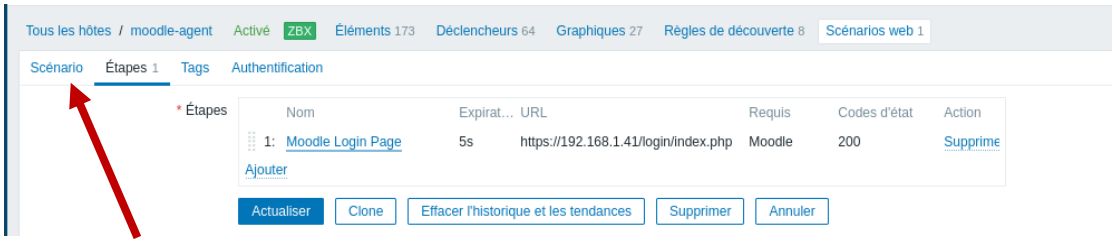
Screenshot 128 : Configuration d'une étape de scenario web



Screenshot 129 : Configuration d'une étape de scenario web (suite)

Etape 5 : Le champ **NOM** Identifie cette étape spécifique dans le scénario. **URL** (https://192.168.1.41/login/index.php) définit le **protocole** utilisé, **l'adresse IP** et le **chemin** de la page e connexion Moodle. Cocher **suivre les redirections.** Sélectionner Corps pour récupérer le contenu HTML de la page. Dans le champ **Expiration** définir le timeout maximum(15s) pour cette étape. C'est-à-dire qu'après 15 secondes sans réponse c'est échec. Le champ **code d'état requis** (200) indique que la page est chargée avec succès.

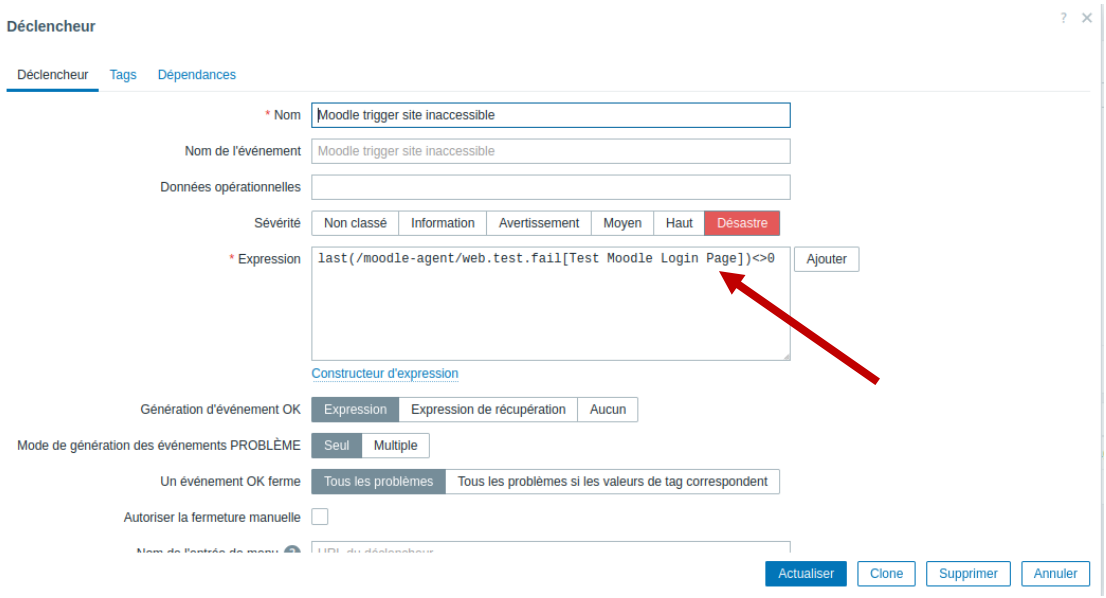
Etape 6 : Cliquer sur le bouton **Ajouter** pour enregistrer l'étape créée.



Screenshot 127 : ajout d'une étape de scenario web

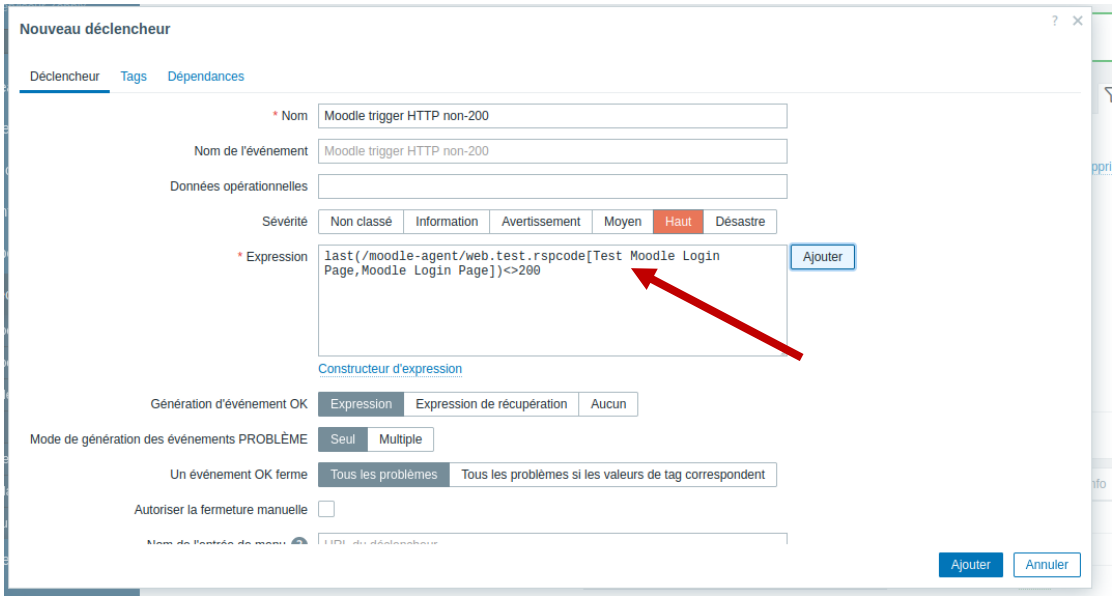
Etape 7 : Retour sur l'onglet **Scénario** et Cliquer sur le bouton **ajouter en bleu** en bas de la page.

En testant l'accessibilité de la page de connexion Moodle toutes les minutes avec une validation du contenu (présence du mot **Moodle**) et du code HTTP (**200**), ce scénario garantit une détection rapide de toute indisponibilité ou dysfonctionnement de la plateforme pédagogique.



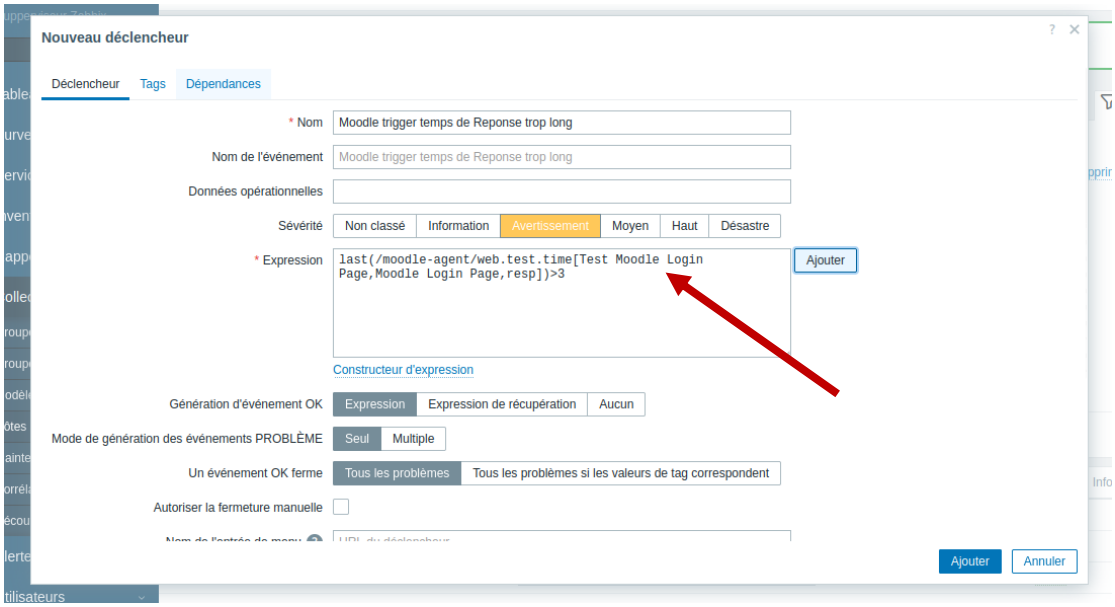
Screenshot 130 : Création déclencheur Moodle inaccessible

Ce déclencheur détecte l'**indisponibilité totale** du site Moodle lorsque le web scenario échoue complètement à récupérer la page, indiquant une panne majeure de l'application web.



Screenshot 131 : Création du déclencheur échec de la charge de la page http

Ce déclencheur détecte les erreurs **HTTP** retournées par le serveur Moodle (codes **404**, **500**) indiquant que le serveur répond mais renvoie une erreur au lieu de la page demandée. Moins grave qu'une indisponibilité totale, mais signale un dysfonctionnement applicatif.



Screenshot 132 : Création du déclencheur retard de réponse du site Moodle

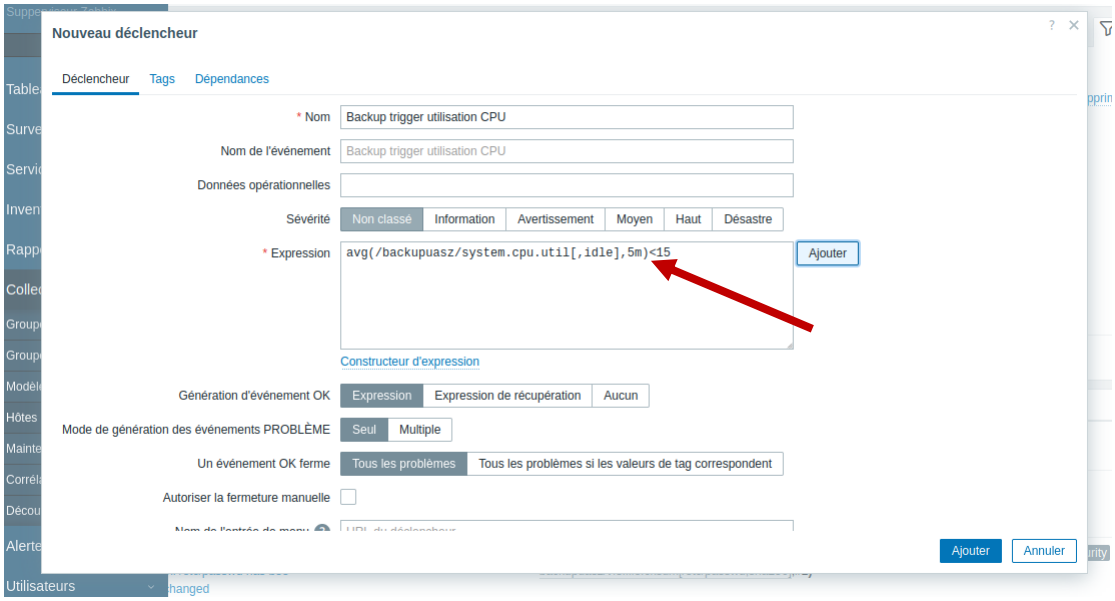
Ce déclencheur détecte les dégradations de performance du site Moodle en surveillant le temps de réponse de la page. Même si le site est accessible et retourne code 200, un temps de réponse excessif indique des problèmes de performance impactant l'expérience utilisateur.

Les trois triggers basés sur le Web Scenario forment un **système de surveillance défense en profondeur** de la disponibilité et de la performance de Moodle. En détectant successivement l'indisponibilité totale (site inaccessible), les erreurs applicatives (HTTP non-200), et les dégradations de performance (temps de réponse), cette approche permet des **réponses adaptées** selon la gravité du problème.

Dans le contexte de l'UASZ, où Moodle supporte quotidiennement des centaines d'utilisateurs pour des activités académiques critiques, cette surveillance orientée **expérience utilisateur** garantit non seulement que l'infrastructure technique fonctionne, mais que le **service pédagogique est réellement accessible et performant**.

4.3.9 Supervision du serveur backup (borgbackup)

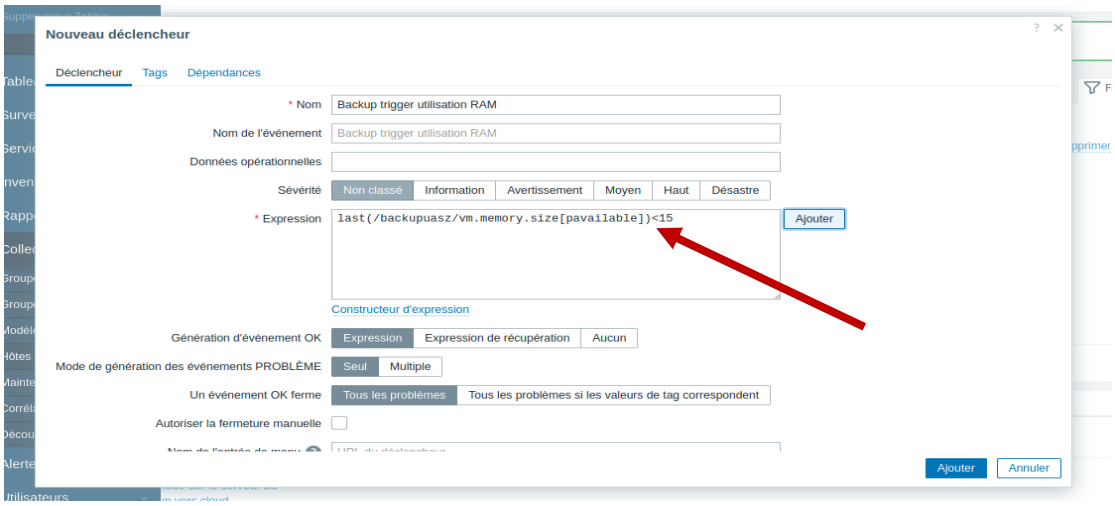
a) Surveillance de l'utilisation du Processeur (CPU)



Screenshot 145 : Configuration du déclencheur de l'utilisation du Processeur (CPU)

Le trigger **Backup trigger utilisation CPU** surveille la charge processeur du serveur de sauvegardes de l'UASZ avec l'expression **avg(/backupuasz/system.cpu.util[,idle],5m)<15,** déclenchant une alerte lorsque l'utilisation CPU dépasse 85% sur une fenêtre de 5 minutes.

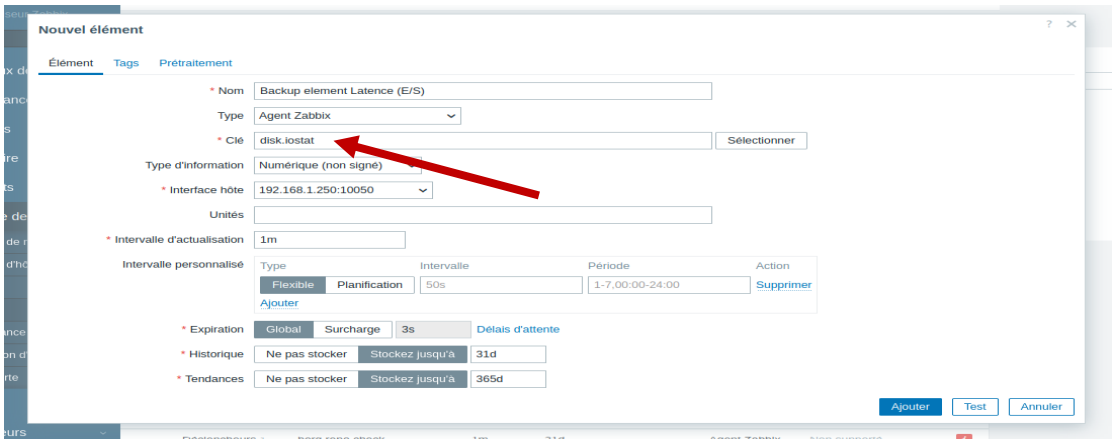
b) Surveillance de la disponibilité de la RAM



Screenshot 146 : Configuration déclencheur de la disponibilité de la RAM

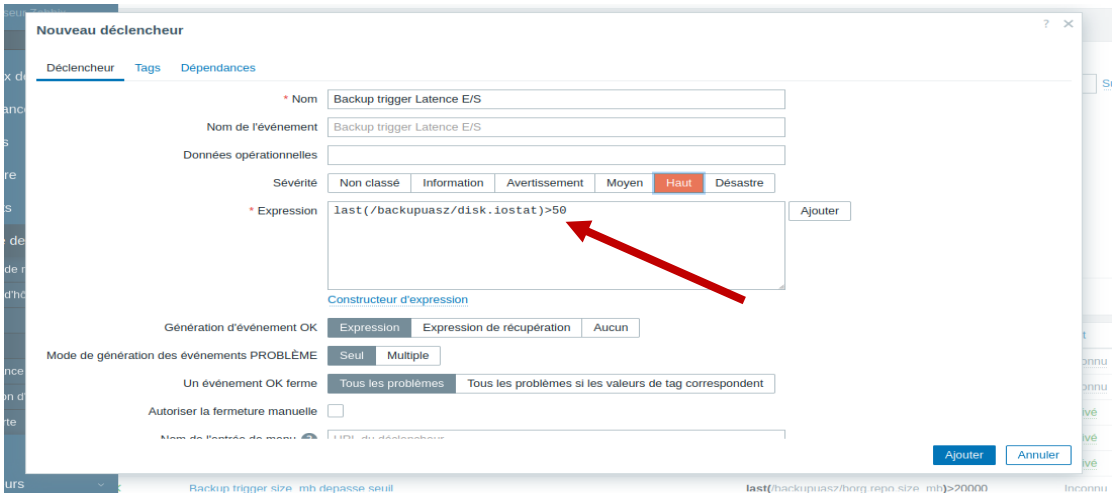
Ce déclencheur a été mis en place pour assurer une surveillance proactive de l'utilisation de la mémoire RAM sur le serveur backup. Nommé **Backup trigger utilisation RAM**, il évalue en temps réel la disponibilité de la mémoire via l'expression **last(/Backupussx/vm.memory.size[pavailable])<15**. Cette condition déclenche une alerte lorsque le pourcentage de mémoire disponible descend en dessous du seuil critique de 15%.

c) Surveillance des entrées et sorties



Screenshot 147 : Configuration de l'élément pour la latence réseau

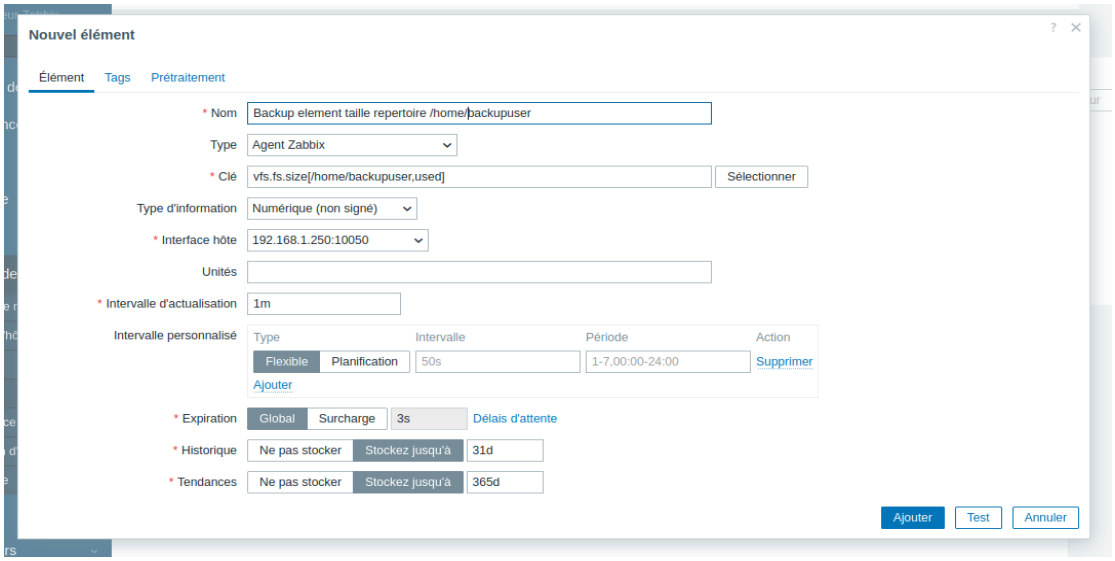
L'élément **Backup élément Latence (E/S)** a été configuré pour collecter des métriques critiques concernant la latence des opérations d'entrée/sortie sur le disque. Cette configuration est essentielle pour identifier rapidement les dégradations de performance qui pourraient impacter les opérations de sauvegarde, puisque des latences élevées sur les E/S peuvent significativement prolonger la durée des processus de backup et affecter la fenêtre de sauvegarde.



Screenshot 148 : Configuration du déclencheur de surveillance de la latence E/S

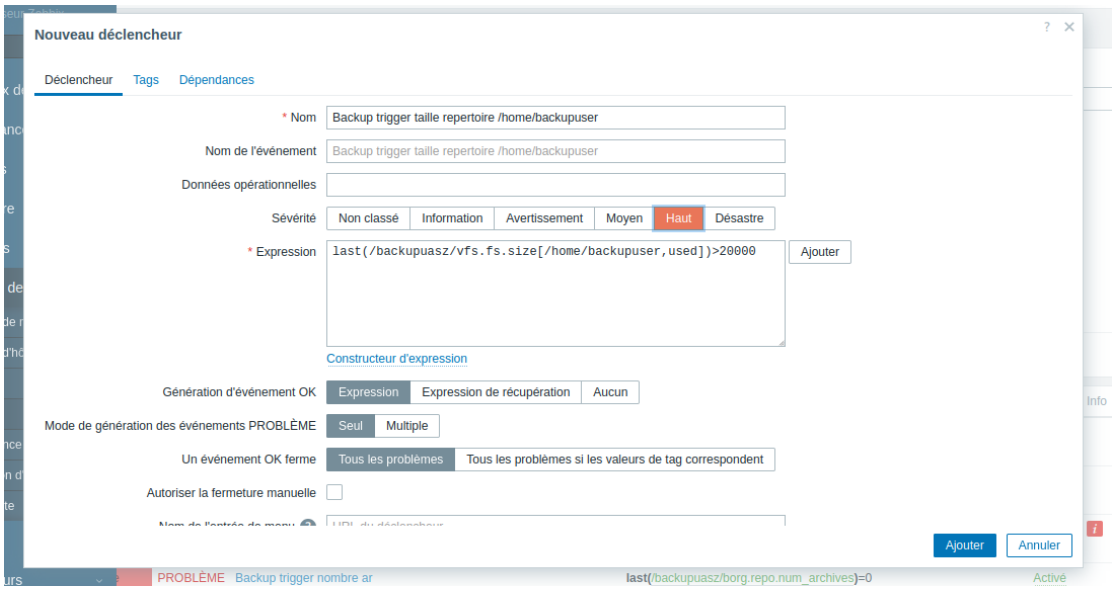
Le déclencheur **Backup trigger Latence E/S** permet de déclencher une alerte lorsque la latence dépasse le seuil de 50 millisecondes.

d) Surveillance du répertoire /home/backupuser (repository)



Screenshot 149 : Configuration de l'élément pour le répertoire de sauvegarde (/home/backupuser)

L'élément **Backup élément taille répertoire /home/backupuser** a été créé pour monitorer en temps réel l'utilisation de l'espace disque du répertoire dédié aux sauvegardes. Cet élément collecte des valeurs représentant l'espace disque consommé dans le répertoire /home/backupuser.



Screenshot 150 : Configuration du déclencheur pour le répertoire de sauvegarde (/home/backupuser)

Le déclencheur **Backup trigger taille répertoire /home/backupuser** représente un dispositif d'alerte critique pour la gestion de l'espace de stockage dédié aux sauvegardes. Il surveille en permanence l'occupation du répertoire /home/backupuser via l'expression **last(/backupussx/vfs.fs.size[/home/backupuser,used])>20000.**
Cette condition déclenche une alerte lorsque l'espace utilisé dépasse 20 000 unités (généralement exprimées en mégaoctets, soit environ 20 Go). Ce seuil a été établi pour anticiper les risques de saturation du système de fichiers qui pourraient entraîner l'échec des sauvegardes planifiées ou l'impossibilité de stocker de nouvelles données.

4.3.10 Supervision des sauvegardes et synchronisations

La supervision des processus de sauvegarde et de synchronisation constitue un élément clé du système mis en place. Elle permet de vérifier non seulement l'exécution correcte des sauvegardes, mais également leur régularité, leur durée et leur volume. Cette supervision concerne deux flux principaux :

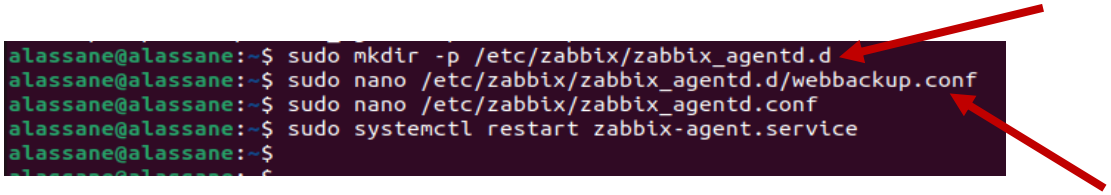
- La sauvegarde des données du serveur Moodle vers le serveur Backup à l'aide de BorgBackup ;
- La synchronisation du dépôt Borg vers le cloud via l'outil rclone.

À la fin de chaque script de sauvegarde (Moodle → Backup) et de synchronisation (Backup → Cloud), un fichier d'état est généré dans le répertoire **/var/run/**.
Ce fichier contient des informations essentielles sur l'exécution du processus, telles que :

- Le statut de la sauvegarde (succès ou échec),
- La date et l'heure de la dernière exécution,
- La durée totale du processus,
- La taille des données sauvegardées

Exemple de fichier : /var/run/web_backup_status.env

A) La sauvegarde du serveur Moodle vers le serveur Backup



Screenshot 133 : Création et configuration du fichier de supervision des sauvegardes dans Zabbix Agent (sur le serveur Moodle)

Dans un premier temps, le répertoire /etc/zabbix/zabbix_agentd.d est créé afin d'héberger les fichiers de paramètres utilisateurs (*UserParameters*). Ensuite, le fichier webbackup.conf est créé et édité.

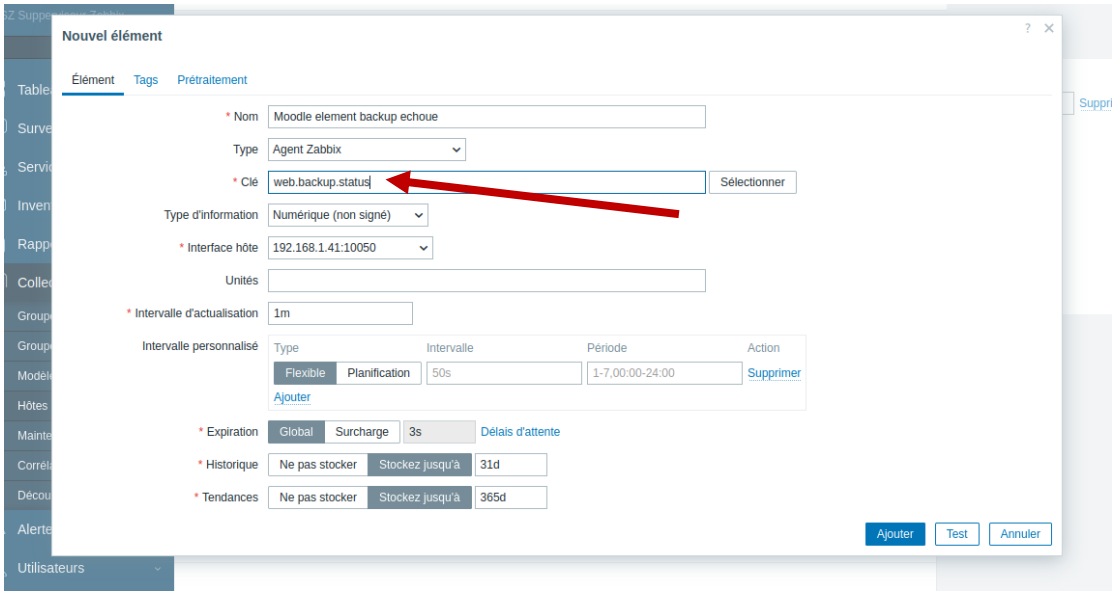
```
alassane@alassane:~$ sudo cat /etc/zabbix/zabbix_agentd.d/webbackup.conf
UserParameter=web.backup.status,grep LAST_WEB_BACKUP_STATUS /var/run/web_backup_status.env | cut -d= -f2
UserParameter=web.backup.time,grep LAST_WEB_BACKUP_TIME /var/run/web_backup_status.env | cut -d= -f2
UserParameter=web.backup.duration,grep LAST_WEB_BACKUP_DURATION /var/run/web_backup_status.env | cut -d= -f2
UserParameter=web.backup.db.size,grep LAST_DB_DUMP_MB /var/run/web_backup_status.env | cut -d= -f2
UserParameter=web.backup.mdata.size,grep LAST_MOODLEDATA_TAR_MB /var/run/web_backup_status.env | cut -d= -f2
alassane@alassane:~$
```

Screenshot 134 : Définition des UserParameters pour la supervision des sauvegardes (sur le serveur Moodle)

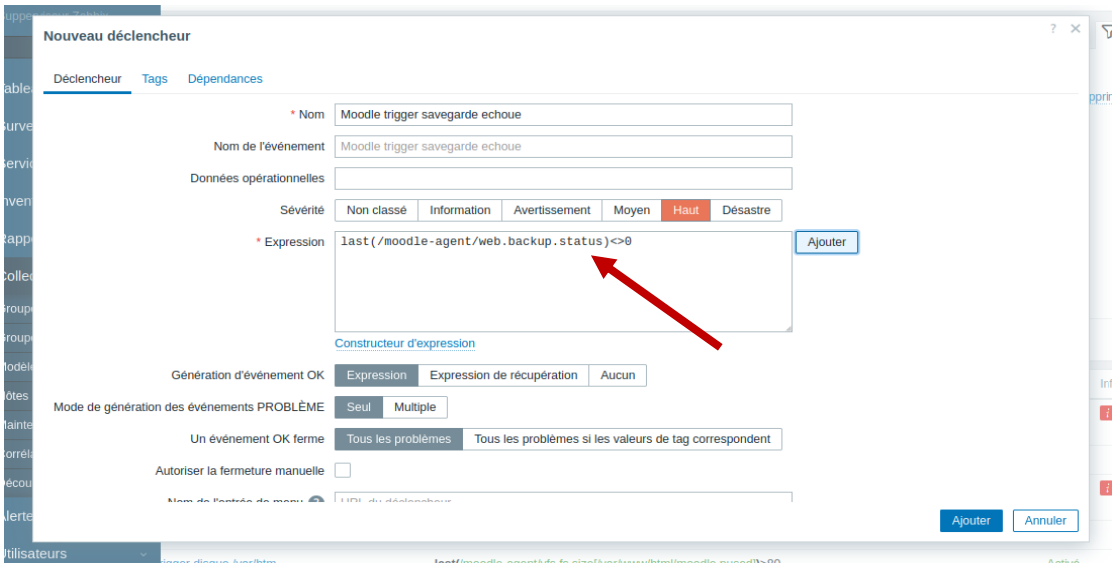
Cette capture présente le contenu du fichier webbackup.conf, qui définit plusieurs **UserParameters** utilisés par Zabbix pour superviser les processus de sauvegarde.
Chaque paramètre permet d'extraire une information précise depuis le fichier **/var/run/web_backup_status.env**, généré automatiquement par le script de sauvegarde.

- **web.backup.status** : permet de vérifier le statut global de la dernière sauvegarde (succès ou échec).
- **web.backup.time** : récupère la date et l'heure de la dernière sauvegarde exécutée.
- **web.backup.duration** : indique la durée totale d'exécution de la sauvegarde.
- **web.backup.db.size** : fournit la taille du dump de la base de données Moodle.
- **web.backup.mdata.size** : renseigne la taille de l'archive contenant les données Moodle.

Ces paramètres personnalisés permettent une supervision fine et automatisée des sauvegardes, facilitant la détection rapide des anomalies et garantissant la fiabilité du système de sauvegarde.

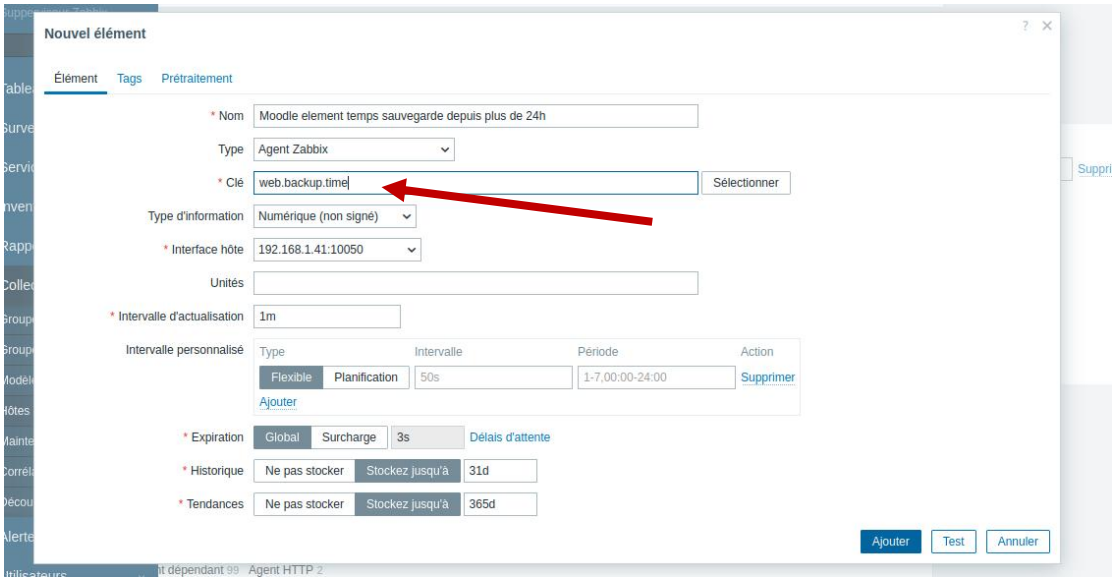


Screenshot 135 : Configuration de l'élément de vérification de l'état de sauvegarde vers le serveur backup



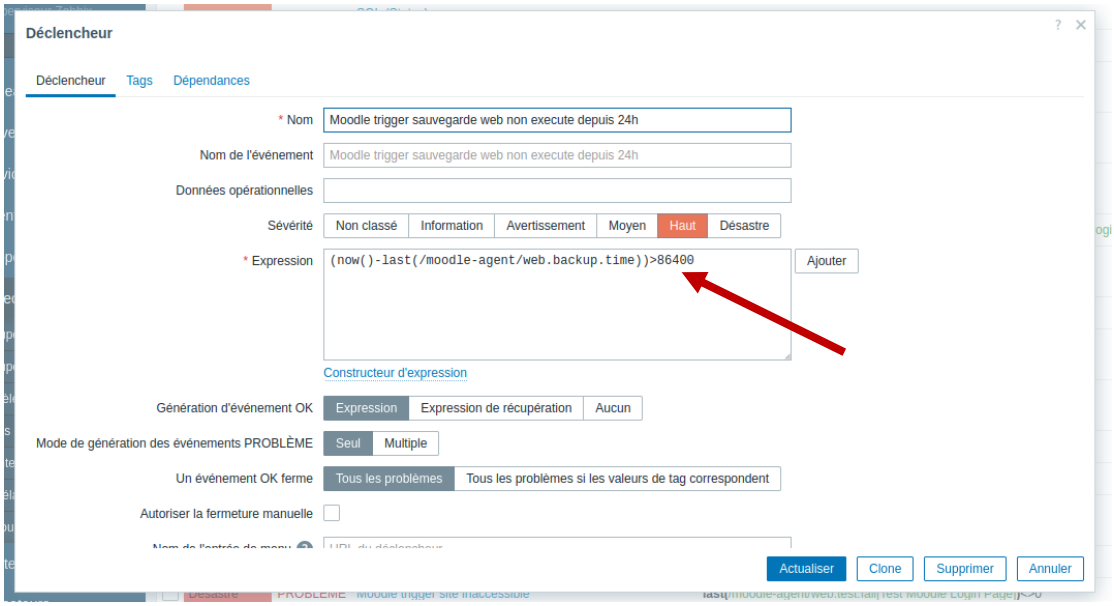
Screenshot 136 : Configuration du déclencheur d'échec de sauvegarde vers le serveur backup

Ce déclencheur génère une alerte automatique lorsque la dernière sauvegarde Moodle échoue, permettant une intervention rapide avant que l'absence de backup ne devienne critique (par exemple, avant un incident nécessitant restauration).



Screenshot 137 : Configuration de l'élément de vérification de la dernière sauvegarde vers le serveur backup

Cet élément surveille **l'âge du dernier fichier de sauvegarde** en mesurant le temps écoulé depuis sa dernière modification. Il permet de détecter si la sauvegarde quotidienne ne s'est pas exécutée, même en l'absence d'erreur explicite.



Screenshot 138 : Configuration du déclencheur de retard de sauvegarde vers le serveur backup

Ce déclencheur génère une alerte lorsque le dernier fichier de sauvegarde n'a pas été modifié depuis plus de 24 heures, indiquant que la sauvegarde quotidienne ne s'est probablement pas exécutée. Contrairement au déclencheur d'échec, il détecte l'absence d'exécution plutôt qu'un échec lors de l'exécution.

B) La synchronisation du dépôt Borg vers le cloud via rclone.

Procéder la même chose pour la création et la configuration du fichier de supervision des synchronisations vers le cloud sur le serveur de backup.

```
adminbackup@backupuasz:~$ sudo mkdir -p /etc/zabbix/zabbix_agentd.d
adminbackup@backupuasz:~$ sudo nano /etc/zabbix/zabbix_agentd.d/borg.conf
adminbackup@backupuasz:~$ sudo systemctl restart zabbix-agent.service
adminbackup@backupuasz:~$ sudo systemctl enable zabbix-agent.service
Synchronizing state of zabbix-agent.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable zabbix-agent
```

Screenshot 139 : Création et configuration du fichier de supervision des synchronisations vers le cloud (cloudsync.conf) dans Zabbix Agent

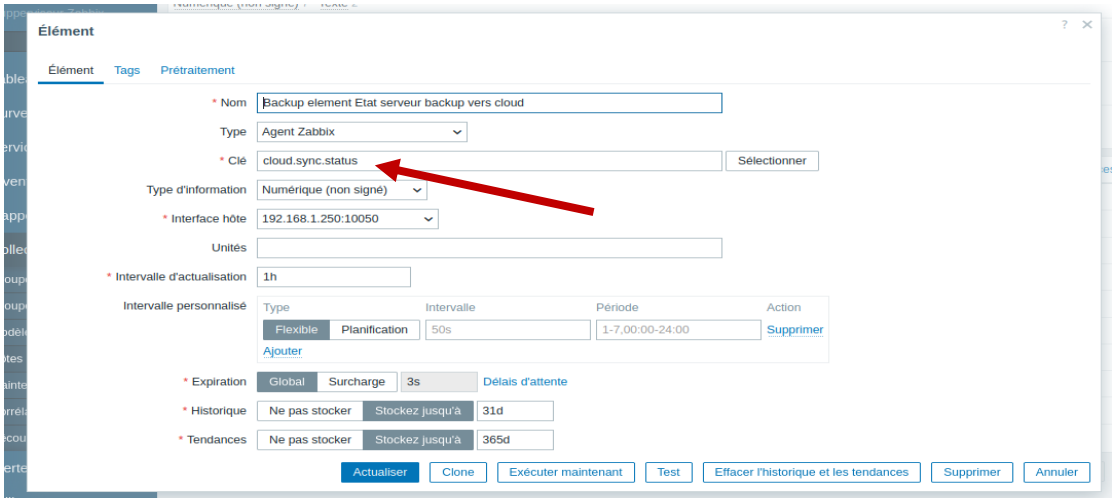
```
adminbackup@backupuasz:~$ sudo cat /etc/zabbix/zabbix_agentd.d/cloudsync.conf
UserParameter=cloud.sync.status,awk -F'=' '/SYNC_STATUS/{print $2}' /var/run/cloud_sync_status.env
UserParameter=cloud.sync.time,awk -F'=' '/SYNC_TIME/{print $2}' /var/run/cloud_sync_status.env
UserParameter=cloud.sync.duration,awk -F'=' '/SYNC_DURATION/{print $2}' /var/run/cloud_sync_status.env
UserParameter=cloud.sync.size,awk -F'=' '/SYNC_SIZE_MB/{print $2}' /var/run/cloud_sync_status.env
```

Screenshot 140 : Définition des UserParameters pour la supervision des synchronisations vers le cloud (sur le serveur de backup)

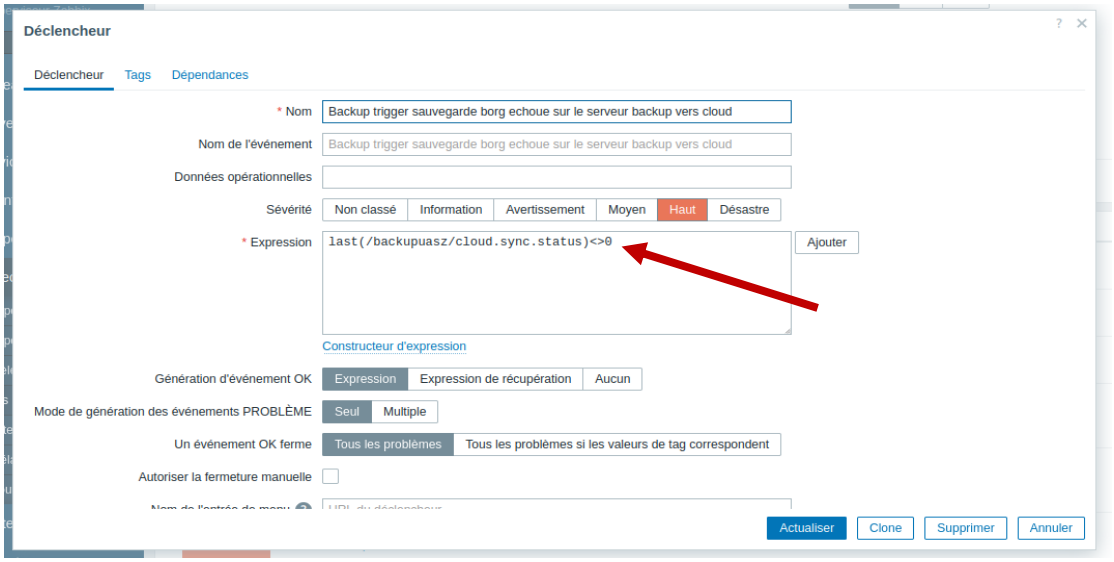
Le fichier **cloudsync.conf** définit des **UserParameters personnalisés** permettant à l'agent Zabbix de collecter des métriques spécifiques sur un **processus de synchronisation cloud.**

Rôle des UserParameters :

Les **UserParameters** étendent les capacités natives de l'agent Zabbix en permettant l'exécution de commandes shell personnalisées dont les résultats sont collectés comme items Zabbix. Ils sont essentiels pour surveiller des applications ou processus spécifiques non couverts par les items Zabbix standard.

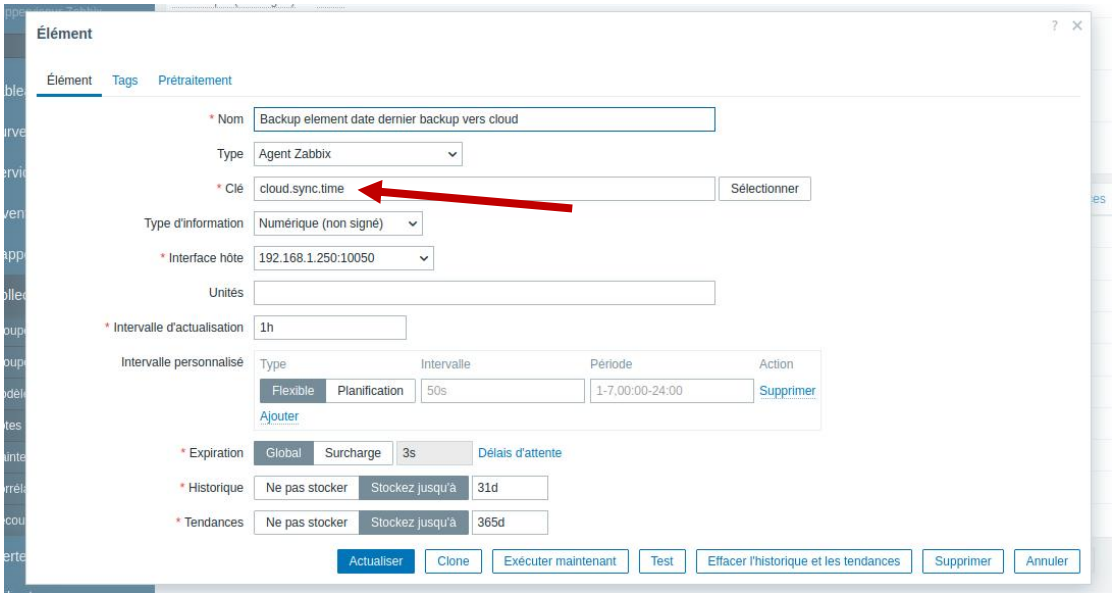


Screenshot 141 : Configuration de l'élément de vérification de l'état de synchronisation vers le cloud

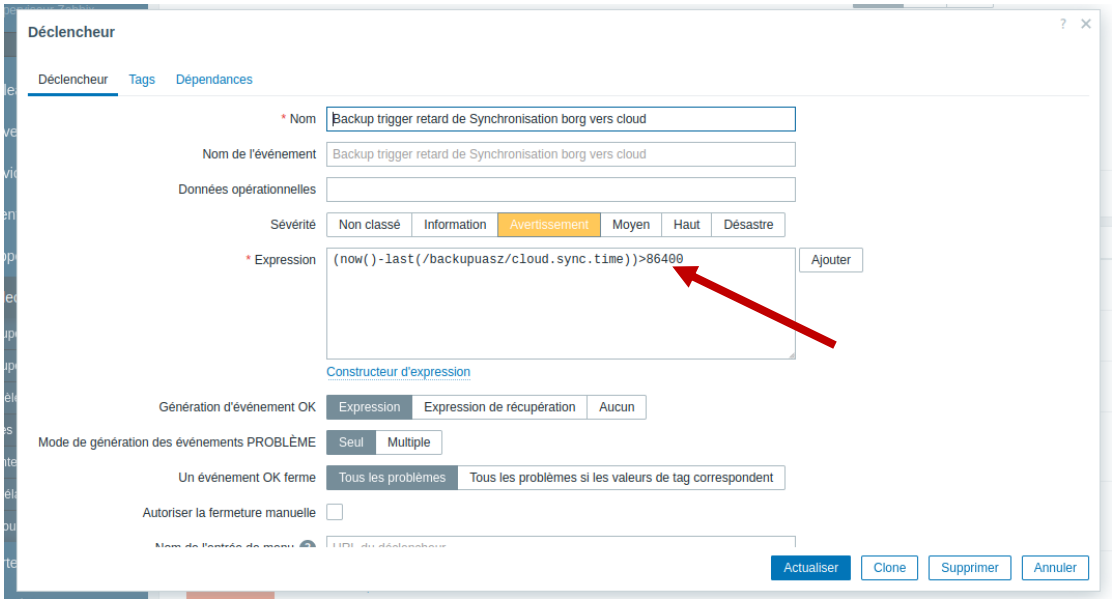


Screenshot 142 : Configuration du déclencheur d'échec de synchronisation vers le cloud

Ce déclencheur génère une alerte automatique lorsqu'il y a échec de synchronisation vers le cloud.



Screenshot 143 : Configuration de l'élément de vérification de la dernière synchronisation vers le cloud



Screenshot 144 : Configuration du déclencheur de retard de synchronisation vers le cloud

Ce déclencheur génère une alerte automatique lorsque la synchronisation vers le cloud est en retard de plus de 24h.